A Survey on Metamorphic Testing

Sergio Segura, *Member, IEEE*, Gordon Fraser, *Member, IEEE*, Ana B. Sanchez, and Antonio Ruiz-Cortés

Abstract—A test oracle determines whether a test execution reveals a fault, often by comparing the observed program output to the expected output. This is not always practical, for example when a program's input-output relation is complex and difficult to capture formally. *Metamorphic testing* provides an alternative, where correctness is not determined by checking an individual concrete output, but by applying a transformation to a test input and observing how the program output "morphs" into a different one as a result. Since the introduction of such *metamorphic relations* in 1998, many contributions on metamorphic testing have been made, and the technique has seen successful applications in a variety of domains, ranging from web services to computer graphics. This article provides a comprehensive survey on metamorphic testing: It summarises the research results and application areas, and analyses common practice in empirical studies of metamorphic testing as well as the main open challenges.

Index Terms—Metamorphic testing, oracle problem, survey

1 Introduction

Software testing is an essential but costly activity applied during software development to detect faults in programs. Testing consists of executing a program with test inputs, and to detect faults there needs to be some procedure by which testers can decide whether the output of the program is correct or not, a so-called *test oracle* [1]. Often, the test oracle consists of comparing an expected output value with the observed output, but this may not always be feasible. For example, consider programs that produce complex output, like complicated numerical simulations, or code generated by a compiler—predicting the correct output for a given input and then comparing it with the observed output may be non-trivial and error-prone. This problem is referred to as the *oracle problem* and it is recognised as one of the fundamental challenges of software testing [1], [2], [3], [4].

Metamorphic testing [5] is a technique conceived to alleviate the oracle problem. It is based on the idea that often it is simpler to reason about relations between outputs of a program, than it is to fully understand or formalise its input-output behaviour. The prototypical example is that of a program that computes the sine function: What is the exact value of $\sin(12)$? Is an observed output of -0.5365 correct? A mathematical property of the sine function states that $\sin(x) = \sin(\pi - x)$, and we can use this to test whether $\sin(12) = \sin(\pi - 12)$ without knowing the concrete values of either sine calculation. This is an example of a *metamorphic relation*: an input transformation that can be used to generate new test cases from existing test data, and an output relation, that compares the outputs produced by a pair of test cases. Metamorphic testing does not only alleviate the oracle problem, but it can also be highly automated.

The introduction of metamorphic testing can be traced back to a technical report by Chen et al. [5] published in 1998. However, the use of identity relations to check program outputs can be found in earlier articles on testing of numerical programs [6], [7] and fault tolerance [8]. Since its introduction, the literature on metamorphic testing has flourished with numerous techniques, applications and assessment studies that have not been fully reviewed until now. Although some papers present overviews of metamorphic testing, they are usually the result of the authors' own experience [9], [10], [11], [12], [13], review of selected articles [14], [15], [16] or surveys on related testing topics [3]. At the time of writing this article, the only known survey on metamorphic testing is written in Chinese and was published in 2009[1] [17]. As a result, publications on metamorphic testing remain scattered in the literature, and this hinders the analysis of the state of the art and the identification of new research directions.

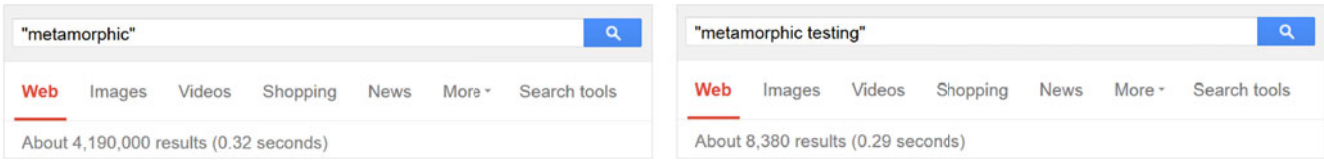
In this article, we present an exhaustive survey on metamorphic testing, covering 119 papers published between 1998 and 2015. To provide researchers and practitioners with an entry point, Section 2 contains an introduction to metamorphic testing. All papers were carefully reviewed and classified, and the review methodology followed in our survey as well as a brief summary and analysis of the selected papers are detailed in Section 3. We summarise the state of the art by capturing the main advances on metamorphic testing in Section 4. Across all surveyed papers, we identified more than 12 different application areas, ranging from web services through simulation and modelling to computer graphics (Section 5). Of particular interest for researchers is a detailed analysis of experimental studies and evaluation metrics (Section 6). As a result of our survey, a number of research challenges emerge, providing avenues for future research (Section 7); in particular, there are open questions on how to derive effective metamorphic relations, as well as how to reduce the costs of testing with them.

- *S. Segura, A.B. Sánchez, and A. Ruiz-Cortés are with the Department of Computer Languages and Systems, Universidad de Sevilla, Spain. E-mail: {sergiosegura, anabsanchez, aruiz}@us.es.*
- *G. Fraser is with the Department of Computer Science, University of Sheffield, Sheffield, United Kingdom. E-mail: gordon.fraser@sheffield.ac.uk.*

Manuscript received 11 July 2015; revised 9 Feb. 2016; accepted 14 Feb. 2016. Date of publication 28 Feb. 2016; date of current version 23 Sept. 2016. Recommended for acceptance by P. Tonella.
For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.
Digital Object Identifier no. 10.1109/TSE.2016.2532875

1. Note that 86 out of the 119 papers reviewed in our survey were published in 2009 or later.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.
0098-5589 © 2016 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.



(a) Source test case. (b) Follow-up test case.

Fig. 1. Metamorphic test on the Google search engine checking the relation $Count(q) \geq Count(q+k)$.

2 Metamorphic Testing

Identity relations are a well-known concept in testing, and have been used even before the introduction of metamorphic relations. For example, Blum et al. [7] checked whether numerical programs satisfy identity relations such as $P(x) = P(x_1) + P(x_2)$ for random values of x_1 and x_2. In the context of fault tolerance, the technique of data diversity [8] runs the program on re-expressed forms of the original input; e.g., $\sin(x) = \sin(a) \times \sin(\pi/2 - b) + \sin(\pi/2 - a) \times \sin(b)$ where $a + b = x$. The concept of metamorphic testing, introduced by Chen [5] in 1998, generalises these ideas from identity relations to any type of relation, such as equalities, inequalities, periodicity properties, convergence constraints, subsumption relationships and many others. In general, a metamorphic relation for a function f is expressed as a relation among a series of function inputs $x_1, x_2, \ldots, x_n$ (with $n > 1$), and their corresponding output values $f(x_1), f(x_2), \ldots, f(x_n)$ [18]. For instance, for the sine example from the introduction the relation between x_1 and x_2 would be $\pi - x_1 = x_2$, and the relation between $f(x_1)$ and $f(x_2)$ would be equality, i.e.,:

$$R = \{(x_1, x_2, \sin x_1, \sin x_2) \mid \pi - x_1 = x_2 \rightarrow \sin x_1 = \sin x_2\}.$$

This resembles the traditional concept of program invariants, which are properties (for example expressed as assert statements) that hold at certain points in programs [19]. However, the key difference is that an invariant has to hold for every possible program execution, whereas a metamorphic relation is a relation between *different* executions. A relation between two executions implicitly defines how, given an existing source test case (x_1), one has to transform this into a follow-up test case (x_2), such that an abstract relation R (e.g., $\sin x_1 = \sin x_2$) can be checked on the inputs represented by x_1 and x_2, as well as the outputs produced by executing x_1 and x_2. The term metamorphic relation presumably refers to this "metamorphosis" of test inputs and outputs. If the relation R does not hold on a pair of source and follow-up test cases x_1 and x_2, then a fault has been detected. In this article, we use the term *metamorphic test case* to refer to a pair of a source test case and its follow-up test case.

The basic process for the application of metamorphic testing can be summarised as follows:

1) *Construction of metamorphic relations.* Identify necessary properties of the program under test and represent them as metamorphic relations among multiple test case inputs and their expected outputs, together with some method to generate a follow-up test case based on a source test case. Note that metamorphic relations may be associated with preconditions that restrict the source test cases to which they can be applied.
2) *Generation of source test cases.* Generate or select a set of source test cases for the program under test using any traditional testing technique (e.g., random testing).
3) *Execution of metamorphic test cases.* Use the metamorphic relations to generate follow-up test cases, execute source and follow-up test cases, and check the relations. If the outputs of a source test case and its follow-up test case violate the metamorphic relation, the metamorphic test case is said to have failed, indicating that the program under test contains a bug.

As an illustrative example, consider a program that computes the shortest path between a source vertex s and destination vertex d in a graph G, $SP(G, s, d)$. A metamorphic relation of the program is that if the source and destination vertices are swapped, the length of the shortest path should be equal: $|SP(G, s, d)| = |SP(G, d, s)|$. Suppose that a source test case (G, a, b) is selected according to some testing method (e.g., randomly). Based on the metamorphic relation, we can now easily generate a new follow–up test case by swapping the source and destination vertices (G, b, a). After executing the program with both test cases, their outputs can be checked against the relation to confirm whether it is satisfied or not, i.e., whether the outputs are equal. If the metamorphic relation is violated, it can be concluded that the metamorphic test has failed and the program is faulty.

As a further example, consider testing an online search engine such as Google or Yahoo [20]. Let $Count(q)$ be the number of results returned for a search query q. Intuitively, the number of returned results for q should be greater or equal than that obtained when refining the search with another keyword k. This can be expressed as the following metamorphic relation: $Count(q) \geq Count(q + k)$, where $+$ denotes the concatenation of two keywords. Fig. 1 illustrates the application of this metamorphic relation on Google. Consider a source test case consisting in a search for the keyword "metamorphic", resulting in "*About*" 4.2M results. Suppose that a follow-up test case is constructed by searching for the keywords "metamorphic testing": This leads to 8,380 results which is less than the result for "metamorphic", and thus satisfies the relation. If more results were found, then that would violate the metamorphic relation, revealing a bug in the system.

If source test cases are generated automatically, then metamorphic testing enables full test automation, i.e., input generation and output checking. In the sine example presented in Section 1, for instance, metamorphic testing could be used together with random testing to automatically generate random source test cases (x) and their respective follow-up test cases ($\pi - x$), until a pair is found that violates the metamorphic relation, or a maximum time out is

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.

reached. Similarly, in the search engine example, metamorphic testing could also be used together with a random word generator to automatically construct source test cases (e.g., "algorithm") and their respective follow-up test cases (e.g., "algorithm colour") until a pair that reveals a bug is found, if any such pairs exists.

3 Review Method

To perform a survey on metamorphic testing we followed a systematic and structured method inspired by the guidelines of Kitchenham [21] and Webster et al. [22]. A similar approach was followed by some of the authors in the context of software product lines [23]. To report the results, we also took inspiration from recent surveys on related topics such as the oracle problem [3], search-based testing [24], automated test case generation [2] and mutation analysis [25]. Below, we detail the main data regarding the review process and its results.

3.1 Research Questions

The aim of this survey is to answer the following research questions on metamorphic testing:

*RQ*1: *What improvements to the technique have been made?*
*RQ*2: *What are its known application domains?*
*RQ*3: *How are experimental evaluations performed?*
*RQ*4: *What are the future research challenges?*

We propose RQ1 to obtain an in-depth view on metamorphic testing outlining the state of the art in terms of the main advances in the application of the technique since its original introduction. RQ2 is proposed to give an insight into the scope of metamorphic testing and its applicability to different domains including its integration with other testing techniques. We also want to know how different approaches of performing metamorphic testing are evaluated including the subject programs used, types of detected faults, evaluation metrics, and empirical studies involving humans. Finally, based on the answer to the previous questions, we expect to identify unresolved problems and research opportunities in response to RQ4.

3.2 Inclusion and Exclusion Criteria

We scrutinised the existing literature, looking for papers addressing any topic related to metamorphic testing, including methods, tools or guidelines for the application of the technique, applications to specific testing problems, empirical evaluations, and surveys. Articles of the same authors but with very similar content were intentionally classified and evaluated as separate contributions for a more rigorous analysis. Later, in the presentation of results, we grouped those articles with no major differences. We excluded PhD theses as well as those papers not related to the computer sciences field, not written in English, or not accessible on the Web.

3.3 Source Material and Search Strategy

The search for relevant papers was carried out in the online repositories of the main technical publishers, including ACM, Elsevier, IEEE, Springer and Wiley. We collected computer science papers published between January 1st 1998 (when Chen's report was published) and November 30th 2015 which have either "metamorphic test," "metamorphic testing," "metamorphic relation" or "metamorphic relations" in their title, abstract or keywords. We refer the reader to the technical report [26] which contains all data forms and query details. After a quick review of the results, we noticed that some articles on metamorphic testing with many citations were not among the candidate papers, including the technical report of Chen et al. [5] where the technique was introduced. To include those papers, we performed the search in the Google Scholar database, and additionally selected all papers with 5 or more citations published outside our target publication sources[2]. These were merged with our previous results, resulting in a final set of 362 candidate papers.

TABLE 1
Search Engines and Number of Primary Studies

Search engine	Primary studies
ACM digital library	15
Elsevier ScienceDirect	6
IEEEXplore digital library	65
Springer online library	13
Wiley InterScience	4
Google Scholar (+5 citations)	16
Total	**119**

Next, we examined the abstracts of the papers identified in the previous step and filtered them according to our inclusion and exclusion criteria, checking the content of the papers when unsure. This step was performed by two different authors who agreed on the results. The set of candidate papers was filtered to 116 publications within the scope of our survey. Then, we contacted the corresponding authors of the 116 selected papers and asked them to inform us about any missing papers within the scope of our search. Based on the feedback received, we included 3 new papers meeting our search criteria, except for the inclusion of the search terms in their title, abstract or keywords. As a result, the search was finally narrowed to *119 publications* that were in the scope of this survey. These papers are referred to as the *primary studies* [21]. Table 1 presents the number of primary studies retrieved from each source.

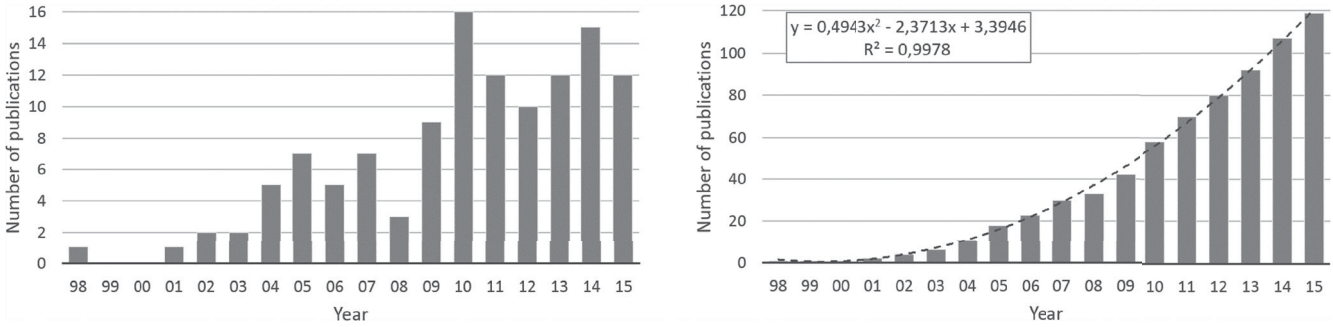
It is possible that our search has failed to find all papers since we focused on a subset of reputed publishers. However, we remain confident that the overall trends we report are accurate and provide a fair picture of the state of the art on metamorphic testing.

3.4 Data Collection

All 119 primary studies were carefully analysed to answer our research questions. For each study, we extracted the following information: full reference, brief summary, type of contribution (e.g., case study), application domains, integration with other testing techniques, number of metamorphic relations proposed, evaluation details, lessons learned and suggested challenges. To facilitate the process, we filled in a data extraction form for each primary study. All data forms were collected and published in a technical report [26].

2. The search was performed on December 30th, 2015.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.



(a) Number of publications per year.

(b) Cumulative number of publications per year.

Fig. 2. Metamorphic testing papers published between January 1st 1998 and November 30th 2015.

Primary studies were read at least twice by two different authors to reduce misunderstandings or missing information. As a sanity check, we contacted the corresponding author of each primary study and sent them the technical report to confirm that the information collected from their papers was correct. Some minor changes were proposed and corrected. We also asked them to inform us about any missing paper within the scope of our search as described in the previous section.

3.5 Summary of Results

The following sections summarise the primary studies in terms of publication trends, authors, venues, and research topics on metamorphic testing.

3.5.1 Publication Trends

Fig. 2a illustrates the number of publications on metamorphic testing published between January 1st 1998 and November 30th 2015. The graph shows a constant flow of papers on the topic since 2001, in particular from 2010 onwards. The cumulative number of publications is illustrated in Fig. 2b. We found a close fit to a quadratic function with a high determination coefficient ($R^2 = 0.997$), indicating a strong polynomial growth, a sign of continued health and interest in the subject. If the trend continues, there will be more than 170 metamorphic testing papers by 2018, two decades after the introduction of the technique.

3.5.2 Researchers and Organisations

We identified 183 distinct co-authors from 74 different organisations in the 119 primary studies under review. Table 2 presents the top authors on metamorphic testing and their most recent affiliation. Unsurprisingly, Prof. T. Y. Chen, with 44 papers, is the most prolific author on the topic.

3.5.3 Geographical Distribution of Publications

We related the geographical origin of each primary study to the affiliation country of its first co-author. Interestingly, we found that all 119 primary studies originated from only 11 different countries with Australia and China ahead, as presented in Table 3. By continents, 37 percent of the papers originated from Asia, 30 percent from Oceania, 19 percent from Europe and 14 percent from America. This suggests that the metamorphic testing community is formed by a modest number of countries but fairly distributed around the world.

3.5.4 Publication Venues

The 119 primary studies under review were published in 72 distinct venues. This means that the metamorphic testing literature is very dispersed, probably due to its applicability to multiple testing domains. Regarding the type of venue, most papers were presented at conferences and symposia (58 percent), followed by journals (23 percent), workshops (16 percent) and technical reports (3 percent). Table 4 lists the venues where at least three metamorphic testing papers have been presented.

3.5.5 Types of Contributions and Research Topics

Fig. 3a classifies the primary studies according to the type of contribution. We found that half of the papers present case

TABLE 2
Top 10 Co-Authors on Metamorphic Testing

Author	Institution	Papers
T. Y. Chen	Swinburne University of Technology	44
T. H. Tse	The University of Hong Kong	20
F.-C. Kuo	Swinburne University of Technology	17
Z. Q. Zhou	University of Wollongong	14
W. K. Chan	City University of Hong Kong	11
H. Liu	RMIT University	9
C. Murphy	Columbia University	9
G. Kaiser	Columbia University	8
X. Xie	Swinburne University of Technology	7
B. Xu	Nanjing University	7

TABLE 3
Geographical Distribution of Publications

Country	Papers
Australia	36
China	25
United States	17
Hong Kong	12
Germany	8
Spain	7
India	5
United Kingdom	3
Switzerland	3
Malaysia	2
France	1

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.

TABLE 4
Top Venues on Metamorphic Testing

Venue	Papers
Int Conference on Quality Software	9
Int Computer Software & Applications Conference	8
Int Workshop on Automation of Software Test	4
Int Conference on Software Engineering	4
IEEE Transactions on Software Engineering	4
Software Testing, Verification and Reliability	4
Int Conf on Software Testing, Verification and Validation	3
Information and Software Technology	3

studies (50 percent), followed by new techniques and methodologies (31 percent), and assessments and empirical studies (10 percent). We also found a miscellany of papers (7 percent) including related surveys, tutorial synopsis, and guidelines. Only two of the papers (2 percent) presented a tool as their main contribution.

A similar classification based on the main research topic is presented in Fig. 3b. Interestingly, we found that 49 percent of the papers report applications of metamorphic testing to different problem domains. The rest of papers address the construction of metamorphic relations (19 percent), integration with other testing techniques (10 percent), assessment of metamorphic testing (6 percent), execution of metamorphic test cases (5 percent) and generation of source test cases (4 percent). Finally, a few papers (7 percent) present brief overviews on the technique, its applications and research directions.

4 State of the Art in Metamorphic Testing

In this section, we address RQ1 by summarising the main contributions to metamorphic testing in the literature. First, we review the papers studying the properties of effective metamorphic relations. Then, approaches are classified according to the step they contribute to in the metamorphic testing process presented in Section 2, namely, construction of metamorphic relations, generation of source test cases, and execution of metamorphic test cases.

4.1 Properties of Good Metamorphic Relations

The effectiveness of metamorphic testing is highly dependent on the specific metamorphic relations that are used, and designing effective metamorphic relations is thus a critical step when applying metamorphic testing. For most problems, a variety of metamorphic relations with different fault-detection capability can be identified [9], [16], [18], [27], [28], [29], [30], [31], [32], [33], [34], [35]. Therefore, it is advisable to use a variety of diverse metamorphic relations to effectively test a given program. Several authors even suggest using as many metamorphic relations as possible during testing [28], [29], [36], [37]. However, because defining metamorphic relations can be difficult, it is important to know how to select the most effective ones. In this section, we review papers studying the properties that make metamorphic relations *good* at detecting faults.

Defining good metamorphic relations requires knowledge of the problem domain. Chen et al. [27] compared the effectiveness of metamorphic relations solely based on the theoretical knowledge of the problem (black-box) versus those derived from the program structure (white-box) using two case studies. They concluded that theoretical knowledge of the problem domain is not adequate for distinguishing good metamorphic relations. Instead, good metamorphic relations should be preferably selected with regard to the algorithm under test following a white-box approach. However, this was later disputed by Mayer and Guderlei [38], who studied six subject programs for matrix determinant computation with seeded faults. They concluded that metamorphic relations in the form of equalities or linear equations[3] as well as those close to the implementation strategy have limited effectiveness. Conversely, they reported that good metamorphic relations are usually strongly inspired by the semantics of the program under test. Other studies have also emphasised the knowledge of the problem domain as a requirement for the application of metamorphic testing [30], [39], [40].

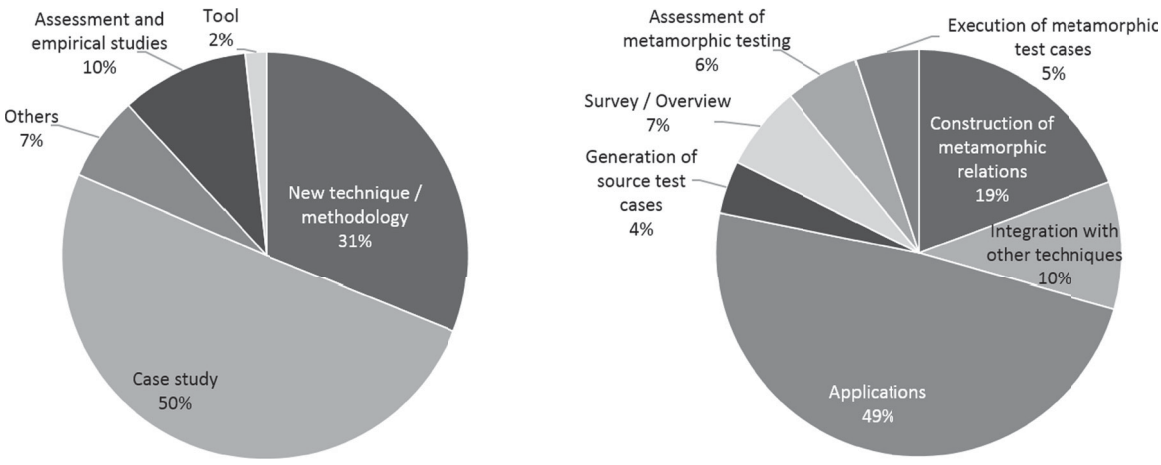
Metamorphic relations should make execution of the follow-up test case as different as possible from the source test case. Chen et al. [27] reported that good metamorphic relations are those that can make the execution of the source-test case as different as possible to its follow-up test case. They defined the "difference among executions" as any aspects of program runs (e.g., paths traversed). This observation has been confirmed by several later studies [9], [41], [42], [43], [44], [45]. In particular, Asrafi et al. [46] hypothesised that the higher the combined code coverage of the source and follow-up test cases, the more different are the executions, and the more effective is the metamorphic relation. Their study on two subject programs showed a strong correlation between coverage and fault-detection effectiveness in one of the two. In a similar study, Cao et al. [47] assessed the relation between fault-detection effectiveness of metamorphic relations and test case dissimilarity. An extensive experiment with 83 faulty programs and 7 distance metrics between the execution profiles of source and follow-up test cases revealed a strong and statistically significant correlation between the fault-detection capability of metamorphic relations and the distance among test cases, in particular when using branch coverage Manhattan distance [48].

Metamorphic relations derived from specific parts of the system are more effective than those targeting the whole system. Several authors have explored the applicability of metamorphic testing for integration testing with some helpful conclusions for the construction of good metamorphic relations. Just and Schweiggert [49], [50] assessed the applicability of metamorphic testing for system and integration testing in the context of an image encoder. Among other results, they concluded that the metamorphic relations derived from the components of a system are usually better at detecting faults than those metamorphic relations derived from the whole system. This finding was later confirmed by Xie et al. [51], who reported that metamorphic relations targeting specific parts of the program under test are easier to construct, more constrained, and therefore more effective in detecting faults than metamorphic relations at the system level.

Metamorphic relations should be formally described. Chan et al. [52] formally described metamorphic relations and

3. The authors literally refer to "equations with linear combinations on each side (with at least two terms on one of the sides)".

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.



(a) Type of contribution. (b) Research topic.

Fig. 3. Classification of primary studies by publication type and research topic.

metamorphic testing for a precise definition of the technique. Their formalisation was reused by several authors [29], [36] and later revised by Chan and Tse [12]. Hui and Huang [53] pointed out that most metamorphic relations in the literature are informally described using natural language, which makes them easily misunderstood, ambiguous and hard to reuse. The authors suggested that good metamorphic relations should be formally described and proposed a formal model for the rigorous description of metamorphic relations using predicate logic, inspired by the work of Chan et al. [52]. In particular, they proposed representing a metamorphic relation as a 3-tuple composed of i) relation between the inputs of source and follow-up test cases, ii) relation between the outputs of source and follow-up test cases, and iii) program function.

4.2 Construction of Metamorphic Relations

Constructing metamorphic relations is typically a manual task that demands thorough knowledge of the program under test. In this section, we review proposed alternative ways to create metamorphic relations, either by combining existing relations, or by generating them automatically.

Liu et al. [54] proposed a method named *Composition of Metamorphic Relations (CMR)* to construct new metamorphic relations by combining several existing relations. A similar idea had been superficially explored previously by Dong et al. [55]. The rationale behind this method is that the resulting relations should embed all properties of the original metamorphic relations, and thus they should provide similar effectiveness with a fewer number of metamorphic relations and test executions. Intuitively, Liu et al. defined two metamorphic relations as *"compositable"* if the follow-up test cases of one of the relations can always be used as source test case of the other. The composition is sensitive to the order of metamorphic relations and generalisable to any number of them. Determining whether two metamorphic relations are composable is a manual task. The results of a case study with a bioinformatics program processing an input matrix show that the composition of a set of metamorphic relations usually produces a composite relation with higher (or at least similar) fault-detection effectiveness than the original metamorphic relations, provided that all component relations have similar "tightness." The tightness of a relation determines how hard it is to satisfy it by mere chance—the tighter a relation is, the more difficult it is to satisfy it with some random outputs; e.g., $\sin(x) = \sin(\pi - x)$ is tighter than $\sin(x) \neq \sin(\pi - x/2)$. They also concluded that the CMR method delivers higher cost-effectiveness than classic metamorphic testing since it involves fewer test executions.

Kanewala and Bieman [56], [57] proposed a method that determines, given a predefined set of relations that they believe to hold for many numerical programs, which of these are exhibited by a given numerical program. Their method works by extracting a function's control flow graph and building a predictive model using machine learning techniques; i.e., it is a white-box method that requires static access to the source code. The approach was evaluated by constructing a prediction model using a code corpus of 48 mathematical functions with numerical inputs and outputs. The model was designed to predict three specific types of metamorphic relations: permutative, additive and inclusive [58]. In addition, they checked the fault–detection effectiveness of the predictive metamorphic relations using seeded faults. The results revealed that 66 percent of the faults (655 out of 988) were detected by the predicted metamorphic relations. In later work [59], the authors extended their method using graph kernels, which provide various ways of measuring similarity among graphs. The intuition behind their approach was that functions that have similar control flow and data dependency graphs may have similar metamorphic relations. Empirical results on the prediction of six different types of metamorphic relations on a corpus of 100 numerical programs revealed that graph kernels lead to higher prediction accuracy.

Zhang et al. [60] proposed a search-based approach for the inference of polynomial metamorphic relations. More specifically, the algorithm searches for metamorphic relations in the form of linear or quadratic equations (e.g., $\cos(2x) = 2\cos^2(x) - 1$). Relations are inferred by running the program under test repeatedly, searching for relations among the inputs and outputs. It is therefore a black-box approach which requires no access to the source code. Since running the program with all the possible input values is rarely possible, the relations identified are strictly referred

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.

to as *likely* metamorphic relations, until they are confirmed by a domain expert. Their work was evaluated inferring hundreds of likely metamorphic relations for 189 functions of 4 commercial and open source mathematical libraries. The results showed that the generated metamorphic relations are effective in detecting mutants. Notice that in contrast to the work of Kanewala and Bieman [56], [57], this approach does not predict whether the program exhibits a previously defined metamorphic relation, but rather infers the metamorphic relation from scratch.

Carzinaga et al. [61] proposed to generate oracles by exploiting the redundancy contained in programs. Given a source test case, they generate a test with the same code in which some operations are replaced with redundant ones. For instance, in the `AbstractMultimap<K,V>` class of the Google Guava library,[4] the methods `put(k,v)` and `putAll(k,c)` are equivalent when `c` is a collection containing a single element `v`. If the outputs of both test cases are not equal, the code must contain a bug. The author presented an implementation of their approach using aspects. The identification of redundant methods is a manual task. Although the core of their contribution was not related to metamorphic testing, their approach can be considered a specific application of the technique. In a related article, Goffi et al. [62], [63] presented a search-based algorithm for the automated synthesis of *likely-equivalent* method sequences in object-oriented programs. The authors suggest that such likely-equivalent sequences could be used as metamorphic relations during testing. The approach was evaluated using 47 methods of 7 classes taken from the Stack Java Standard Library and the Graphstream library. The algorithm automatically synthesised 87 percent (123 out of 141) of the equivalent method sequences manually identified.

Su et al. [64] presented an approach named KABU for the dynamic inference of likely metamorphic relations inspired by previous work on the inference of program invariants [19]. The inference process is constrained by searching for a set of predefined metamorphic relations [58]. A Java tool implementing the approach was presented and evaluated on the inference of likely metamorphic relations in two sample programs. As a result, KABU found more likely metamorphic relations than a group of 23 students trained in the task. Authors also proposed a method, *Metamorphic Differential Testing (MDT)*, built upon KABU, to compare the metamorphic relations between different versions of the same program reporting the differences as potential bugs. Experimental results on different versions of two classification algorithms showed that MDT successfully detected the changes reported in the logs of the Weka library.

Chen et al. [65] presented a specification-based methodology and associated tool called METRIC for the identification of metamorphic relations based on the category-choice framework [66]. In this framework, the program specification is used to partition the input domain in terms of categories, choices and complete test frames. Roughly speaking, a complete test frame is an abstract test case defining possible combinations of inputs, e.g., *{type of vehicle, weekday, parking hours}*. Given a set of complete test frames, METRIC guides testers on the identification of metamorphic relations and related source and follow-up test cases. The results of an empirical study with 19 participants suggest that METRIC is effective and efficient at identifying metamorphic relations.

4.3 Generation of Source Test Cases

As mentioned in Section 6.2, most contributions on metamorphic testing use either random test data or existing test suites for the creation of source test cases. In this section, we review the papers proposing alternative methods for the generation of source test cases.

Gotlieb and Botella [67] presented a framework named *Automated Metamorphic Testing (AMT)* to automatically generate test data for metamorphic relations. Given the source code of a program written in a subset of C and a metamorphic relation, AMT tries to find test cases that violate the relation. The underlying method is based on the translation of the code into an equivalent constraint logic program over finite domains. The solving process is executed until a solution is found or a timeout is reached. The supported types of metamorphic relations are limited to numeric expressions over integers. The framework was evaluated using three laboratory programs with seeded faults.

Chen et al. [28] compared the effectiveness of "special values" and random testing as source test cases for metamorphic testing. Special values are test inputs for which the expected output is well known (e.g., $\sin(\pi/2) = 1$). Since test cases with special values must be manually constructed we consider them as manual testing. The authors found that manual and metamorphic testing are complementary techniques, but they also note that random testing has the advantage of being able to provide much larger test data sets. In a closely related study, Wu et al. [68] concluded that random source test cases result in more effective metamorphic test cases than those derived from manual test cases (special values). Segura et al. [69] compared the effectiveness of random testing and a manually designed test suite as the source test cases for metamorphic testing, and their results also showed that random source test cases are more effective at detecting faults than manually designed source test cases in all the subject programs. Even though this suggests that random testing is more effective, there are also indications that *combining* random testing with manual tests may be even better: Chen et al [28] concluded that random testing is an efficient mechanism to augment the number of source test cases; Segura et al. [69] observed that combining manual tests with random tests leads to faster fault detection compared to using random tests only.

Batra and Sengupta [41] presented a genetic algorithm for the selection of source test cases maximising the paths traversed in the program under test. The goal is to generate a small but highly effective set of source test cases. Their algorithm was evaluated by generating source test cases for several metamorphic relations in a small C program, which determines the type of a triangle, where 4 mutants were generated and killed. In related work, Chen et al. [42] addressed the same problem from a black-box perspective. They proposed partitioning the input domain of the program under test into equivalence classes, in which the program is expected to process the inputs in a similar way. Then, they proposed an algorithm to select test cases that cover those equivalence classes. Evaluation on the triangle

4. https://github.com/google/guava.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.

program suggests that their algorithm can generate a small set of test cases with high detection rate.

Dong and Zhang [44] presented a method for the construction of metamorphic relations and their corresponding source test cases using symbolic execution. The method first analyses the source code of the program to determine the symbolic inputs that cause the execution of each path. Then, the symbolic inputs are manually inspected and used to guide the construction of metamorphic relations that can exercise all the paths of the program. Finally, source test cases are generated by replacing the symbolic inputs by real values. As in previous work, the approach was evaluated using a small C program with seeded faults.

4.4 Execution of Metamorphic Test Cases

The execution of a metamorphic test case is typically performed in two steps. First, a follow-up test case is generated by applying a transformation to the inputs of a source test case. Second, source and follow-up test cases are executed, checking whether their outputs violate the metamorphic relation. In this section, we present those articles that either propose a different approach for the execution of metamorphic test cases, or to automate part of the process.

Several papers have contributed to the execution and assessment of metamorphic test cases. Wu [70] presented a method named *Iterative Metamorphic Testing (IMT)* to systematically exploit more information from metamorphic tests, by applying metamorphic relations iteratively. In IMT, a sequence of metamorphic relations are applied in a chain style, by reusing the follow-up test case of each metamorphic relation as the source test case of the next metamorphic relation. A case study was presented with a program for sparse matrix multiplication and more than 1,300 mutants. The results revealed that IMT detects more faults than classic metamorphic testing and special value testing. Dong et al. [71] presented an algorithm integrating IMT and program path analysis. The algorithm runs metamorphic tests iteratively until a certain path coverage criterion is satisfied. Segura et al. [69], [72], [73] presented a metamorphic testing approach for the detection of faults in variability analysis tools. Their method is based on the iterative application of a small set of metamorphic relations. Each relation relates two input variability models and their corresponding set of configurations, (i.e., output). In practice, the process can generate an unlimited number of random test cases of any size. In certain domains, it was necessary to apply the metamorphic relations in a certain order. Their approach was proven effective in detecting 19 real bugs in 7 different tools.

Guderlei and Mayer [74] proposed *Statistical Metamorphic Testing (SMT)* for the application of metamorphic testing to non-deterministic programs. SMT does not consider a single execution, but is based on studying the statistical properties of multiple invocations to the program under test. The method works by generating two or more sequences of outputs by executing source and follow-up test cases. Then, the sequences of outputs are compared according to their statistical properties using statistical hypothesis tests. The applicability of the approach was illustrated with a single metamorphic relation on a subject program with seeded faults. In later work, Murphy et al. [75], [76] successfully applied SMT to the detection of faults in a health care simulation program with non-deterministic time events.

Murphy et al. [77], [78] presented an extension of the Java Modelling Language (JML) [79] for the specification and runtime checking of metamorphic relations. Their approach extends the JML syntax to enable the specification of metamorphic properties, which are included in the Java source code as annotations. The extension was designed so it could express the typical metamorphic relations observed by the authors in the domain of machine learning [80]. Additionally, they presented a tool, named Corduroy, that pre-processes the specification of metamorphic relations and generates test code that can be executed using JML runtime assertion checking, ensuring that the relations hold during program execution. For the evaluation, they specified 25 metamorphic relations on several machine learning applications uncovering a few defects.

Murphy et al. [81] presented a framework named Amsterdam for the automated application of metamorphic testing. The tool takes as inputs the program under test and a set of metamorphic relations, defined in an XML file. Then, Amsterdam automatically runs the program, applies the metamorphic relations and checks the results. The authors argue that in certain cases slight variations in the outputs are not actually indicative of errors, e.g., floating point calculations. To address this issue, the authors propose the concept of *heuristic test oracles*, by defining a function that determines whether the outputs are "close enough" to be considered equals. This idea was also used in a later empirical study [75] comparing the effectiveness of three different techniques to test programs without oracles: "niche oracle" (i.e., inputs with known expected outputs), metamorphic testing and assertion checking. The study revealed that metamorphic testing outperforms the other techniques, also when testing non-deterministic programs.

Ding et al. [43] proposed a method named *Self-Checked Metamorphic Testing (SCMT)* combining metamorphic testing and structural testing. SCMT checks the code coverage of source and follow-up test cases during test execution to evaluate the quality of metamorphic relations. It is assumed that the higher the coverage, the more effective the metamorphic relation. The test coverage data obtained may be used to refine test cases by creating, replacing or updating metamorphic relations and their test data. It is also suggested that unexpected coverage outcomes could help detect false-positive results, which they define as a metamorphic relation that holds despite the program being faulty. The approach was evaluated using a cellular image processing program with one seeded bug.

Zhu [82] presented JFuzz, a Java unit testing tool using metamorphic testing. In JFuzz, tests are specified in three parts, namely i) source test case inputs (x), ii) possible transformations on the test inputs ($y = \pi - x$), and iii) metamorphic relations implemented as code assertions ($\sin(x) = \sin(\pi - x)$). Once these elements are defined, the tool automatically generates follow-up test cases by applying the transformations to the source test inputs, it executes source and follow-up test cases, and checks whether the metamorphic relations are violated.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.

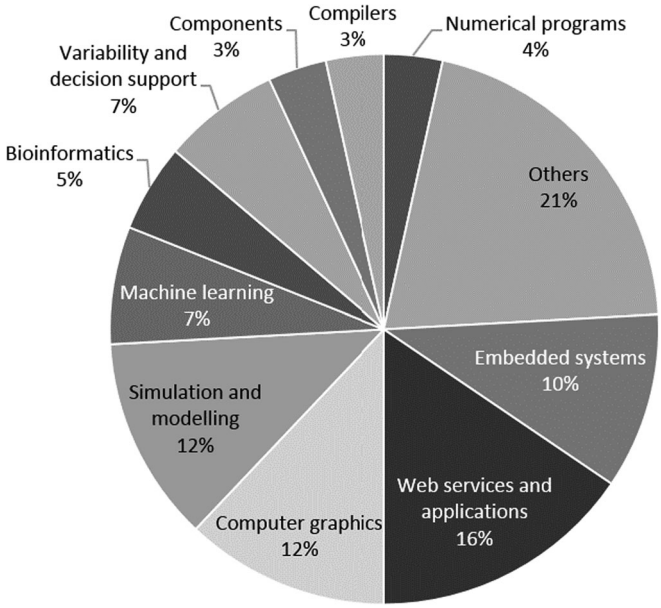


Fig. 4. Metamorphic testing application domains.

5 The Application of Metamorphic Testing

In this section, we answer RQ2 by investigating the scope of metamorphic testing and its applications. In particular, we review applications of metamorphic testing to specific problem domains, and summarise approaches that use metamorphic testing to enhance other testing techniques.

5.1 Application Domains

In this section, we review those papers where the main contribution is a case study on the application of metamorphic testing to specific testing problems (58 out of 119). Fig. 4 classifies these papers according to their application domain. In total, we identified more than 12 different application areas. The most popular domains are web services and applications (16 percent) followed by computer graphics (12 percent), simulation and modeling (12 percent) and embedded systems (10 percent). We also found a variety of applications to other fields (21 percent) such as financial software, optimisation programs or encryption programs. Each of the other domains is explored in no more than four papers, to date. Interestingly, we found that only 4 percent of the papers reported results in numerical programs, even though this seems to be the dominant domain used to illustrate metamorphic testing in the literature.

Fig. 5 shows the domains where metamorphic testing applications have been reported in chronological order. Domains marked with (T) were only explored theoretically. As illustrated, the first application of metamorphic testing was reported in the domain of numerical programs back in 2002. While in the subsequent years the potential applications of metamorphic testing were mainly explored at a theoretical level, there are applications in multiple domains from 2007 onwards. The rest of this section introduces the papers reporting results in each application domain.

5.1.1 Web Services and Applications

Chan et al. [83], [84] presented a metamorphic testing methodology for Service-Oriented Applications (SOA). Their method relies on the use of so-called *metamorphic services* to encapsulate the services under test, execute source and follow-up test cases and check their results. Similarly, Sun et al. [34], [85] proposed to manually derive metamorphic relations from the WSDL description of web services. Their technique automatically generates random source test cases from the WSDL specification and applies the metamorphic relations. They presented a tool to partially automate the process, and evaluated it with three subject web services and mutation analysis. In a related project, Castro-Cabrera and Medina-Bulo [86], [87] presented a metamorphic testing-based approach for web service compositions using the Web Service Business Process Execution Language (WS-BPEL) [88]. To this end, they proposed to analyse the XML description of the service composition to select adequate metamorphic relations. Test cases were defined in terms of the inputs and outputs of the participant services.

In a related set of papers, Zhou et al. [20], [89] used metamorphic testing for the detection of inconsistencies in online web search applications. Several metamorphic relations were proposed and used in a number of experiments with the web search engines Google, Yahoo! and Live Search. Their results showed that metamorphic testing effectively detected inconsistencies in the searches in terms of both returned content and ranking quality. In later work [90], the authors performed an extensive empirical study on the web

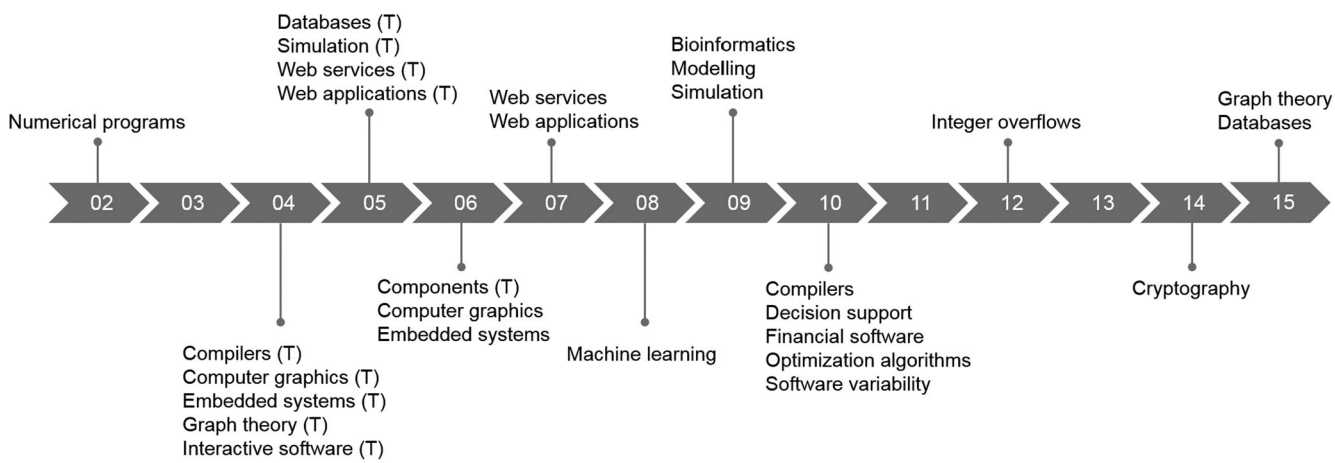


Fig. 5. Timeline of metamorphic testing applications. Domains marked with (T) were only explored theoretically.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.

search engines Google, Bing, Chinese Bing and Baidu. As a novel contribution, metamorphic relations were defined from the user perspective, representing the properties that a user expects from a "good" search engine, regardless of how the engine is designed. In practice, as previously noticed by Xie et al. [31], this means that metamorphic relations are not only suitable to detect faults in the software under test (verification) but also to check whether the program behaves as the user expects (validation). The authors also proposed using metamorphic testing to assess quality related properties such as reliability, usability or performance. Experimental results revealed a number of failures in the search engines under test.

5.1.2 Computer Graphics

Mayer and Guderlei [91], [92] compared several random image generation techniques for testing image processing programs. The study was performed on the implementation of several image operators as the Euclidean distance transform. Several metamorphic relations were used for the generation of follow-up test cases and the assessment of test results. Chan et al. [93], [94] presented a testing approach for mesh simplification programs using pattern classification and metamorphic testing. Metamorphic relations were used to detect test cases erroneously labelled as passed by a trained pattern classifier. Just and Schweiggert [95] used mutation analysis to evaluate the effectiveness of test data generation techniques and metamorphic relations for a jpeg2000 image encoder. Kuo et al. [33] presented a metamorphic testing approach for programs dealing with the surface visibility problem. A real bug was revealed in a binary space partitioning tree program. Finally, Jameel et al. [96] presented a case study on the application of metamorphic testing to detect faults in morphological image operations such as dilation and erosion. Eight metamorphic relations were reported and assessed on the detection of seeded faults in a binary image dilation program.

5.1.3 Embedded Systems

Tse et al. [97] proposed the application of metamorphic testing to context-sensitive middleware-based software programs. Context-based applications adapt their behaviour according to the information from its environment referred to as *context*. The process of updating the context information typically relies on a *middleware*. Intuitively, their approach generates different context situations and checks whether the outcomes of the programs under test satisfy certain relations. This work was extended to deal with changes in the context during test execution [52], [98]. Chan et al. [99] applied metamorphic testing to wireless sensor networks. As a novel contribution, they proposed to check not only the functional output of source and follow-up test cases but also the energy consumed during the execution, thus targeting both functional and non-functional bugs. Kuo et al. [100] reported a case study on the use of metamorphic testing for the detection of faults in a wireless metering system. A metamorphic relation was identified and used to test the meter reading function of a commercial device from the electric industry in which two real defects were uncovered. Finally, Jiang et al. [101] presented several metamorphic relations for the detection of faults in Central Processing Unit (CPU) scheduling algorithms. Two real bugs were detected in one of the simulators under test.

5.1.4 Simulation and Modelling

Sim et al. [102] presented an application of metamorphic testing for casting simulation, exploiting the properties of the medial axis geometry function. Several metamorphic relations were introduced but no empirical results were presented. Chen et al. [103] proposed the application of metamorphic testing to check the conformance between network protocols and network simulators. A case study was presented testing the OMNeT++ simulator [104] for conformance with the ad-hoc on-demand distance vector protocol. In a related project, Chen et al. [37] proposed using metamorphic testing for the detection of faults in open queuing network modelling, a technique for planning the capacity of computer and communication systems. Ding et al. [105] presented a case study on the detection of faults in a Monte Carlo modelling program for the simulation of photon propagation. Based on their previous work [43], the authors used code coverage criteria to guide the selection of effective metamorphic relations and the creation of test cases. Murphy et al. [76] proposed using metamorphic relations to systematically test health care simulation programs, and presented a case study with two real-world simulators and mutation testing. More recently, Núñez and Hierons [106] proposed using metamorphic relations to detect unexpected behaviour when simulating cloud provisioning and usage. A case study using two cloud models on the iCanCloud simulator [107] was reported. Cañizares et al. [108] presented some preliminary ideas on the use of simulation and metamorphic testing for the detection of bugs related to energy consumption in distributed systems as cloud environments.

5.1.5 Machine Learning

Murphy et al. [58] identified six metamorphic relations that they believe exist in most machine learning applications, namely: additive, multiplicative, permutative, invertive, inclusive, and exclusive relations. The effectiveness of the relations was assessed on three specific machine learning tools in which some real bugs were detected. In a related project, Xie et al. [31], [109] proposed using metamorphic testing for the detection of faults in supervised classifiers. It was argued that metamorphic relations may represent both necessary and expected properties of the algorithm under test. Violations of necessary properties are caused by faults in the algorithm and therefore are helpful for the purpose of verification. Violations of expected properties indicate divergences between what the algorithm does and what the user expects, and thus are helpful for the purpose of validation. Two specific algorithms were studied: K-Nearest neighbors and Naïve Bayes classifier. The results revealed that both algorithms violated some of the necessary properties identified as metamorphic relations indicating faults or unexpected behaviours. Also, some real faults were detected in the open-source machine learning tool Weka [110]. Finally, Jing et al. [111] presented a set of metamorphic relations for association rule algorithms and evaluated them using a contact-lenses data set and the Weka tool.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.

5.1.6 Variability and Decision Support

Segura et al. [69], [72] presented a test data generator for feature model analysis tools. Test cases are automatically generated from scratch using step-wise transformations that ensure that certain constraints (metamorphic relations) hold at each step. In later work [73], the authors generalised their approach to other variability domains, namely CUDF documents and Boolean formulas. An extensive evaluation of effectiveness showed, among other results, fully automatic detection of 19 real bugs in 7 tools. In a related domain,[5] Kuo et al. [45] presented a metamorphic testing approach for the automated detection of faults in decision support systems. In particular, they focused on the so-called multi-criteria group decision making, in which decision problems are modelled as a three-dimensional matrix representing alternatives, criteria and experts. Several metamorphic relations were presented and used to test the research tool Decider [45], where a bug was uncovered.

5.1.7 Bioinformatics

Chen et al. [40] presented several metamorphic relations for the detection of faults in two open-source bioinformatics programs for gene regulatory networks simulations and short sequence mapping. Also, the authors discussed how metamorphic testing could be used to address the oracle problem in other bioinformatics domains such as phylogenetic, microarray analysis and biological database retrieval. Pullum and Ozmen [112] proposed using metamorphic testing for the detection of faults in predictive models for disease spread. A case study on the detection of faults in two disease–spread models of the 1918 Spanish flu was presented, revealing no bugs. In a related project, Ramanathan et al. [113] proposed using metamorphic testing, data visualisation, and model checking techniques to formally verify and validate compartmental epidemiological models.

5.1.8 Components

Beydeda [114] proposed a self-testing method for commercial off-the-shelf components using metamorphic testing. In this method, components are augmented with self-testing functionality including test case generation, execution and evaluation. In practice, this method allows users of a component to test it even without access to its source code. Lu et al. [115] presented a metamorphic testing methodology for component–based software applications, both at the unit and integration level. The underlying idea is to run test cases against the interfaces of the components under test, using metamorphic relations to construct follow-up test cases and to check their results.

5.1.9 Numerical programs

Chen et al. [116] presented a case study on the application of metamorphic testing to programs implementing partial differential equations. The case study focused on a practical problem in thermodynamics, namely the distribution of temperatures in a square plate. They injected a seeded fault in the program under test and compared the effectiveness of "special" test cases and metamorphic testing in detecting the fault. Special test cases were unable to detect the fault, while metamorphic testing was effective at revealing it using a single metamorphic relation. Aruna and Prasad [117] presented several metamorphic relations for multiplication and division of multi-precision arithmetic software applications. The work was evaluated with four real-time mathematical projects and mutation analysis.

5.1.10 Compilers

Tao et al. [118] presented a so-called "equivalence preservation" metamorphic relation to test compilers. Given an input program, the relation is used to generate an equivalent variant of it, checking whether the behaviours of the resulting executables are the same for a random set of inputs. The authors proposed three different strategies for the generation of equivalent source programs, such as replacing an expression with an equivalent one (e.g., $e \times 2 \equiv e + e$). The evaluation of their approach revealed two real bugs in two C compilers. A closely related idea was presented by Le et al. [119]. Given a program and a set of input values, the authors proposed to create equivalent versions of the program by profiling its execution and pruning unexecuted code. Once a program and its equivalent variant are constructed, both are used as input of the compiler under test, checking for inconsistencies in their results. So far, this method has been used to detect 147 confirmed bugs in two open source C compilers, GCC and LLVM.

5.1.11 Other Domains

Zhou et al. [39] presented several illustrative applications of metamorphic testing in the context of numerical programs, graph theory, computer graphics, compilers and interactive software. Chen et al. [120] claimed that metamorphic testing is both practical and effective for end-user programmers. To support their claim, the authors briefly suggested how metamorphic relations could be used to detect bugs in spreadsheet, database and web applications. Sim et al. [121] presented a metamorphic testing approach for financial software. Several metamorphic relations were integrated into the commercial tool MetaTrader [122] following a self-testing strategy. Source and follow-up test cases were derived from the real-time input price data received at different time periods. Metamorphic testing has also been applied to optimisation programs using both stochastic [123] and heuristic algorithms [32]. Yao et al. [124], [125], [126] presented preliminary results on the use of metamorphic testing to detect integer overflows. Batra and Singh [127] proposed using UML diagrams to guide the selection of metamorphic relations and presented a small case study using a banking application. Sun et al. [128] reported several metamorphic relations for encryption programs. Aruna and Prasad [129] presented a small case study on the application of metamorphic testing to two popular graph theory algorithms. Finally, Lindvall et al. [130] presented an experience report on the use of metamorphic testing to address acceptance testing of NASA's Data Access Toolkit (DAT). DAT is a huge database of telemetry data collected from different NASA missions, and an advance

5. Note that variability models can be used as decision models during software configuration.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.

query interface to search and mine the available data. Due to the massive amount of data contained in the database, checking the correctness of the query results is challenging. To address this issue, metamorphic testing was used by formulating the same query in different equivalent ways, and asserting that the resulting datasets are the same. Several issues were detected with this approach.

5.2 Other Testing Applications

Besides direct application as a testing technique, metamorphic testing has been integrated into other testing techniques, in order to improve their applicability and effectiveness. In this section, we review these approaches.

Chen et al. [18], [131] proposed using metamorphic testing with fault-based testing. Fault-based testing uses symbolic evaluation [132], [133] and constraint solving [133] techniques to prove the absence of certain types of faults in the program under test. The authors used several numerical programs to illustrate how real and symbolic inputs can be used to discard certain types of faults even in the absence of an oracle. In a related project [30], [134], the authors presented a method called *semi-proving* integrating global symbolic execution and constraint solving for program proving, testing and debugging. Their method uses symbolic execution to prove whether the program satisfies certain metamorphic relations or identify the inputs that violate them. It also supports debugging by identifying violated constraint expressions that reveal failures.

Dong et al. [135] proposed improving the efficiency of Structural Evolutionary Testing (SET) using metamorphic relations. In SET, evolutionary algorithms are used to generate test data that satisfy a certain coverage criteria (e.g., condition coverage). This is often achieved by minimising the distance of the test input to execute the program conditions in the desired way. To improve the efficiency of the process, the authors proposed to use metamorphic relations during the search to consider both source and follow-up test cases as candidate solutions, accelerating the chances of reaching the coverage target. Their approach was evaluated with two numerical programs.

Xie et al. [136], [137] proposed the combination of metamorphic testing and Spectrum-Based Fault Localisation (SBFL) for debugging programs without an oracle. SBFL uses the results of test cases and the corresponding coverage information to estimate the risk of each program entity (e.g., statements) of being faulty. Rather than a regular test oracle, the authors proposed to use the violation or non-violation information from metamorphic relations rather than the actual output of test cases. Among other results, their approach was used to uncover two real bugs in the Siemens Suite [138]. In a related project, Lei et al. [139] applied the same idea to address the oracle problem in a variant of SBFL named Backward-Slice Statistical Fault Localisation (BSSFL) [140]. Rao et al. [141] investigated the ratio between non-violated and violated metamorphic relations in SBFL. They concluded that the higher the ratio of non-violated metamorphic relations to violated metamorphic relations, the less effective the technique. Aruna et al. [142] proposed integrating metamorphic testing with the Ochiai algorithm [143] for fault localisation in dynamic web applications. Five metamorphic relations for a classification algorithm were presented as well as some experimental results.

Liu et al. [144] presented a theoretical description of a new method called *Metamorphic Fault Tolerance (MFT)*. In MFT, the trustworthiness of test inputs is determined in terms of the number of violated and non-violated metamorphic relations. The more relations are satisfied and the fewer relations are violated, the more trustworthy the input is. Also, if an output is judged as untrustworthy, the outputs provided by metamorphic relations can be used to provide a more accurate output.

Jin et al. [145] presented an approach called *Concolic Metamorphic Debugging*, which integrates concolic testing, metamorphic testing, and branch switching debugging, in order to localise potential bugs. Concolic testing is a technique that executes the program under test with both, symbolic and concrete inputs, and then uses symbolic path conditions to derive new test inputs for paths not yet explored. Based on a failure-inducing test input, the proposed method explores all possible program paths in depth-first-order, searching for the first one that passes the metamorphic relation. The final goal is to isolate a minimum amount of code to obtain a passing input, and use that isolation point to localise the fault. The approach, implemented in a tool called Comedy, was evaluated on 21 small programs with seeded faults. Comedy successfully generated debugging reports in 88 percent of the faulty programs and precisely located the fault in 36 percent of them.

6 Experimental Evaluations

In this section, we address RQ3 by reviewing the experimental evaluations of the surveyed papers. In particular, we summarise their main characteristics in terms of subject programs, source test cases, types of faults, number of metamorphic relations and evaluation metrics. Additionally, we review the results of empirical studies involving humans.

6.1 Subject Programs

As a part of the review process, we collected information about the subject programs used for the evaluation of metamorphic testing contributions. The table provided as supplemental material, which can be found on the Computer Society Digital Library at http://doi.ieeecomputersociety.org/10.1109/TSE.2016.2532875, shows the name, language, size, description and the references of the papers reporting results for each program. In the cases where the information was unavailable in the literature, it is indicated with "NR" (Not Reported). The table is ordered by the number of papers that use the subject programs. Thus, the programs at the top of the list are the most studied subject programs in the metamorphic testing literature. Overall, we identified 145 different subject programs. Most of them are written in Java (46.2 percent) and C/C++ (35.5 percent), with reported sizes ranging between 12 and 12,795 lines of code.

In experimentation, the use of real world programs, rather than research programs, is commonly recognised as an indicator of the maturity of a discipline [25]. To assess this maturity, we studied the relationship between the use of research and real world programs in metamorphic testing experiments. Similarly to previous surveys [25], we consider a program to

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.

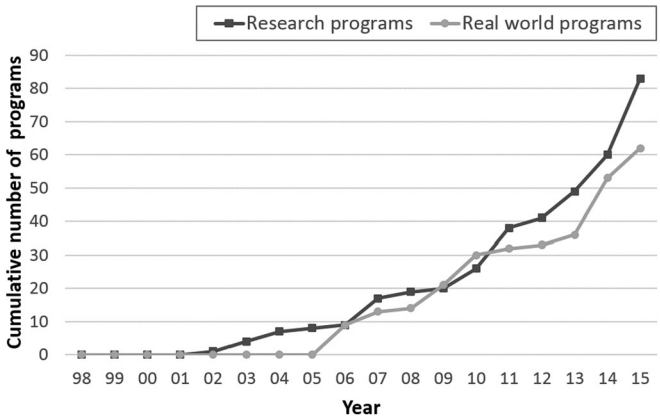


Fig. 6. Research versus real world subject programs.

be a "real world" program if it is either a commercial or an open-source program, otherwise we consider it as a "research program". As an exception to this rule, we consider all open source projects that are designed as benchmarks rather than applications as research programs (e.g., the Siemens suite). Fig. 6 presents the cumulative view of the number of each type of program, research and real world, by year. As illustrated, research programs are used since 2002, while real world programs were not introduced in metamorphic testing experiments until 2006. Since then, the use of both types of programs has increased with similar trends. It is noteworthy that the number of real world programs in 2010 was higher than the number of research programs. The cumulative number in 2015 shows a significant advantage of research programs (83) over real world programs (62). The overall trend, however, suggests that metamorphic testing is maturing.

6.2 Source Test Cases

Metamorphic testing requires the use of source test cases that serve as seed for the generation of follow-up test cases. Source test cases can be generated using any traditional testing techniques. We studied the different techniques used in the literature and counted the number of papers using each of them; the results are presented in Fig. 7. As illustrated, a majority of studies used random testing for the generation of source test cases (57 percent), followed by those using an existing test suite (34 percent). Also, several papers (6 percent) use tool-based techniques such as constraint programming, search-based testing or symbolic execution. This diversity of usable sources supports the applicability of metamorphic testing. It also supports the use of random testing as a cost-effective and straightforward approach for the generation of the initial test suite (cf., Section 4.3).

6.3 Types of Faults

The effectiveness of metamorphic testing approaches is assessed according to their ability to detect failures caused by faults in the programs under test. Uncovering real bugs is the primary goal, but they are not always available for evaluation. Thus, most authors introduce artificial faults (a.k.a. mutants) in the subject programs either manually or automatically, using mutation testing tools [25]. To study the relationship between real bugs and mutants in metamorphic testing evaluations, we calculated the cumulative number of

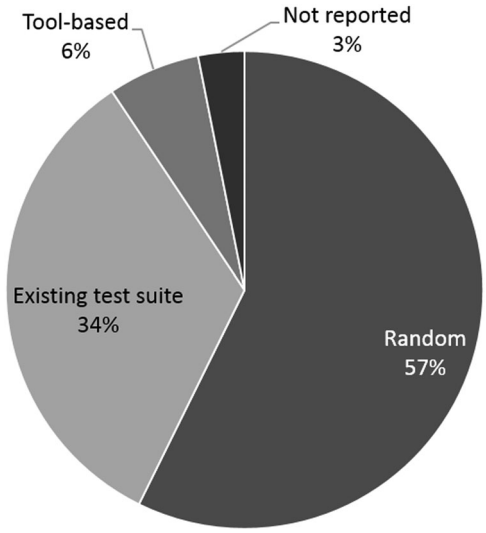


Fig. 7. Source test case generation techniques.

papers reporting results with artificial and real bugs by year, depicted in Fig. 8. We consider a real bug to be a latent, initially unknown, fault in the subject program. As illustrated in Fig. 8, the first experimental results with mutants were presented back in 2002, while the first real bugs were reported in 2007. Since then, the number of papers reporting results with both types of faults has increased, although artificial faults show a steeper angle representing a stronger trend. Besides this, we also counted the number of faults used in each paper. To date, metamorphic testing has been used to detect about 295 distinct real faults in 36 different tools, 23 of which are real world programs, suggesting that metamorphic testing is effective at detecting real bugs.

6.4 Metamorphic Relations

The number of metamorphic relations used in experimentation may be a good indicator of the effort required to apply metamorphic testing. As a part of the data collection process, we counted the number of metamorphic relations presented in each paper containing experimental results. Fig. 9 classifies the papers based on the number of metamorphic relations reported. As illustrated, the largest portion of studies report between five and nine metamorphic relations (39 percent), followed by those presenting between one and four metamorphic relations (24 percent). Interestingly, only nine studies (13 percent) presented more than 25 metamorphic

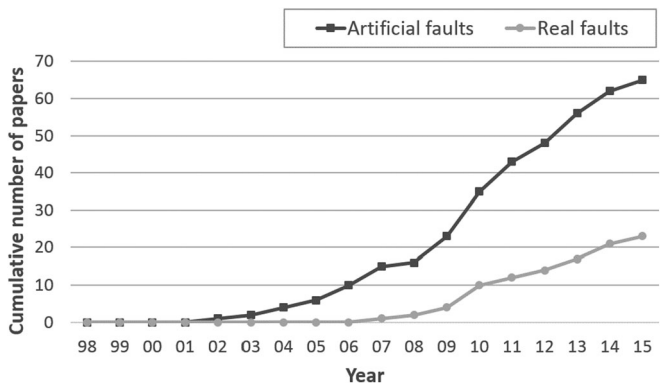


Fig. 8. Artificial versus real faults.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.

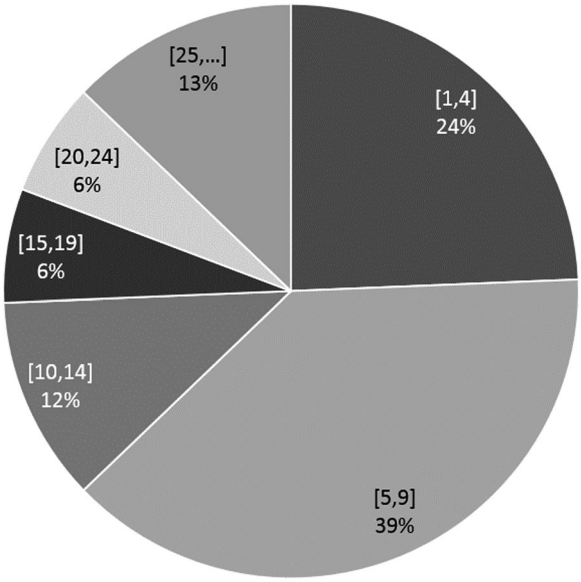


Fig. 9. Number of metamorphic relations.

relations. We took a closer look at those nine papers and observed that all of them reported results for several subject programs. These findings suggest that a modest number of metamorphic relations (less than 10) is usually sufficient to apply metamorphic testing with positive results.

6.5 Evaluation Metrics

Numerous metrics to evaluate the effectiveness of metamorphic testing approaches have been proposed. Among them, we identified two metrics intensively used in the surveyed papers, such that they could be considered as a de-facto standard in the metamorphic testing literature.

6.5.1 Mutation Score

This metric is based on mutation analysis, where mutation operators are applied to systematically produce versions of the program under test containing artificial faults ("mutants") [25]. The mutation score is the ratio of detected ("killed") mutants to the total number of mutants. Mutants that do not change the program's semantics and thus cannot be detected are referred to as *equivalent* [25]. In theory, equivalent mutants should be excluded from the total number of mutants, but in practice this is not always possible since program equivalence is undecidable. Suppose a metamorphic test suite t composed of a set of metamorphic tests, i.e., pairs of source and follow-up test cases. The Mutation Score (MS) of t is calculated as follows:

$$MS(t) = \frac{M_k}{M_t - M_e}, \tag{1}$$

where M_k is the number of killed mutants by the metamorphic tests in t, M_t is the total number of mutants and M_e is the number of equivalent mutants. A variant of this metric [71], [91], [121] is often used to calculate the ratio of mutants detected by a given metamorphic relation r as follows:

$$MS(t, r) = \frac{M_{kr}}{M_t - M_e}, \tag{2}$$

where M_{kr} is the number of mutants killed by the metamorphic tests in t derived from r. This metric is also called mutation detection ratio [36].

6.5.2 Fault Detection Ratio

This metric calculates the ratio of test cases that detect a given fault [41], [55], [68], [70], [71], [101], [124], [126]. The Fault Detection Ratio (FDR) of a metamorphic test suite t and a fault f is calculated as follows:

$$FDR(t, f) = \frac{T_f}{T_t}, \tag{3}$$

where T_f is the number of tests that detect f and T_t is the total number of tests in t. A variant of this metric [27], [32], [33], [37], [45], [54], [71] calculates the ratio of test cases that detect a fault f using a given metamorphic relation r as follows:

$$FDR(t, f, r) = \frac{T_{fr}}{T_r}, \tag{4}$$

where M_{fr} is the number of tests in t derived from the relation r that detect the fault f, and T_r is the total number of metamorphic tests derived from r. This metric is also called fault discovery rate [34], [85], [128].

6.6 Empirical Studies with Humans

Hu et al. [29], [36] reported on a controlled experiment to investigate the cost-effectiveness of using metamorphic testing by 38 testers on three open-source programs. The experiment participants were either asked to write metamorphic relations, or tests with assertions to check whether the final or intermediate state of the program under test is correct. The experiment revealed a trade-off between both techniques, with metamorphic testing being less efficient but more effective at detecting faults than tests with assertions.

Liu et al. [146] reported on a three-year experience in teaching metamorphic testing to various groups of students at Swinburne University of Technology (Australia). The authors explained the teaching approach followed and the lesson learned, concluding that metamorphic testing is a suitable technique for end-user testing. In a later paper, Liu et al. [4] presented an empirical study to investigate the effectiveness of metamorphic testing addressing the oracle problem compared with random testing. For the study, several groups of undergraduate and postgraduate students from two different universities were recruited to identify metamorphic relations in five subject programs of algorithmic type. Metamorphic testing was compared to random testing with and without oracle. Their experiment showed that metamorphic testing was able to find more faults than random testing with and without oracle in most subject programs. Furthermore, it was concluded that a small number of diverse metamorphic relations (between 3 and 6), even those identified in an ad-hoc manner, had a similar fault-detection capability to a test oracle, i.e., comparing the program output with the expected one.

7 Challenges

A number of open research challenges emerge from this survey, based on problems repeatedly encountered throughout

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.

the reviewed papers, or gaps in the literature. These challenges answer RQ4.

Challenge 1. Guidelines for the construction of good metamorphic relations. For most problems, a variety of metamorphic relations with different fault-detection capability can be identified. It is therefore key to know the properties of effective metamorphic relations and to provide systematic methods for their construction. Although several authors have reported lessons learned on the properties of good metamorphic relations (cf., Section 4.1), these are often complementary or even contradictory (e.g., [27], [38]). Therefore, there is a lack of reliable guidelines for the construction of effective metamorphic relations. Such guidelines should provide a step-by-step process to guide testers, both experts and beginners, in the construction of good metamorphic relations.

Challenge 2. Prioritisation and minimisation of metamorphic relations. In certain cases using all the available metamorphic relations may be too expensive and a subset of them must be selected. It is therefore important to know how to prioritise the most effective metamorphic relations. To this end, several authors have proposed using code coverage [43], [46] or test case similarity [47] with promising results. However, the applicability of those approaches as domain-independent prioritisation criteria still needs to be explored. Furthermore, analogously to the concept of test suite minimisation, where redundant test cases are removed from a suite as it evolves [147], the use of minimisation techniques to remove redundant metamorphic relations is an open problem where research is needed. It is worth mentioning that test case minimisation is a NP-hard problem and therefore heuristic techniques should be explored.

Challenge 3. Generation of likely metamorphic relations. The generation of metamorphic relations is probably the most challenging problem to be addressed. Although some promising results have been reported, those are mainly restricted to the scope of numerical programs. The generation of metamorphic relations in other domains as well as the use of different techniques for rule inference are topics where contributions are expected. We also foresee a fruitful line of research exploring the synergies between the problem of generating metamorphic relations and the detection of program invariants [64], [148].

Challenge 4. Combination of metamorphic relations. As presented in Section 4.2, several authors have explored the benefits of combining metamorphic relations following two different strategies, namely applying metamorphic relations in a chain style (IMT) and composing metamorphic relations to construct new relations (CMR). It remains an open problem, however, to compare both approaches and to provide heuristics to decide when to use one or the other. Also, these techniques raise new research problems such us determining whether a given set of metamorphic relations can be combined and in which order.

Challenge 5. Automated generation of source test cases. As described in Section 4.3, most papers use either randomly generated or existing test suites as source tests when applying metamorphic testing. However, there is evidence that the source test cases influence the effectiveness of metamorphic relations [28], [68], [69]. Promising initial results in generating source test cases specifically for given metamorphic relations have been achieved, but many open questions remain about what constitutes the best possible source test cases and how to generate them.

Challenge 6. Metamorphic testing tools. Only two out of all 119 presented a tool as main contribution [78], [82], and very few of the papers on metamorphic testing mentioned a tool implementing the presented techniques [64], [65], [67], [73], [81], [89], [118], [145]. Indeed, if practitioners want to apply metamorphic testing today, they would have to implement their own tool, as there are no publicly available and maintained tools. This is a significant obstacle for a wide-spread use of metamorphic testing in empirical research as well as in practice.

8 Conclusions

In this article, we presented a survey on metamorphic testing covering 119 papers published between 1998 and 2015. We analysed ratios and trends indicating the main advances on the technique, its application domains and the characteristics of experimental evaluations. The results of the survey show that metamorphic testing is a thriving topic with an increasing trend of contributions on the subject. We also found evidence of the applicability of the technique to multiple domains far beyond numerical programs, as well as its integration with other testing techniques. Furthermore, we identified an increasing number of papers reporting the detection of faults in real world programs. All these findings suggest that metamorphic testing is gaining maturity as an effective testing technique, not only to alleviate the oracle problem, but also for the automated generation of test data. Finally, despite the advances on metamorphic testing, our survey points to areas where research is needed. We trust that this work may become a helpful reference for future development on metamorphic testing as well as to introduce newcomers in this promising testing technique.

Acknowledgments

We would like to thank T. Y. Chen, Robert M. Hierons, Phil McMinn, Amador Durán, Zhi Quan Zhou, Christian Murphy, Huai Liu, Xiaoyuan Xie, Alberto Goffi, Gagandeep, Carmen Castro-Cabrera, Yan Lei and Peng Wu for their helpful comments in an earlier version of this article. We are also grateful to the members of the SSE research group led by Mark Harman for the insightful and inspiring discussion during our visit at the University College London. This work has been partially supported by the European Commission (FEDER) and Spanish Government under CICYT projects TAPAS (TIN2012-32273) and BELI (TIN2015-70560-R), and the Andalusian Government projects THEOS (TIC-5906) and COPAS (P12-TIC-1867).

References

[1] E. J. Weyuker, "On testing non-testable programs," *Comput. J.*, vol. 25, no. 4, pp. 465–470, 1982.

[2] S. Anand, E. K. Burke, T. Y. Chen, J. Clark, M. B. Cohen, W. Grieskamp, M. Harman, M. J. Harrold, and P. Mcminn. (Aug. 2013). An orchestrated survey of methodologies for automated software test case generation. *J. Syst. Softw.*[Online] *86(8)*, pp. 1978–2001. Available: http://dx.doi.org/10.1016/j.jss.2013.02.061.

[3] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Trans. Softw. Eng.*, vol. 41, no. 5, pp. 507–525, May 2015.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.

[4] H. Liu, F.-C. Kuo, D. Towey, and T. Y. Chen, "How effectively does metamorphic testing alleviate the oracle problem?" *IEEE Trans. Softw. Eng.*, vol. 40, no. 1, pp. 4–22, Jan. 2014.

[5] T. Y. Chen, S. C. Cheung, and S. M. Yiu, "Metamorphic testing: A new approach for generating next test cases," Dept. Comput. Sci., The Hong Kong Univ. Sci. Technol., Hong Kong, Tech. Rep. HKUST-CS98-01, 1998.

[6] W. J. Cody, *Software Manual for the Elementary Functions*. Upper Saddle River, NJ, USA: Prentice-Hall, 1980.

[7] M. Blum, M. Luby, and R. Rubinfeld. (1993). Self-testing/correcting with applications to numerical problems. *J. Comput. Syst. Sci.* [Online]. *47(3)*, pp. 549–595. Available: http://www.sciencedirect.com/science/article/pii/002200009390044W.

[8] P. E. Ammann and J. C. Knight. (Apr. 1988). Data diversity: An approach to software fault tolerance. *IEEE Trans. Comput.* [Online]. *37(4)*, pp. 418–425. Available: http://dx.doi.org/10.1109/12.2185.

[9] T. Y. Chen, F.-C. Kuo, T. H. Tse, and Z. Q. Zhou, "Metamorphic testing and beyond," in *Proc. 11th Annu. Int. Workshop Softw. Technol. Eng. Practice*, Sep. 2003, pp. 94–100.

[10] T. H. Tse, "Research directions on model-based metamorphic testing and verification," in *Proc. 29th Annu. Int. Comput. Softw. Appl. Conf.*, vol. 1, Jul. 2005, p. 332.

[11] T. Y. Chen, "Metamorphic testing: A simple approach to alleviate the oracle problem," in *Proc. 5th IEEE Int. Symp. Service Oriented Syst. Eng.*, Jun. 2010, pp. 1–2.

[12] W. K. Chan and T. H. Tse, "Oracles are hardly attain'd, and hardly understood: Confessions of software testing researchers," in *Proc. 13th Int. Conf. Quality Softw.*, Jul. 2013, pp. 245–252.

[13] T. Y. Chen, "Metamorphic testing: A simple method for alleviating the test oracle problem," in *Proc. 10th Int. Workshop Autom. Softw. Test*, 2015, pp. 53–54. [Online]. Available: http://dl.acm.org/citation.cfm?id=2819261.2819278.

[14] T. Y. Chen, F.-C. Kuo, D. Towey, and Z. Q. Zhou, "Metamorphic testing: Applications and integration with other methods: Tutorial synopsis," in *Proc. 12th Int. Conf. Quality Softw.*, Aug. 2012, pp. 285–288.

[15] Z. Hui and S. Huang, "Achievements and challenges of metamorphic testing," in *Proc. 4th World Congr. Softw. Eng.*, Dec. 2013, pp. 73–77.

[16] U. Kanewala and J. M. Bieman, "Techniques for testing scientific programs without an oracle," in *Proc. 5th Int. Workshop Softw. Eng. Comput. Sci. Eng.*, May 2013, pp. 48–57.

[17] G. Dong, B. Xu, L. Chen, C. Nie, and L. Wang, "Survey of metamorphic testing," *J. Frontiers Comput. Sci. Technol.*, vol. 3, no. 2, pp. 130–143, 2009.

[18] T. Y. Chen, T. H. Tse, and Z. Q. Zhou. (2003). Fault-based testing without the need of oracles. *Inform. Softw. Technol.* [Online]. *45 (1)*, pp. 1–9. Available: http://dx.doi.org/10.1016/S0950-5849(02)00129-5.

[19] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao. (2007, Dec.). The Daikon system for dynamic detection of likely invariants. *Sci. Comput. Program.* [Online]. *69(1-3)*, pp. 35–45. Available: http://dx.doi.org/10.1016/j.scico.2007.01.015.

[20] Z. Q. Zhou, S. Zhang, M. Hagenbuchner, T. H. Tse, F.-C. Kuo, and T. Y. Chen. (Jun. 2012). Automated functional testing of online search services. *Softw. Testing, Verification Rel. J.* [Online]. *22(4)*, pp. 221–243. Available: http://dx.doi.org/10.1002/stvr.437.

[21] B. Kitchenham, "Procedures for performing systematic reviews," Keele Univ. and NICTA, Staffordshire, U.K., Tech. Rep. TR/SE-0401, 2004.

[22] J. Webster and R. Watson. (2002). Analyzing the past to prepare for the future: Writing a literature review. *MIS Quart.* [Online]. *26(2)*, pp. xiii–xxiii. Available: http://www.misq.org/archivist/vol/no26/issue2/GuestEd.pdf.

[23] D. Benavides, S. Segura, and A. Ruiz-Cortés, "Automated analysis of feature models 20 years later: A literature review," *Inform. Syst.*, vol. 35, no. 6, pp. 615–636, 2010.

[24] Y. J. M. Harman and Y. Zhang, "Achievements, open problems and challenges for search based software testing," in *Proc. 8th IEEE Int. Conf. Softw. Testing, Verification Validation*, Apr. 2015, pp. 1–12.

[25] Y. Jia and M. Harman. (2011, Sep.). An analysis and survey of the development of mutation testing. *IEEE Trans. Softw. Eng.*, vol. 37, no. 5, pp. 649–678, Sep. 2011. [Online]. Available: http://dx.doi.org/10.1109/TSE.2010.62.

[26] S. Segura, G. Fraser, A. B. Sánchez, and A. Ruiz-Cortés. (2016, Feb.). Metamorphic testing: A literature review. Appl. Softw. Eng. Res. Group, Univ. Seville, Seville, Spain. Tech. Rep. ISA-16-TR-02. [Online]. Available: http://www.isa.us.es/sites/default/files/isa-16-tr-02_0.pdf.

[27] T. Y. Chen, D. H. Huang, T. H. Tse, and Z. Q. Zhou, "Case studies on the selection of useful relations in metamorphic testing," in *Proc. 4th Ibero-Amer. Symp. Softw. Eng. Knowl. Eng.*, 2004, pp. 569–583.

[28] T. Y. Chen, F.-C. Kuo, Y. Liu, and A. Tang, "Metamorphic testing and testing with special values," in *Proc. 5th ACIS Int. Conf. Softw. Eng., Artif. Intell., Netw. Parallel/Distrib. Comput.*, 2004, pp. 128–134.

[29] Z. Zhang, W. K. Chan, T. H. Tse, and P. Hu, "Experimental study to compare the use of metamorphic testing and assertion checking," *J. Softw.*, vol. 20, no. 10, pp. 2637–2654, 2009.

[30] T. Y. Chen, T. H. Tse, and Z. Q. Zhou, "Semi-proving: An integrated method for program proving, testing, and debugging," *IEEE Trans. Softw. Eng.*, vol. 37, no. 1, pp. 109–125, Jan. 2011.

[31] X. Xie, J. W. K. Ho, C. Murphy, G. Kaiser, B. Xu, and T. Y. Chen. (2011, Apr.). Testing and validating machine learning classifiers by metamorphic testing. *J. Syst. Softw.* [Online]. *84 (4)*, pp. 544–558. Available: http://dx.doi.org/10.1016/j.jss.2010.11.920.

[32] A. C. Barus, T. Y. Chen, D. Grant, F.-C. Kuo, and M. F. Lau, "Testing of heuristic methods: A case study of greedy algorithm," in *Software Engineering Techniques*, Z. Huzar, R. Koci, B. Meyer, B. Walter, and J. Zendulka, Eds. Berlin, Germany: Springer, 2011, vol. 4980, pp. 246–260. [Online]. Available: http://dx.doi.org/10.1007/978-3-642-22386-0_19.

[33] F.-C. Kuo, S. Liu, and T. Y. Chen, "Testing a binary space partitioning algorithm with metamorphic testing," in *Proc. ACM Symp. Appl. Comput.*, 2011, pp. 1482–1489. [Online]. Available: http://doi.acm.org/10.1145/1982185.1982502.

[34] C. Sun, G. Wang, B. Mu, H. Liu, Z. Wang, and T. Y. Chen, "Metamorphic testing for web services: Framework and a case study," in *Proc. IEEE Int. Conf. Web Services*, Jul. 2011, pp. 283–290.

[35] Z.-W. Hui, S. Huang, H. Li, J.-H. Liu, and L.-P. Rao, "Measurable metrics for qualitative guidelines of metamorphic relation," in *Proc. IEEE 39th Annu. Comput. Softw. Appl. Conf.*, vol. 3, Jul. 2015, pp. 417–422.

[36] P. Hu, Z. Zhang, W. K. Chan, and T. H. Tse, "An empirical comparison between direct and indirect test result checking approaches," in *Proc. 3rd Int. Workshop Softw. Quality Assurance*, 2006, pp. 6–13. [Online]. Available: http://doi.acm.org/10.1145/1188895.1188901.

[37] T. Y. Chen, F.-C. Kuo, R. Merkel, and W. K. Tam, "Testing an open source suite for open queuing network modelling using metamorphic testing technique," in *Proc. IEEE 14th Int. Conf. Eng. Complex Comput. Syst.*, Jun. 2009, pp. 23–29.

[38] J. Mayer and R. Guderlei, "An empirical study on the selection of good metamorphic relations," in *Proc. 30th Annu. Int. Comput. Softw. Appl. Conf.*, vol. 1, Sep. 2006, pp. 475–484.

[39] Z. Q. Zhou, D. H. Huang, T. H. Tse, Z. Yang, H. Huang, and T. Y. Chen, "Metamorphic testing and its applications," in presented at the 8th Int. Symp. Future Software Technology, Xian, China, 2004.

[40] T. Y. Chen, J. W. K. Ho, H. Liu, and X. Xie. (2009). An innovative approach for testing bioinformatics programs using metamorphic testing. *BioMed. Central Bioinform. J.* [Online]. *10(1)*, p. 24. Available: http://www.biomedcentral.com/1471-2105/10/24.

[41] G. Batra and J. Sengupta, "An efficient metamorphic testing technique using genetic algorithm," in *Information Intelligence, Systems, Technology and Management* (ser. Communications in Computer and Information Science), S. Dua, S. Sahni, and D. Goyal, Eds., Berlin, Germany: Springer, 2011, vol. 141, pp. 180–188. [Online]. Available: http://dx.doi.org/10.1007/978-3-642-19423-8_19.

[42] L. Chen, L. Cai, J. Liu, Z. Liu, S. Wei, and P. Liu, "An optimized method for generating cases of metamorphic testing," in *Proc. 6th Int. Conf. New Trends Inform. Sci. Service Sci. Data Mining*, Oct. 2012, pp. 439–443.

[43] J. Ding, T. Wu, J. Q. Lu, and X. Hu, "Self-checked metamorphic testing of an image processing program," in *Proc. 4th Int. Conf. Secure Softw. Integr. Rel. Improvement*, Jun. 2010, pp. 190–197.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.

[44] G. Dong, T. Guo, and P. Zhang, "Security assurance with program path analysis and metamorphic testing," in *Proc. 4th IEEE Int. Conf. Softw. Eng. Service Sci.*, May 2013, pp. 193–197.

[45] F.-C. Kuo, Z. Q. Zhou, J. Ma, and G. Zhang, "Metamorphic testing of decision support systems: A case study," *IET Softw.*, vol. 4, no. 4, pp. 294–301, Aug. 2010.

[46] M. Asrafi, H. Liu, and F.-C. Kuo, "On testing effectiveness of metamorphic relations: A case study," in *Proc. 5th Int. Conf. Secure Softw. Integr. Rel. Improvement*, Jun. 2011, pp. 147–156.

[47] Y. Cao, Z. Q. Zhou, and T. Y. Chen, "On the correlation between the effectiveness of metamorphic relations and dissimilarities of test case executions," in *Proc. 13th Int. Conf. Quality Softw.*, Jul. 2013, pp. 153–162.

[48] Z. Q. Zhou, "Using coverage information to guide test case selection in adaptive random testing," in *Proc. Comput. Softw. Appl. Conf. Workshops*, Jul. 2010, pp. 208–213.

[49] R. Just and F. Schweiggert, "Automating software tests with partial oracles in integrated environments," in *Proc. 5th Workshop Autom. Softw. Test*, 2010, pp. 91–94. [Online]. Available: http://doi.acm.org/10.1145/1808266.1808280.

[50] R. Just and F. Schweiggert. (2011). Automating unit and integration testing with partial oracles. *Softw. Quality J.* [Online]. *19(4)*, pp. 753–769. Available: http://dx.doi.org/10.1007/s11219-011-9151-x.

[51] X. Xie, J. Tu, T. Y. Chen, and B. Xu, "Bottom-up integration testing with the technique of metamorphic testing," in *Proc. 14th Int. Conf. Quality Softw.*, Oct. 2014, pp. 73–78.

[52] W. K. Chan, T. Y. Chen, H. Lu, T. H. Tse, and S. S. Yau. (2006). Integration testing of context-sensitive middleware-based applications: A metamorphic approach. *Int. J. Softw. Eng. Knowl. Eng.* [Online]. *16(5)*, pp. 677–704. Available: http://dblp.uni-trier.de/db/journals/ijseke/ijseke16.html#ChanCLTY06.

[53] Z. Hui and S. Huang, "A formal model for metamorphic relation decomposition," in *Proc. 4th World Congr. Softw. Eng.*, Dec. 2013, pp. 64–68.

[54] H. Liu, X. Liu, and T. Y. Chen, "A new method for constructing metamorphic relations," in *Proc. 12th Int. Conf. Quality Softw.*, Aug. 2012, pp. 59–68.

[55] G. Dong, B. Xu, L. Chen, C. Nie, and L. Wang, "Case studies on testing with compositional metamorphic relations," *J. Southeast Univ. (English Edition)*, vol. 24, no. 4, pp. 437–443, 2008.

[56] U. Kanewala and J. M. Bieman, "Using machine learning techniques to detect metamorphic relations for programs without test oracles," in *Proc. IEEE 24th Int. Symp. Softw. Rel. Eng.*, Nov. 2013, pp. 1–10.

[57] U. Kanewala, "Techniques for automatic detection of metamorphic relations," in *Proc. IEEE 7th Int. Conf. Softw. Testing, Verification Validation Workshops*, Mar. 2014, pp. 237–238.

[58] C. Murphy, G. Kaiser, and L. Hu, "Properties of machine learning applications for use in metamorphic testing," Dept. Comput. Sci., Columbia Univ., New York NY, USA, Tech. Rep. CUCS-011-08, 2008.

[59] U. Kanewala, J. M. Bieman, and A. Ben-Hur. (2015). Predicting metamorphic relations for testing scientific software: A machine learning approach using graph kernels. *Softw. Testing, Verification Rel.* [Online]. Available: http://dx.doi.org/10.1002/stvr.1594.

[60] J. Zhang, J. Chen, D. Hao, Y. Xiong, B. Xie, L. Zhang, and H. Mei, "Search-based inference of polynomial metamorphic relations," in *Proc. 29th ACM/IEEE Int. Conf. Automat. Softw. Eng.*, 2014, pp. 701–712. [Online]. Available: http://doi.acm.org/10.1145/2642937.2642994.

[61] A. Carzaniga, A. Goffi, A. Gorla, A. Mattavelli, and M. Pezzè, "Cross-checking oracles from intrinsic software redundancy," in *Proc. 36th Int. Conf. Softw. Eng.*, 2014, pp. 931–942. [Online]. Available: http://doi.acm.org/10.1145/2568225.2568287.

[62] A. Goffi, A. Gorla, A. Mattavelli, M. Pezze, and P. Tonella, "Search-based synthesis of equivalent method sequences," in *Proc. 22nd ACM SIGSOFT Int. Symp. Found.Softw. Eng.*, 2014, pp. 366–376. [Online]. Available: http://doi.acm.org/10.1145/2635868.2635888.

[63] A. Goffi, "Automatic generation of cost-effective test oracles," in *Proc. 36th Int. Conf. Softw. Eng.*, 2014, pp. 678–681. [Online]. Available: http://doi.acm.org/10.1145/2591062.2591078.

[64] F. Su, J. Bell, C. Murphy, and G. Kaiser, "Dynamic inference of likely metamorphic properties to support differential testing," in *Proc. 10th Int. Workshop Autom. Softw. Test*, 2015, pp. 55–59. [Online]. Available: http://dl.acm.org/citation.cfm?id=2819261.2819279.

[65] T. Y. Chen, P. Poon, and X. Xie. (2015). METRIC: METamorphic Relation Identification based on the category-choice framework. *J. Syst. Softw.* [Online]. Available: http://www.sciencedirect.com/science/article/pii/S0164121215001624.

[66] T. Y. Chen, P.-L. Poon, S.-F. Tang, and T. H. Tse, "Dessert: a divide-and-conquer methodology for identifying categories, choices, and choice relations for test case generation," *IEEE Trans. Softw. Eng.*, vol. 38, no. 4, pp. 794–809, Jul./Aug. 2012.

[67] A. Gotlieb and B. Botella, "Automated metamorphic testing," in *Proc. 27th Annu. Int. Conf. Comput. Softw. Appl.*, 2003, pp. 34–40. [Online]. Available: http://dl.acm.org/citation.cfm?id=950785.950794.

[68] P. Wu, X. Shi, J. Tang, H. Lin, and T. Y. Chen, "Metamorphic testing and special case testing: A case study," *J. Softw.*, vol. 16, pp. 1210–1220, 2005.

[69] S. Segura, R. M. Hierons, D. Benavides, and A. Ruiz-Cortés. Automated metamorphic testing on the analyses of feature models. *Inform. Softw. Technol.* [Online]. *53(3)*, pp. 245–258. Available: http://www.sciencedirect.com/science/article/pii/S0950584910001904.

[70] P. Wu, "Iterative metamorphic testing," in *Proc. 29th Annu. Int. Comput. Softw. Appl. Conf.*, vol. 1, pp. 19–24, Jul. 2005.

[71] G. Dong, C. Nie, B. Xu, and L. Wang, "An effective iterative metamorphic testing algorithm based on program path analysis," in *Proc. 7th Int. Conf. Quality Softw.*, Oct. 2007, pp. 292–297.

[72] S. Segura, R. M. Hierons, D. Benavides, and A. Ruiz-Cortes, "Automated test data generation on the analyses of feature models: A metamorphic testing approach," in *Proc. 3rd Int. Conf. Softw. Testing, Verification Validation*, Apr. 2010, pp. 35–44.

[73] S. Segura, A. Durán, A. B. Sánchez, D. L. Berre, E. Lonca, and A. Ruiz-Cortés. (2015). Automated metamorphic testing of variability analysis tools. *Softw. Testing, Verification Rel.* [Online]. *25(2)*, pp. 138–163. Available: http://dx.doi.org/10.1002/stvr.1566.

[74] R. Guderlei and J. Mayer, "Statistical metamorphic testing: Testing programs with random output by means of statistical hypothesis tests and metamorphic testing," in *Proc. 7th Int. Conf. Quality Softw.*, Oct. 2007, pp. 404–409.

[75] C. Murphy and G. Kaiser. (2010). Empirical evaluation of approaches to testing applications without test oracles. Columbia Univ. Comput. Sci., New York, NY, Tech. Rep. CUCS-039-10. [Online]. Available: http://hdl.handle.net/10022/AC:P:10525.

[76] C. Murphy, M. S. Raunak, A. King, S. Chen, C. Imbriano, G. Kaiser, I. Lee, O. Sokolsky, L. Clarke, and L. Osterweil, "On effective testing of health care simulation software," in *Proc. 3rd Workshop Softw. Eng. Health Care*, 2011, pp. 40–47. [Online]. Available: http://doi.acm.org/10.1145/1987993.1988003.

[77] C. Murphy, "Using runtime testing to detect defects in applications without test oracles," in *Proc. Found.Softw. Eng. Doctoral Symp.*, 2008, pp. 21–24. [Online]. Available: http://doi.acm.org/10.1145/1496653.1496659.

[78] C. Murphy, K. Shen, and G. Kaiser, "Using JML runtime assertion checking to automate metamorphic testing in applications without test oracles," in *Proc. Second Int. Conf. Softw. Testing Verification Validation*, 2009, pp. 436–445.

[79] (2016). "Java Modeling Language (JML). [Online]. Available: http://www.jmlspecs.org.

[80] C. Murphy, G. Kaiser, L. Hu, and L. Wu, "Properties of machine learning applications for use in metamorphic testing," in *Proc. Int. Conf. Softw. Eng. Knowl. Eng.*, 2008, pp. 867–872.

[81] C. Murphy, K. Shen, and G. Kaiser, "Automatic system testing of programs without test oracles," in *Proc. 18th Int. Symp. Softw. Testing Anal.*, 2009, pp. 189–200. [Online]. Available: http://doi.acm.org/10.1145/1572272.1572295.

[82] H. Zhu, "Jfuzz: A tool for automated Java unit testing based on data mutation and metamorphic testing methods," in *Proc. 2nd Int. Conf. Trustworthy Syst. Appl.*, Jul. 2015, pp. 8–15.

[83] W. K. Chan, S. C. Cheung, and K. R. P. Leung, "Towards a metamorphic testing methodology for service-oriented software applications," in *Proc. 5th Int. Conf. Quality Softw.*, Sep. 2005, pp. 470–476.

[84] W. K. Chan, S. C. Cheung, and K. R. P. H. Leung. (2007). A metamorphic testing approach for online testing of service-oriented software applications. *Int. J. Web Serv. Res.* [Online]. *4(2)*, pp. 61–81. Available: http://dblp.uni-trier.de/db/journals/jwsr/jwsr4.html#ChanCL07.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.

[85] C. Sun, G. Wang, B. Mu, H. Liu, Z. Wang, and T. Y. Chen. (2012, Jan.). A metamorphic relation-based approach to testing web services without oracles. *Int. J. Web Serv. Res.* [Online]. *9(1)*, pp. 51–73. Available: http://dx.doi.org/10.4018/jwsr.2012010103.

[86] C. Castro-Cabrera and I. Medina-Bulo, "An approach to metamorphic testing for WS-BPEL compositions," in *Proc. Int. Conf. e-Bus.*, Jul. 2011, pp. 1–6.

[87] C. Castro-Cabrera and I. Medina-Bulo, "Application of metamorphic testing to a case study in web services compositions," in *E-Business and Telecommunications* (ser. Communications in Computer and Information Science), M. Obaidat, J. Sevillano, and J. Filipe, Eds., Berlin, Germany: Springer, 2012, vol. 314, pp. 168–181. [Online]. Available: http://dx.doi.org/10.1007/978-3-642-35755-8_13.

[88] OASIS: Web Services Business Process Execution Language. (2007, Apr.). [Online]. Available: 2.0. http://docs.oasis-open.org/wsbpel/2.0/OS/wsbpel-v2.0-OS.html.

[89] Z. Q. Zhou, T. H. Tse, F.-C. Kuo, and T. Y. Chen, "Automated functional testing of web search engines in the absence of an oracle," Dept. Comput. Sci., The Univ. Hong Kong, Tech. Rep. TR-2007-06, 2007.

[90] Z. Q. Zhou, S. Xiang, and T. Y. Chen, "Metamorphic testing for software quality assessment: A study of search engines," *IEEE Trans. Softw. Eng.*, 2015, Doi: 10.1109/TSE.2015.2478001.

[91] J. Mayer and R. Guderlei, "On random testing of image processing applications," in *Proc. 6th Int. Conf. Quality Softw.*, Oct. 2006, pp. 85–92.

[92] R. Guderlei and J. Mayer. (2007). Towards automatic testing of imaging software by means of random and metamorphic testing. *Int. J. Softw. Eng. Knowl. Eng.* [Online]. *17(06)*, pp. 757–781. Available: http://www.worldscientific.com/doi/abs/10.1142/S0218194007003471.

[93] W. K. Chan, J. C. F. Ho, and T. H. Tse, "Piping classification to metamorphic testing: An empirical study towards better effectiveness for the identification of failures in mesh simplification programs," in *Proc. 31st Annu. Int. Comput. Softw. Appl. Conf.*, vol. 1, pp. 397–404, Jul. 2007.

[94] W. K. Chan, J. C. F. Ho, and T. H. Tse. (2010, Jun.). Finding failures from passed test cases: Improving the pattern classification approach to the testing of mesh simplification programs. *Softw. Testing, Verification Rel. J.* [Online]. *20(2)*, pp. 89–120. Available: http://dx.doi.org/10.1002/stvr.v20:2.

[95] R. Just and F. Schweiggert, "Evaluating testing strategies for imaging software by means of mutation analysis," in *Proc. Int. Conf. Softw. Testing, Verification Validation Workshops*, Apr. 2009, pp. 205–209.

[96] T. Jameel, L. Mengxiang, and C. Liu, "Test oracles based on metamorphic relations for image processing applications," in *Proc. 16th IEEE/ACIS Int. Conf. Softw. Eng., Artif. Intell., Netw. Parallel/Distrib. Comput.*, Jun. 2015, pp. 1–6.

[97] T. H. Tse, S. S. Yau, W. K. Chan, H. Lu, and T. Y. Chen, "Testing context-sensitive middleware-based software applications," in *Proc. 28th Annu. Int. Comput. Softw. Appl. Conf.*, vol. 1, pp. 458–466, Sep. 2004.

[98] W. K. Chan, T. Y. Chen, H. Lu, T. H. Tse, and S. S. Yau, "A metamorphic approach to integration testing of context-sensitive middleware-based applications," in *Proc. 5th Int. Conf. Quality Softw.*, Sep. 2005, pp. 241–249.

[99] W. K. Chan, T. Y. Chen, S. C. Cheung, T. H. Tse, and Z. Zhang, " Towards the testing of power-aware software applications for wireless sensor networks," in *Ada Europe 2007 - Reliable Software Technologies*, N. Abdennadher and F. Kordon, Eds., Berlin, Germany: Springer, 2007, vol. 4498, pp. 84–99. [Online]. Available: http://dx.doi.org/10.1007/978-3-540-73230-3_7.

[100] F.-C. Kuo, T. Y. Chen, and W. K. Tam, "Testing embedded software by metamorphic testing: A wireless metering system case study," in *Proc. IEEE 36th Conf. Local Comput. Netw.*, Oct. 2011, pp. 291–294.

[101] M. Jiang, T. Y. Chen, F.-C. Kuo, and Z. Ding, "Testing central processing unit scheduling algorithms using metamorphic testing," in *Proc. 4th IEEE Int. Conf. Softw. Eng. Service Sci.*, May 2013, pp. 530–536.

[102] K. Y. Sim, W. K. S. Pao, and C. Lin, "Metamorphic testing using geometric interrogation technique and its application," in *Proc. 2nd Int. Conf. Elect. Eng./Electron., Comput., Telecommun., Inform. Technol.*, 2005, pp. 91–95.

[103] T. Y. Chen, F.-C. Kuo, H. Liu, and S. Wang, " Conformance testing of network simulators based on metamorphic testing technique," in *Formal Techniques for Distributed Systems*, D. Lee, A. Lopes, and A. Poetzsch-Heffter, Eds., Berlin, Germany: Springer, vol. 5522, pp. 243–248, 2009. [Online]. Available: http://dx.doi.org/10.1007/978-3-642-02138-1_19.

[104] OMNeT++ system. (2015, Dec.). [Online]. Available: http://www.omnetpp.org.

[105] J. Ding, T. Wu, D. Xu, J. Q. Lu, and X. Hu, "Metamorphic testing of a Monte Carlo modeling program," in *Proc. 6th Int. Workshop Autom. Softw. Test*, 2011, pp. 1–7. [Online]. Available: http://doi.acm.org/10.1145/1982595.1982597.

[106] A. Nuñez and R. M. Hierons. (2014). A methodology for validating cloud models using metamorphic testing. *Ann. Telecommun. – annales des télécommunications* [Online]. pp. 1–9. Available: http://dx.doi.org/10.1007/s12243-014-0442-7.

[107] A. Nuñez, J. L. Vazquez-Poletti, A. C. Caminero, G. G. Castañe, J. Carretero, and I. M. Llorente. (2012). icancloud: A flexible and scalable cloud infrastructure simulator. *J. Grid Comput.*, 10(1), pp. 185–209. [Online]. Available: http://dx.doi.org/10.1007/s10723-012-9208-5.

[108] P. C. Cañizares, A. Núñez, M. Núñez, and J. J. Pardo, "A methodology for designing energy-aware systems for computational science," *Procedia Comput. Sci.*, vol. 51, pp. 2804–2808, 2015, (Int. Conf. Comput. Sci.) [Online]. Available: http://www.sciencedirect.com/science/article/pii/S1877050915012466.

[109] X. Xie, J. Ho, C. Murphy, G. Kaiser, B. Xu, and T. Y. Chen, "Application of metamorphic testing to supervised classifiers," in *Proc. 9th Int. Conf. Quality Softw.*, Aug. 2009, pp. 135–144.

[110] J. E. Gewehr, M. Szugat, and R. Zimmer. (2007, Feb.). Bioweka—extending the weka framework for bioinformatics. *Bioinformatics*. [Online]. *23(5)*, pp. 651–653. Available: http://dx.doi.org/10.1093/bioinformatics/btl671.

[111] Z. Jing, H. Xuegang, and Z. Bin. (2011). An evaluation approach for the program of association rules algorithm based on metamorphic relations. *J. Electron. (China)* [Online]. *28(4-6)*, pp. 623–631. Available: http://dx.doi.org/10.1007/s11767-012-0743-9.

[112] L. L. Pullum and O. Ozmen, "Early results from metamorphic testing of epidemiological models," in *Proc. ASE/IEEE Int. Conf. BioMedical Comput.*, Dec. 2012, pp. 62–67.

[113] A. Ramanathan, C. A. Steed, and L. L. Pullum, "Verification of compartmental epidemiological models using metamorphic testing, model checking and visual analytics," in *Proc. ASE/IEEE Int. Conf. BioMed. Comput.*, Dec. 2012, pp. 68–73.

[114] S. Beydeda, "Self-metamorphic-testing components," in *Proc. 30th Annu. Int. Comput. Softw. Appl. Conf.*, vol. 2, pp. 265–272, Sep. 2006.

[115] X. Lu, Y. Dong, and C. Luo, "Testing of component-based software: A metamorphic testing methodology," in *Proc. Int. Conf. Ubiquitous Intell. Comput. Int. Conf. Auton. Trusted Comput.*, Oct. 2010, pp. 272–276.

[116] T. Y. Chen, J. Feng, and T. H. Tse, "Metamorphic testing of programs on partial differential equations: A case study," in *Proc. 26th Int. Comput. Softw. Appl. Conf. Prolonging Softw. Life: Develop. Redevelop.*, 2002, pp. 327–333. [Online]. Available: http://dl.acm.org/citation.cfm?id=645984.675903.

[117] C. Aruna and R. S. R. Prasad, "Metamorphic relations to improve the test accuracy of multi precision arithmetic software applications," in *Proc. Int. Conf. Adv. Comput., Commun. Informat.*, Sep. 2014, pp. 2244–2248.

[118] Q. Tao, W. Wu, C. Zhao, and W. Shen, "An automatic testing approach for compiler based on metamorphic testing technique," in *Proc. 17th Asia Pacific Softw. Eng. Conf.*, Nov. 2010, pp. 270–279.

[119] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," in *Proc. 35th ACM SIGPLAN Conf. Program. Language Des. Implementation*, 2014, pp. 216–226. [Online]. Available: http://doi.acm.org/10.1145/2594291.2594334.

[120] T. Y. Chen, F.-C. Kuo, and Z. Q. Zhou, "An effective testing method for end-user programmers," in *Proc. 1st Workshop End-User Softw. Eng.*, 2005, pp. 1–5. [Online]. Available: http://doi.acm.org/10.1145/1082983.1083236.

[121] K. Y. Sim, C. S. Low, and F.-C. Kuo, "Detecting faults in technical indicator computations for financial market analysis," in *Proc. 2nd Int. Conf. Inform. Sci. Eng.*, Dec. 2010, pp. 2749–2754.

[122] (2016). MetaTrader 4 Trading Terminal. [Online]. Available: http://www.metaquotes.net/en/metatrader4/trading_terminal.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.

[123] S. Yoo, "Metamorphic testing of stochastic optimisation," in *Proc. 3rd Int. Conf. Softw. Testing, Verification, Validation Workshops*, Apr. 2010, pp. 192–201.
[124] Y. Yao, S. Huang, and M. Ji, "Research on metamorphic testing for oracle problem of integer bugs," in *Proc. 4th Int. Conf. Adv. Comput. Sci. Inf. Eng.*, ser. Advances in Intelligent and Soft Computing, 2012, vol. 168, pp. 95–100. [Online]. Available: http://dx.doi.org/10.1007/978-3-642-30126-1_16.
[125] Y. Yao, C. Zheng, S. Huang, and Z. Ren, "Research on metamorphic testing: A case study in integer bugs detection," in *Proc. 4th Int. Conf. Intell. Syst. Design Eng. Appl., 2013*, Nov. 2013, pp. 488–493.
[126] Z. Hui, S. Huang, Z. Ren, and Y. Yao, "Metamorphic testing integer overflow faults of mission critical program: A case study," *Math. Problems Eng.*, vol. 2013, pp. 1–6, 2013.
[127] G. Batra and G. Singh, "An automated metamorphic testing technique for designing effective metamorphic relations," *Contemporary Computing* (ser. Communications in Computer and Information Science), M. Parashar, D. Kaushik, O. Rana, R. Samtaney, Y. Yang, and A. Zomaya, Eds., Berlin, Germany: Springer, 2012, vol. 306, pp. 152–163. [Online]. Available: http://dx.doi.org/10.1007/978-3-642-32129-0_20.
[128] C. Sun, Z. Wang, and G. Wang. (2014). A property-based testing framework for encryption programs. *Frontiers Comput. Sci.* [Online]. *8(3)*, pp. 478–489. Available: http://dx.doi.org/10.1007/s11704-014-3040-y.
[129] C. Aruna and R. S. R. Prasad, "Adopting metamorphic relations to verify non-testable graph theory algorithms," in *Proc. 2nd Int. Conf. Adv. Comput. Commun. Eng.*, May 2015, pp. 673–678.
[130] M. Lindvall, D. Ganesan, R. Ardal, and R. Wiegand, "Metamorphic model-based testing applied on NASA DAT—An experience report," in *Proc. 37th IEEE Int. Conf. Softw. Eng.*, vol. 2, pp. 129–138, May 2015.
[131] T. Y. Chen, T. H. Tse, and Z. Q. Zhou, "Fault-based testing in the absence of an oracle," in *Proc. 25th Annu. Int. Comput. Softw. Appl. Conf.*, 2001, pp. 172–178.
[132] C. Cadar and K. Sen. (Feb 2013). Symbolic execution for software testing: Three decades later. *Commun. ACM* [Online]. *56(2)*, pp. 82–90. Available: http://doi.acm.org/10.1145/2408776.2408795.
[133] I. Erete and A. Orso, "Optimizing constraint solving to better support symbolic execution," in *Proc. Workshop Constraints Softw. Testing, Verification, Anal.*, 2011, pp. 310–315.
[134] T. Y. Chen, T. H. Tse, and Z. Q. Zhou, "Semi-proving: An integrated method based on global symbolic evaluation and metamorphic testing," in *Proc. ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2002, pp. 191–195. [Online]. Available: http://doi.acm.org/10.1145/566172.566202.
[135] G. Dong, S. Wu, G. Wang, T. Guo, and Y. Huang, "Security assurance with metamorphic testing and genetic algorithm," in *Proc. IEEE/WIC/ACM Int. Conf. Web Intell. Intell. Agent Technol.*, vol. 3, pp. 397–401, Aug. 2010.
[136] X. Xie, W. E. Wong, T. Y. Chen, and B. Xu, "Spectrum-based fault localization: Testing oracles are no longer mandatory," in *Proc. 11th Int. Conf. Quality Softw.*, Jul. 2011, pp. 1–10.
[137] X. Xie, W. E. Wong, T. Y. Chen, and B. Xu. (2013). Metamorphic slice: An application in spectrum-based fault localization," *Inform. Softw. Technol.* [Online]. *55(5)*, pp. 866–879. Available: http://www.sciencedirect.com/science/article/pii/S0950584912001759.
[138] (2016). Siemens Suite. [Online]. Available: http://sir.unl.edu/portal/bios/tcas.php#siemens.
[139] Y. Lei, X. Mao, and T. Y. Chen, "Backward-slice-based statistical fault localization without test oracles," in *Proc. 13th Int. Conf. Quality Softw.*, Jul. 2013, pp. 212–221.
[140] Y. Lei, X. Mao, Z. Dai, and C. Wang, "Effective statistical fault localization using program slices," in *Proc. Comput. Softw. Appl. Conf.*, Jul. 2012, pp. 1–10.
[141] P. Rao, Z. Zheng, T. Y. Chen, N. Wang, and K. Cai, "Impacts of test suite's class imbalance on spectrum-based fault localization techniques," in *Proc. 13th Int. Conf. Quality Softw.*, Jul. 2013, pp. 260–267.
[142] C. Aruna and R. S. R. Prasad, "Testing approach for dynamic web applications based on automated test strategies," in *ICT and Critical Infrastructure: Proc. 48th Annu. Convention of Comput. Soc. India—Vol II*, ser. Advances in Intelligent Systems and Computing, vol. 249, pp. 399–410, 2014. [Online]. Available: http://dx.doi.org/10.1007/978-3-319-03095-1_43.
[143] S. Artzi, A. Kiezun, J. Dolby, F. Tip, D. Dig, A. Paradkar, and M. D. Ernst, "Finding bugs in dynamic web applications," in *Proc. Int. Symp. Softw. Testing Anal.*, 2008, pp. 261–272. [Online]. Available: http://doi.acm.org/10.1145/1390630.1390662.
[144] H. Liu, I. I. Yusuf, H. W. Schmidt, and T. Y. Chen, "Metamorphic fault tolerance: An automated and systematic methodology for fault tolerance in the absence of test oracle," in *Proc. 36th Int. Conf. Softw. Eng.*, 2014, pp. 420–423. [Online]. Available: http://doi.acm.org/10.1145/2591062.2591109.
[145] H. Jin, Y. Jiang, N. Liu, C. Xu, X. Ma, and J. Lu, "Concolic metamorphic debugging," in *Proc. IEEE 39th Annu. Comput. Softw. Appl. Conf.*, vol. 2, pp. 232–241, Jul. 2015.
[146] H. Liu, F.-C. Kuo, and T. Y. Chen, "Teaching an end-user testing methodology," in *Proc. 23rd IEEE Conf. Softw. Eng. Educ. Training*, Mar. 2010, pp. 81–88.
[147] S. Yoo and M. Harman. (2012). Regression testing minimization, selection and prioritization: A survey. *Softw. Testing, Verification Rel.* [Online]. *22(2)*, pp. 67–120. Available: http://dx.doi.org/10.1002/stvr.430.
[148] M. D. Ernst, J. Cockrell, W. G. Griswold, and D. Notkin, "Dynamically discovering likely program invariants to support program evolution," in *Proc. 21st Int. Conf. Softw. Eng.*, 1999, pp. 213–224. [Online]. Available: http://doi.acm.org/10.1145/302405.302467.



Sergio Segura received the PhD degree in software engineering (with honours) from Seville University, Sevilla, Spain, where he currently works as a senior lecturer. His research interests include software testing, software variability and search-based software engineering. He has co-authored some highly cited papers as well as tools used by universities and companies in various countries. He also serves regularly as a reviewer for international journals and conferences.



Gordon Fraser received the PhD degree from the Graz University of Technology, Graz, Austria, in 2007, and worked as a postdoctoral researcher at Saarland University, Germany. He is a senior lecturer in computer science at the University of Sheffield, United Kingdom. His research is on improving software quality and programmer productivity. He is the chair of the steering committee of the International Conference on Software Testing, Verification, and Validation, and is regular organising- and programme-committee member of software engineering conferences and workshops. His work on software testing has achieved wide recognition both in research (e.g., DFG and EPSRC grants, ACM SIGSOFT distinguished and best paper awards at FSE, ISSTA, ASE, SSBSE, and GECCO) as well as industry (e.g., Google Focused Research Award or Microsoft Software Engineering Innovation Foundation Award).

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.



Ana B. Sánchez is currently working toward the PhD degree at the Applied Software Engineering Research Group, University of Seville where she received the MSc degree. Her research focuses on automating the detection of faults in highly configurable systems.



Antonio Ruiz-Cortés received the PhD and MSc degrees in computer science from the University of Seville, Spain. He is an accredited full professor and the head of the Applied Software Engineering Research Group, University of Seville. His research interests include the areas of service oriented computing, software variability, software testing, and business process management.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/publications/dlib.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 11:06:17 UTC from IEEE Xplore. Restrictions apply.

2023 IEEE International Conference On Artificial Intelligence Testing (AITest) | 979-8-3503-3629-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/AITest58265.2023.00014

Metamorphic Testing For Machine Learning: Applicability, Challenges, and Research Opportunities

Faqeer ur Rehman
Gianforte School of Computing
Montana State University
Bozeman, USA
faqeer.rehman@student.montana.edu

Madhusudan Srinivasan
University of Nebraska
Omaha, USA
msrinivasan@unomaha.edu

***Abstract*—The wide adoption and growth of Machine Learning (ML) have made tremendous advancements in revolutionizing a number of fields i.e., manufacturing, transportation, bio-informatics, and self-driving cars. Its ability to extract patterns from a large set of data and then use this knowledge to make future predictions is beyond the human imagination. However, the complex calculations internally performed in them make these systems suffer from the oracle problem; thus, hard to test them for identifying bugs in them and enhancing their quality. An application not properly tested can have disastrous consequences in the production environment. Metamorphic Testing (MT) has been widely accepted by researchers to address the oracle problem in testing both supervised and unsupervised ML-based systems. However, MT has several limitations (when used for testing ML) that the existing literature lacks in capturing them in a centralized place. Applying MT to test ML-based critical systems without prior knowledge/understanding of those limitations can cost organizations a waste of time and resources. In this study, we highlight those limitations to help both the researchers and practitioners to be aware of them for better testing of ML applications. Our efforts result in making the following contributions in this paper, i) providing insights into various challenges faced in testing ML-based solutions, ii) highlighting a number of key challenges faced when applying MT to test ML applications, and iii) presenting the potential future research opportunities/directions for the research community to address them.**

***Index Terms*—Metamorphic Testing, Machine Learning, Metamorphic Testing for Machine Learning, Testing Machine Learning, Challenges in Testing Machine Learning, Applicability of Metamorphic Testing in Machine Learning, Challenges of Metamorphic Testing**

I. Introduction

Machine Learning (ML) has progressed dramatically and is expanding its roots in almost every field of life i.e., manufacturing, marketing, healthcare, education, autonomous vehicles, and collaborative robots. Its high success and progress are driven by the availability of high computing resources, the abundance of data, and the development of efficient algorithms.

Since ML models are an integral part of various critical systems, they must be properly verified and validated before their deployment to the production environment. An ML-based system that lacks the existence of a systematic testing pipeline(s) may end up in resulting both financial and human loss. As evidence, history witnessed on its face when in Mar 2018, a woman in Arizona became a victim of Uber's self-driving car, resulting in her death [1]. This sad incident forced Uber to stop its services (i.e., testing self-driving vehicles); thus, resulting in huge financial and reputational losses for the company. Similarly, in Oct 2018, it was reported that the Amazon AI-based recruiting software became biased (favoring men over women) during the selection of candidates for the company-wide open positions, resultantly forcing Amazon to scrap it [2]; thus, causing wastage of time, cost, and the human resources. Last but not the least, Microsoft's AI-based chatbot (named Tay) became racist and generated offensive/hurtful tweets within its first 24 hours after its deployment [3]; thus, pushing the IT giant to immediately take it off, which seriously hurt its reputation around the globe.

The aforementioned failures strongly urge researchers to focus more on the quality assurance of ML models and propose effective testing strategies to better verify their correctness and robustness. One of the challenges faced in testing the correctness and robustness of such computationally complex AI-based models is their large input space for which they must be verified. This ultimately put these complex models under the category of non-testable programs, known as suffering from the *Oracle Problem* [4]. To better understand the problem domain, consider there is a Deep Neural Network (DNN) that takes executable files as input and predicts whether the files are malicious or not. Suppose, for the given executable file the model predicts (along with some probability score) that it belongs to the category of 'malicious' class label. However, the important question that arises here is *'how can we verify and test that the output produced by this model is correct and reliable?'* One of the potential solutions is to use human involvement i.e., a security expert first creates a virtual machine, configures the testing environment, and then executes the given input file to observe whether it is really malicious or not, and at the end using this knowledge to verify the output

979-8-3503-3629-0/23/$31.00 ©2023 IEEE
DOI 10.1109/AITest58265.2023.00014

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 10:54:03 UTC from IEEE Xplore. Restrictions apply.

produced by the ML model under test. This method is very expensive, laborious, infeasible, and error-prone when we have thousands of files to verify. This further raises an important question to answer *'how to test such complex non-testable programs that suffer from the oracle problem?'*

In comparison to ML, Software Engineering (SE) is a much more mature field having well-known, experimented, and trusted software testing techniques. In this context, SE4ML is a new trending research area in which one of the main focus areas is to leverage the existing SE testing techniques for better quality assurance of ML-based models [5] [6]. Among the traditional SE testing techniques, the most effective and common testing technique that we found to alleviate the oracle problem in testing ML-based models is the Metamorphic Testing (MT) approach [7]. The MT technique does not require the availability of ground truth to verify the correctness of individual input. Instead, it uses the concept of multiple executions (i.e., the source and follow-up executions) to check whether the necessary property captured in the Metamorphic Relation (MR) is satisfied or not. The buggy behavior of the program is said to be detected if the program fails to adhere to the relation specified in the MR(s).

One may argue that among the traditional software testing techniques, the MT technique is not the only testing technique that addresses the Oracle problem. Instead, other popular testing techniques like *Differential Testing* and *N-version programming* could be used to avoid the extra cost (of multiple executions) involved in using MT. This reasoning is valid and unquestionable but one of the major limitations that make these two testing techniques unrealistic/infeasible in a real environment is that they require multiple copies/versions of the same model under test which is not always available in real life. This makes the MT technique more realistic, and the feasible choice among researchers to alleviate the oracle problem in testing ML-based models. However, we have identified a number of limitations in MT (when used for testing ML) that the researchers must be aware of them when applying MT for testing ML applications.

It is important to highlight that a wide range of research studies are available in which MT has been applied in testing both supervised and unsupervised ML applications but to the best of our knowledge, we are unable to find any work that guides/informs the researchers about the open challenges faced in using MT for testing ML-based models and the potential future research opportunities/directions in a centralized place. Applying MT to test ML-based critical systems without prior knowledge/understanding of those limitations can cost organizations a waste of time and resources. The work of Chen et al. [14] does highlight some challenges faced in MT (at a broader level) but their work is solely focused on MT in general, neither discusses those challenges in the context of the emerging field of ML nor shows whether they are applicable in ML domain or not, which is equally the motivator for this research work. We aim to address this gap by making the following contributions.

- To better understand the problem space, we first provide insights into the general challenges encountered in testing ML-based solutions.
- We highlight the limitations/challenges faced in applying MT to test ML-based systems.
- We recognize those limitations/challenges as potential research opportunities and provide further directions to the research community to address them.

We organize the rest of the paper as follows. Section II discusses the related literature review in which we mainly focus on highlighting the applicability of MT in testing both supervised and unsupervised ML. Section III discusses the challenges faced in testing ML applications, whereas, Section IV highlights the limitations/challenges faced in applying MT for testing ML applications along with potential future research opportunities. In the end, we conclude our study in Section V.

II. Literature Review

Metamorphic Testing (MT) [7] is a property-based testing technique that identifies and targets the necessary properties of the program under test. It identifies *relations* known as Metamorphic Relations (MRs) that use a concept of multiple executions (known as *source* and *follow-up* executions) to check whether the necessary characteristic/property expected from the program under test is satisfied or not. The violation of necessary MR will depict that the program under test has some potential bug. Consider a program used for calculating variance among the data points. One of the MRs can be 'if we change the order of data points, it should not change the final calculated variance for the given data points'. Similarly, 'multiplying all the data points with -1 should keep the output consistent'.

The tremendous advancement of ML in almost every field of life is inevitable. ML has been largely embraced by researchers and is used in a variety of domains i.e., to test compilers [10], code obfuscators [14], protein function prediction tools [12], data analytics programs [9], machine translators [11], and autonomous vehicles [13]. However, their computational complexity and large input space make it a challenging task for software testers to devise systematic testing techniques for better testing them and improving their quality. In this context, MT has been shown to be a promising testing technique that has been widely applied in testing such computationally complex programs. To better understand the applications of MT in ML research space, we present a brief literature review targeting the testing of the most common types of ML i.e, supervised and unsupervised ML.

A. MT Applicability in Testing Supervised ML

Zhou et al. [13] used MT for the identification of faults in Apollo's perception module and reported the identified inconsistencies eight days before the incident when Uber's autonomous vehicle hit and killed the pedestrian. Shahri et al. [12] proposed several MRs that perform different transformations to the input (i.e., protein sequences) for performing the quality assurance of a protein function prediction software.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 10:54:03 UTC from IEEE Xplore. Restrictions apply.

Moreira et al. [17] leveraged MT in testing six acoustic scene classifiers by performing sample and attributes permutations. The results obtained show that the proposed approach is able to identity both verification and validation issues in the applications under test. Ma et al. [18] proposed MT-based approach for addressing the oracle problem in testing a fake news detection system. However, there is no clear evidence showing the identification of implementation bugs in the program under test. Jiang et al. [16] proposed a set of 20 MRs and checked their effectiveness by validating four sentiment analysis systems. Further, the authors tried to explore the effect of *false satisfactions* on MT effectiveness and their potential causes when testing sentiment analysis systems. Ma et al. [15] took an advantage of using MT to test a classification model trained on COVID-19 CT images which predict whether the patient has been diagnosed with COVID or not. The authors proposed 8 MRs to test and detect inconsistencies in the COVID-19 diagnosis classification program under investigation. Rehman et al. [19] [20] work uses a combination of MT and statistical concepts for verification of MRs to identify the induced buggy behaviors in the network intrusion detection systems.

B. MT Applicability in Testing Unsupervised ML

Research shows that the researchers are focusing more on using MT for testing supervised ML applications but much-limited research work is available in the space of leveraging MT for testing unsupervised ML algorithms. To the best of our knowledge, we are able to find just four research papers in which MT has been used in testing unsupervised ML algorithms [22] [23] [24] [25]. Yang et al. [22] proposed a list of MRs and used them to test (from a validation perspective) a clustering algorithm (i.e., k-means) implemented in WEKA tool. Similarly, Xie et al. [23] also focused on testing WEKA tool in which they leveraged the MT technique for validating a set of clustering algorithms (i.e., belonging to the category of Prototype-based systems, Hierarchy-based systems, and Density-based systems) to check whether the algorithms under investigation adhere to the user expectations or not. Rehman et al. [24] [25] took MT one step further and proposed a large set of diverse MRs for testing python based library (sci-kit learn) in which they tested k-means, Agglomerative clustering, and DBSCAN algorithms from both the verification (targeting the necessary characteristics of the algorithms under test) and validation perspective (targeting the user expectations expected from the algorithms under test). The limited research available urges researchers to make further significant contributions in this research space.

III. Challenges in Testing Machine Learning Programs

Before discussing the open challenges faced in using MT for testing ML-based applications, we first put some light on highlighting notable challenges posed by testing ML-based systems, which are as follows.

1) Requirements Specification Document: In comparison to traditional software, preparing a requirements specification document for the ML model is a challenging task. The complex concepts involved in the implementation of ML algorithms are the driving force that prevents customers/business analysts to capture the clear requirements expected from the final product. The lack of a specification document ultimately makes it a harder task for the software tester to verify the acceptance criteria for the ML model under test.

2) Strong Cohesiveness Among ML Components: The ML model can broadly be classified into three components, i) the data, ii) the application code written by the ML developer, and iii) frameworks/libraries (i.e., Pytorch, Sci-kit learn, etc.) but the program available for testing/deployment is the final trained ML model. The trained ML model is treated as a single unit, making it a challenging task for the software testers to break down the system into smaller components and then test each component individually for better fault localization and quality improvement.

3) Large Input Space: When testing traditional software systems, a software tester may have some information beforehand to test the program i.e., the availability of a list of valid and invalid users to test the authentication mechanism of the program under test. However, in testing ML-based solutions, a software tester grapples with understanding the large input space e.g., features like price, location, and weather conditions may contain any possible valid value. Hence, validating the ML model for such a large input space is a challenging task. This raises an important question to answer *'how can a well-representative and limited set of data be generated/used to better test the ML solutions within limited resource constraints'*, which is equally an open research problem.

4) Decision Logic: When developing traditional software, a software engineer hard-code the rules and the business logic is fixed to perform the desired functionality. However, in ML-based models the software engineers do not hard-code the decision logic, instead, these rules are learned from the training data. This brings an extra challenge for the software tester to test ML-based applications in which decision logic changes based on the data provided to them.

5) Testing Multiple Components: When testing traditional software, the main component for testing is the application code. However, ML-based models have multiple components to test that include, i) the data, ii) the code written by the software engineer, and iii) the underlying framework/library i.e., Pytorch, and Sci-kit learn. Testing all these components requires extra resources and introduces its own challenges for the software testers to perform their testing within the limited available time and resources.

6) Oracle Problem: The ML models have unique characteristics i.e., they have large input space for which they need to be tested. Specifying/identifying the oracle (in the form of a class label) for such a large input space makes these systems suffer from the oracle problem [4].

7) Non-Deterministic Results: Generally, the business rules written in traditional software output deterministic results

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 10:54:03 UTC from IEEE Xplore. Restrictions apply.

across multiple runs. However, in ML-based models i.e., Neural Networks, each time the model is trained on a fixed set of training data, it is not necessary that the model will generate deterministic predictions for the same set of test data. This stochastic nature makes the traditional software testing techniques infeasible to address the oracle problem in testing such types of computationally complex models.

8) Black-box Nature: In general, complex ML-based models i.e., Deep Neural Networks, are treated as a black-box model because of the computational complexity involved in their training and making predictions for unseen data. This makes them less transparent, difficult to interpret, and harder to test. ML/AI Explainability is another hot research area where researchers focus on addressing such problems at scale.

9) Misleading Performance Metrics: In supervised ML, we strongly rely on using performance metrics to check the prediction power of the model under test. However, it is important to highlight that sometimes these performance metrics i.e., accuracy, may be misleading, especially, when the model under test is producing high accuracy but does contain a hidden implementation bug [26]. Thus, it is important to mention that in addition to using performance metrics, the researchers should also focus on proposing effective testing strategies for better testing of ML programs that are now getting an integral part of mission-critical systems.

IV. Challenges/Limitations of MT in Testing ML and Future Research Opportunities/Directions

The existing literature shows that researchers are widely using the Metamorphic Testing (MT) technique for better testing of a large range of ML applications and is undoubtedly considered one of the most effective testing techniques to address the oracle problem in testing ML-based systems. However, like every testing technique, the MT technique has its own limitations/challenges faced in testing ML but we are unable to find any work that highlights those limitations/challenges to better understand them and provide directions for making future research contributions. Applying MT to test ML-based critical systems without prior knowledge/understanding of those limitations/challenges can cost organizations a waste of time and resources. This motivated us to focus on addressing this gap that result in this study which captures the following list of limitations/challenges along with future research directions. We hope that this research work will enable researchers and practitioners to make well-informed decisions when they aim to apply MT for testing ML.

1) Testing Neural Networks: One of the challenges faced by researchers in finding implementation bugs in Neural Networks (NNs) based models is their non-deterministic nature in making predictions for the same test inputs (due to random assignment of network weights during the training phase) [26] [27]. It is important to highlight that the traditional MT works well for testing traditional ML-based models i.e., SVM, Decision trees, and K-NN but it becomes infeasible when the program under test (i.e., NN-based model) has stochastic nature in the final outputs and the software tester has used equality condition to verify the results (obtained for the source and follow-up inputs). Therefore, in such scenarios, the classical MT technique can't be used directly to verify the MRs for such ML models. The most recent work [19] [20] integrates MT with statistical methods to verify the proposed MRs for testing Deep Neural Networks (DNNs). However, one of the limitations of their approach is the high computational cost because in order to statically verify the MRs, the NN model under test needs to be trained multiple times. A possible research direction can be the optimization of their statistical MT approach to lower the computational cost. As an example, one of the possible approaches to extend their work can be using a less number of well-representative features/samples (that will lower the training time for large ML models) and showing their fault detection effectiveness using empirical studies.

2) Consistent Outputs Can Lead To False Acceptance of MR(s): The MT technique identifies the violation of MR if the output obtained for the source and follow-up inputs are not consistent. The important question to answer is *what if the buggy program version outputs the wrong but consistent outputs for both the source and follow-up inputs?* Such cases will result in high false positives, wrongly assuming that the MR(s) have been satisfied, which ultimately can lead to disastrous consequences when used in testing mission-critical systems. Therefore, we recommend organizations use MT as a supplementary testing technique on top of using other testing techniques, especially, when the cost of a false positive is very high. Apart from that, knowing about such limitations of MT will help organizations in making much informed decisions. Some potential future directions in this area can be to identify which other criteria (other than relying on the final outputs/predictions) can be used to verify the MRs for the program under test. As an example, Rehman et al. [24], uses multiple criteria (i.e., final outputs, cluster centroids, and nearest neighbors) to verify the MRs for testing k-means clustering algorithm. Similarly, instead of relying on the final output, more granular details i.e., the features used by the ML model (for making predictions) can also be used to verify the MRs.

3) Large Set of MRs can Make MT Infeasible: Due to the critical nature of ML-based applications, it is always desirable to have a large set of MRs for their effective testing. However, a large set of MRs may not be applicable in a regression testing environment when the organizations have limited testing resources. A future research direction can be to identify which characteristics of ML-system under test can be utilized for effective MRs prioritization and minimization; thus, saving organizational testing cost and resources. A potential extension to it can be showing the generalization of the prioritized MRs for testing a large number of ML models.

4) Human Involvement in Identification of MRs: One of the challenges faced in using MT for testing ML applications is the human involvement in the identification of MRs for the ML model under test. This is a very challenging task because of the complex nature of ML algorithms which requires their deep

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 10:54:03 UTC from IEEE Xplore. Restrictions apply.

understanding; thus, making it a harder task for a software tester to test them. A possible research direction can be to leverage ML-based techniques for the automatic identification and prediction of suitable MRs for the ML model under test. However, addressing this problem using supervised ML may require generating a sufficient amount of data and then accurately labeling them.

5) Unrealistic Test Cases: MT is not only a testing technique but can also be used for generating new test cases, i.e., the follow-up execution phase of MR helps in the generation of new data. However, in some cases, the generated test cases may not be realistic enough to capture the real-life scenarios encountered by the program under test in a production environment. An example of such MR can be i.e., the multiplication of features (i.e., date of birth, IP address, etc.) with some constant may result in the generation of new follow data that may never be encountered/exist in a real life. Therefore, such MRs will serve no/less fruitful purpose at the cost of wasting organizational testing resources. A potential future research direction can be to identify and propose new approaches that can help in generating more realistic and representative test cases (for both structured and unstructured data) for the MRs that will be encountered by the model(s) in a real environment.

6) MR's Fault Detection Effectiveness: To show the effectiveness of proposed MRs, researchers normally use the *Mutation Score* i.e. the percentage of faults detected, and then use this measure to identify the MRs that are most effective in uncovering the injected bugs [26] [27]. This process requires the execution of both the source and follow-up test cases (to calculate the mutation score) which is a costly and resource-intensive task, especially, when the model under test is complex and takes a few hours to train itself for single program execution. A potential future direction can be using ML-based techniques for the prediction of fault detection effectiveness of each MR (before executing them) and then using this knowledge to predict and execute the MRs that are 'more effective'; thus, saving the organizational's testing cost in preventing the execution of MRs that are least effective.

7) High Execution Cost: In order to verify a single MR, the ML model under test needs to be trained multiple times i.e., one for the source inputs and one for the follow-up inputs. Consider, a deep neural network that takes at least a day to train itself on a given dataset. This execution time will get double when a software tester intends to verify a single MR for the model under test. Similarly, to verify a set of 15 MRs, a software tester will need at least a month to execute all of them. Such a high execution cost makes MT inapplicable for testing large-scale complex ML models. A possible solution can be using a subset of well-representative features and instances that can help in reducing the overall training time for verification of MR(s). However, identifying a small set of well-representative features and samples for such a problem itself is another interesting open research problem.

8) Inability of MRs to Identify The Location of Bug(s): When applying MT to test ML-based models, a violation of MR just reveals the existence of a potential bug in the model under test, however, it tells nothing about its location i.e., where the bug is (either in the data or in the program code)?. A possible approach to address this problem can be, i) using MT for testing each component (i.e., data and code) separately, and ii) integrating MT with other testing techniques i.e., symbolic execution of programs and Unit testing to better help software engineers identify the location of bug in the model under test.

9) Documenting the type of MR(s): In MT, one of the common misconceptions is that the violation of MR will depict the existence of some bug in the program under test. However, it is important to highlight that this may not hold true for every scenario because the violation of MR does not necessarily mean that there is some implementation bug in the program under test [21]. As an example, when testing the k-NN algorithm, the addition of a new data point (during follow-up execution) may change the proportion of data points belonging to some specific class label that can create a tie, and can change the final results. Such a violation of MR will not depict that there is some implementation bug in the algorithm under test. Instead, the algorithm is correctly implemented but the result changed because the algorithm randomly selected one of the class labels to break the tie. Thus, it is very important to explicitly mention/document whether the proposed MR(s) target the verification aspect or the validation aspect of testing the algorithm under investigation. The violation of MR(s) targeting the verification aspect can be characterized as the existence of an implementation bug but the MR(s) targeting the validation aspect does not necessarily mean that there is some bug identified in the program under test. Instead, such a violation captures the deviation of the model from the general user's expectations. If such important information about the MRs is not properly highlighted and documented, this will result in high waste of time and resources for an organization. As an example, imagine a case when a developer spent a significant amount of time and effort to find a bug(s) that in reality was not existing at all because it was not explicitly documented whether the violated MR targets the verification or the validation aspect of testing the model under test.

10) Limited Research in Testing Unsupervised ML: In comparison to supervised ML, we have seen much less research focus on using MT for testing unsupervised ML. To the best of our knowledge, we are able to find just four research papers in which MT has been used in testing unsupervised ML algorithms [22] [23] [24] [25]. This creates a tremendous opportunity for researchers to explore and propose effective testing strategies for using MT in a such less focused area.

11) Lack of MRs Generalization: Another challenge faced when using MT for testing ML programs is the difficulty encountered in the generalization of MRs. An MR applicable in one domain data (i.e., healthcare image data) may not be applicable/scalable to some other similar domain data (i.e., healthcare tabular data). A potential future direction can be proposing a framework that can serve as a guideline for researchers to create/propose MRs that could be generalized to multiple domains effectively.

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 10:54:03 UTC from IEEE Xplore. Restrictions apply.

12) Identification of Sufficient Number of MRs: The literature suggests that different researchers proposed a different number of MRs for the ML models under investigation. Sun et al. [11] proposed a single MR for testing machine translators i.e., Google, and Microsoft, whereas, Ma et al. [15] proposed 8 MRs to test and detect inconsistencies in the COVID-19 diagnosis classification model under investigation. This raises an important question to address *'how many MRs can be considered sufficient to check the correctness and robustness of the ML model under test'*. On one hand, using a large number of MRs may be impractical in a regression testing environment, whereas, on the other hand, a small number of MRs may not be sufficient to effectively test the model under test; thus, finding the optimal number/balance is critical. One potential avenue for future research can be to mine historical data and use that knowledge to suggest/predict the sufficient number of MRs for testing the ML model(s) under test. Furthermore, organizations can tweak their data mining algorithm (for finding a sufficient number of MRs) based on the available resources they have.

V. Conclusion

Metamorphic Testing is undoubtedly a very powerful and simple approach for testing the correctness and robustness of ML-based solutions. However, applying MT to test ML-based critical systems without having prior knowledge about its limitations/challenges may cost organizations a waste of time and resources. In this study, we briefly highlight the applicability of MT in testing ML applications (both supervised and unsupervised ML), its limitations/challenges faced in testing ML solutions, and potential future research opportunities for researchers to address them. We believe that this study will help the researchers and practitioners to be aware of the MT limitations/challenges beforehand (when testing ML systems) and possible consequences during the software testing phase. In the future, we aim to address some of the highlighted limitations/challenges that will further expand the body of knowledge in the emerging field of 'Metamorphic Testing for Machine Learning'.

References

[1] Ohnsman, A. (2018). Lidar maker velodyne 'baffled'by self-driving uber's failure to avoid pedestrian. Forbes, March.

[2] https://www.reuters.com/article/us-amazon-com-jobs-automation-insight/amazon-scraps-secret-ai-recruiting-tool-that-showed-bias-against-women-idUSKCN1MK08G

[3] https://www.theguardian.com/technology/2016/mar/24/tay-microsofts-ai-chatbot-gets-a-crash-course-in-racism-from-twitter

[4] Weyuker, E. J. (1982). On testing non-testable programs. The Computer Journal, 25(4), 465-470.

[5] Kumeno, F. (2019). Sofware engneering challenges for machine learning applications: A literature review. Intelligent Decision Technologies, 13(4), 463-476.

[6] Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., ... & Zimmermann, T. (2019, May). Software engineering for machine learning: A case study. In 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP) (pp. 291-300). IEEE.

[7] Tsong Y Chen, Shing C Cheung, and Shiu Ming Yiu. Metamorphic testing: a new approach for generating next test cases. Technical Report HKUST-CS98-01, Department of Computer Science, Hong Kong University of Science and Technology, Hong Kong, 1998.

[8] Chen, T. Y., Kuo, F. C., Liu, H., Poon, P. L., Towey, D., Tse, T. H., & Zhou, Z. Q. (2018). Metamorphic testing: A review of challenges and opportunities. ACM Computing Surveys (CSUR), 51(1), 1-27.

[9] Jarman, D. C., Zhou, Z. Q., & Chen, T. Y. (2017, May). Metamorphic testing for Adobe data analytics software. In 2017 IEEE/ACM 2nd International Workshop on Metamorphic Testing (MET) (pp. 21-27). IEEE.

[10] Donaldson, A. F., & Lascu, A. (2016, May). Metamorphic testing for (graphics) compilers. In Proceedings of the 1st international workshop on metamorphic testing (pp. 44-47).

[11] Sun, L., & Zhou, Z. Q. (2018, November). Metamorphic testing for machine translations: MT4MT. In 2018 25th Australasian Software Engineering Conference (ASWEC) (pp. 96-100). IEEE.

[12] Shahri, M. P., Srinivasan, M., Reynolds, G., Bimczok, D., Kahanda, I., & Kanewala, U. (2019, April). Metamorphic testing for quality assurance of protein function prediction tools. In 2019 IEEE International Conference On Artificial Intelligence Testing (AITest) (pp. 140-148). IEEE.

[13] Zhou, Z. Q., & Sun, L. (2019). Metamorphic testing of driverless cars. Communications of the ACM, 62(3), 61-67.

[14] Chen, T. Y., Kuo, F. C., Ma, W., Susilo, W., Towey, D., Voas, J., & Zhou, Z. Q. (2016). Metamorphic testing for cybersecurity. Computer, 49(6), 48-55.

[15] Ma, Y., Pan, Y., & Fan, Y. (2022, August). Metamorphic Testing of Classification Program for the COVID-19 Intelligent Diagnosis. In 2022 9th International Conference on Dependable Systems and Their Applications (DSA) (pp. 178-183). IEEE.

[16] Jiang, M., Chen, T. Y., & Wang, S. (2022). On the effectiveness of testing sentiment analysis systems with metamorphic testing. Information and Software Technology, 106966.

[17] Moreira, D., Furtado, A. P., & Nogueira, S. (2020, August). Testing acoustic scene classifiers using Metamorphic Relations. In 2020 IEEE International Conference On Artificial Intelligence Testing (AITest) (pp. 47-54). IEEE.

[18] Ma, Y., Towey, D., Chen, T. Y., & Zhou, Z. Q. (2021, July). Metamorphic Testing of Fake News Detection Software. In 2021 IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC) (pp. 1508-1513). IEEE.

[19] ur Rehman, F., & Izurieta, C. (2021, July). Statistical Metamorphic Testing of Neural Network Based Intrusion Detection Systems. In 2021 IEEE International Conference on Cyber Security and Resilience (CSR) (pp. 20-26). IEEE.

[20] ur Rehman, F., & Izurieta, C. (2021, September). A Hybridized Approach for Testing Neural Network Based Intrusion Detection Systems. In 2021 International Conference on Smart Applications, Communications and Networking (SmartNets) (pp. 1-8). IEEE.

[21] Xie, X., Ho, J. W., Murphy, C., Kaiser, G., Xu, B., & Chen, T. Y. (2011). Testing and validating machine learning classifiers by metamorphic testing. Journal of Systems and Software, 84(4), 544-558.

[22] Yang, S., Towey, D., & Zhou, Z. Q. (2019, May). Metamorphic exploration of an unsupervised clustering program. In 2019 IEEE/ACM 4th International Workshop on Metamorphic Testing (MET) (pp. 48-54). IEEE.

[23] Xie, X., Zhang, Z., Chen, T. Y., Liu, Y., Poon, P. L., & Xu, B. (2020). METTLE: A metamorphic testing approach to assessing and validating unsupervised machine learning systems. IEEE Transactions on Reliability, 69(4), 1293-1322.

[24] Rehman, F. U., & Izurieta, C. (2022, June). MT4UML: Metamorphic Testing for Unsupervised Machine Learning. In 2022 9th Swiss Conference on Data Science (SDS) (pp. 26-32). IEEE.

[25] Rehman, F. U., & Izurieta, C. (2022, August). An Approach For Verifying And Validating Clustering Based Anomaly Detection Systems Using Metamorphic Testing. In 2022 IEEE International Conference On Artificial Intelligence Testing (AITest) (pp. 12-18). IEEE.

[26] Li, Z., Cui, Z., Liu, J., Zheng, L., & Liu, X. (2020, January). Testing neural network classifiers based on metamorphic relations. In 2019 6th International Conference on Dependable Systems and Their Applications (DSA) (pp. 389-394). IEEE.

[27] Dwarakanath, A., Ahuja, M., Sikand, S., Rao, R. M., Bose, R. J. C., Dubash, N., & Podder, S. (2018, July). Identifying implementation bugs in machine learning based image classifiers using metamorphic testing. In Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (pp. 118-128).

Authorized licensed use limited to: University of Auckland. Downloaded on March 30,2026 at 10:54:03 UTC from IEEE Xplore. Restrictions apply.

Check for updates

ACM DIGITAL LIBRARY | Association for Computing Machinery | acm open

Latest updates: https://dl.acm.org/doi/10.1145/3691620.3696020

RESEARCH-ARTICLE

MR-Adopt: Automatic Deduction of Input Transformation Function for Metamorphic Testing

CONGYING XU, Hong Kong University of Science and Technology, Hong Kong, Hong Kong

SONGQIANG CHEN, Hong Kong University of Science and Technology, Hong Kong, Hong Kong

JIARONG WU, Hong Kong University of Science and Technology, Hong Kong, Hong Kong

SHING-CHI CHEUNG, Hong Kong University of Science and Technology, Hong Kong, Hong Kong

VALERIO TERRAGNI, University of Auckland, Auckland, AUK, New Zealand

HENGCHENG ZHU, Hong Kong University of Science and Technology, Hong Kong, Hong Kong

View all

Open Access Support provided by:

Hong Kong University of Science and Technology

University of Auckland

PDF Download
3691620.3696020.pdf
30 March 2026
Total Citations: 6
Total Downloads: 1021

Published: 27 October 2024

Citation in BibTeX format

ASE '24: 39th IEEE/ACM International Conference on Automated Software Engineering
October 27 - November 1, 2024
CA, Sacramento, USA

Conference Sponsors:
SIGAI
SIGSOFT

MR-Adopt: Automatic Deduction of Input Transformation Function for Metamorphic Testing

Congying Xu[a,b], Songqiang Chen[a,b], Jiarong Wu[a,b], Shing-Chi Cheung[a,b*], Valerio Terragni[c], Hengcheng Zhu[a,b], Jialun Cao[a,b*]
[a]The Hong Kong University of Science and Technology, Hong Kong
[b]Guangzhou HKUST Fok Ying Tung Research Institute, China
[c]The University of Auckland, New Zealand

ABSTRACT

While a recent study reveals that many developer-written test cases can encode a reusable Metamorphic Relation (MR), over 70% of them directly hard-code the source input and follow-up input in the encoded relation. Such encoded MRs, which do not contain an explicit input transformation to transform the source inputs to corresponding follow-up inputs, cannot be reused with new source inputs to enhance test adequacy.

In this paper, we propose MR-Adopt (*Automatic Deduction Of inPut Transformation*) to automatically deduce the input transformation from the hard-coded source and follow-up inputs, aiming to enable the encoded MRs to be reused with new source inputs. With typically only one pair of source and follow-up inputs available in an MR-encoded test case as the example, we leveraged LLMs to understand the intention of the test case and generate additional examples of source-followup input pairs. This helps to guide the generation of input transformations generalizable to multiple source inputs. Besides, to mitigate the issue that LLMs generate erroneous code, we refine LLM-generated transformations by removing MR-irrelevant code elements with data-flow analysis. Finally, we assess candidate transformations based on encoded output relations and select the best transformation as the result. Evaluation results show that MR-Adopt can generate input transformations applicable to all experimental source inputs for 72.00% of encoded MRs, which is 33.33% more than using vanilla GPT-3.5. By incorporating MR-Adopt-generated input transformations, encoded MR-based test cases can effectively enhance the test adequacy, increasing the line coverage and mutation score by 10.62% and 18.91%, respectively.

CCS CONCEPTS

• **Software and its engineering → Software testing and debugging**.

KEYWORDS

Software Testing, Metamorphic Testing, Metamorphic Relation, Input Transformation, Code Generation, Large Language Models

ACM Reference Format:
Congying Xu, Songqiang Chen, Jiarong Wu, Shing-Chi Cheung, Valerio Terragni, Hengcheng Zhu, and Jialun Cao. 2024. MR-Adopt: Automatic Deduction of Input Transformation Function for Metamorphic Testing. In *39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24), October 27-November 1, 2024, Sacramento, CA, USA*. ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3691620.3696020

*Shing-Chi Cheung and Jialun Cao are corresponding authors. Emails: scc@cse.ust.hk, jcaoap@cse.ust.hk.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASE '24, October 27-November 1, 2024, Sacramento, CA, USA
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-1248-7/24/10…$15.00
https://doi.org/10.1145/3691620.3696020

1 INTRODUCTION

Metamorphic Testing (MT) is a powerful testing technique to address both the test case generation and the oracle problem [10, 47]. Instead of assessing the outputs of individual inputs, MT validates the behavior of a subject under test (SUT) against Metamorphic Relations (MRs) for the SUT. Each MR defines an *input relation* over a set of related inputs and an *output relation* over the expected outputs for those inputs. One *outstanding benefit* of MT is that once an MR is identified, MT can leverage a wide range of automatically generated inputs (known as *source inputs*) to exercise diverse program behaviors with no need to prepare oracles for individual inputs [65]. MT has achieved success in detecting faults for various software, such as compilers [26, 52], databases [31, 36], machine translation services [5, 64], and question answering systems [63, 69].

Identifying appropriate MRs for a SUT is essential to applying MT. Some studies have focused on MR identification. Earlier approaches either suffer from being labor-intensive and specific to certain domains or pre-defined MR patterns [53, 72, 73] or produce overly generic MRs that are ineffective for testing, as well as recent LLM-based techniques [51, 57]. Recently, Xu et al. [65] report that developers often encode domain knowledge in test cases that exercise MRs. These encoded MRs can be generalized to many new inputs and serve as oracles for more exhaustive testing of the original programs (or programs with similar functionalities), by integrating with automatic input generation techniques [10, 47, 65].

However, Xu et al. [65] show that over 70% of 11,000 MR-encoded test cases (MTCs) in their dataset do not contain explicit input relations. Instead, developers often hard-code the source and follow-up inputs. Figure 1a shows an MR-encoded test case intended to have the follow-up input (`dateB`) one day after the source input (`dateA`), but it simply hard-codes the two inputs. Without an explicit input transformation program, follow-up inputs cannot be directly generated from automatically generated source inputs. This limitation hinders the reuse of valuable encoded MRs to achieve automated MT and enhance test adequacy. **This paper aims to overcome this obstacle by inferring an explicit input relation from a given test case with its hard-coded input pairs.** Specifically,

our goal is to construct an input transformation function that turns a source input into a follow-up input as shown in Figure 1b. With such input transformations, these encoded MRs can apply to a wider range of test inputs to test SUTs more exhaustively (Figure 1c).

This task can be viewed as a programming by example (PBE) problem, where the aim is to synthesize a transformation function that turns a given source input into the corresponding follow-up input. The challenge lies in *correctly interpreting the contextual information, such as the relationship between hard-coded input pairs, output relations, and the properties of the SUT.* Moreover, with only one pair of source and follow-up inputs available as an example [65], there is a risk of generating program overfitted to the given example instead of realizing the true intention, as noted in existing PBE studies [2, 19, 45]. Therefore, *effectively leveraging contextual information is crucial to guide PBE and generate a generalizable input transformation* that aligns with the *semantic* of the encoded MR, ensuring it applies to all potential source inputs with the corresponding output relation.

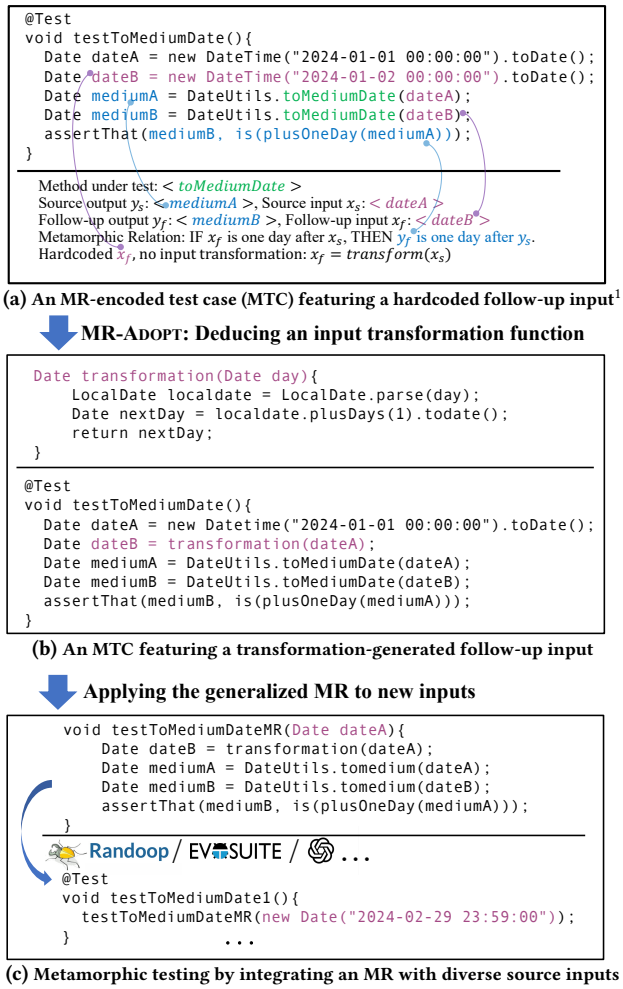
In this paper, we propose MR-Adopt, an approach that leverages large language models (LLMs) to automatically generate input transformation functions for MRs encoded in existing test cases. Trained on extensive code corpora from various domains, LLMs have demonstrated effectiveness in code understanding [18, 35, 39] and generation [7, 14, 23]. Thus, LLMs have the potential to understand contextual information and generate code based on such information. Our insight is to leverage the code understanding ability of LLMs to mine the intention of MR and input relation from the hard-coded test inputs and SUT's function, and take advantage of their code generation ability to produce good input transformation code. We propose three designs to harness LLMs' abilities.

Firstly, we observe that directly providing LLMs with contextual information only results in around 50% generalizable transformations (Section 4.5). This is unsatisfactory. To address this, we need a design that allows LLMs to effectively express the input relation inferred from the hard-coded inputs and generate transformation code. *To realize this goal,* we design MR-Adopt with two phases. In *Phase1*, LLMs perform analogical reasoning [59, 68] on the hard-coded source-followup input pairs to infer new input pairs that obey the same input relation. In *Phase2*, LLMs generate an input transformation function based on (i) the input pair hard-coded by developers and (ii) additional input pairs generated by LLMs in *Phase1*. This design not only enables LLMs to generate code in their familiar setup (where a task description and several examples are provided) [30], but also mitigates the above-mentioned overfitting issue due to the limited number of examples.

Secondly, we found that LLMs often generate task-irrelevant code segments, of which some are even faulty. For example, when tasked with generating a test input, an LLM might include an incorrect assertion statement. MR-Adopt addresses this by refining the LLM-generated code through data-flow analysis, extracting only the relevant code for the given task (i.e., additional input pairs and input transformation generation).

Thirdly, to mitigate the errors in the relevant codes generated by LLMs, MR-Adopt leverages the developer-written output relations

[1]This MR-encoded test case is crafted from `org.hisp.dhis.util` in project DHIS2-core, where long format date is "`yyyy-mm-dd hh:mm:ss`" and medium format date is "`yyyy-mm-dd`".



(a) An MR-encoded test case (MTC) featuring a hardcoded follow-up input[1]

(b) An MTC featuring a transformation-generated follow-up input

(c) Metamorphic testing by integrating an MR with diverse source inputs

Figure 1: Overview of MR-Adopt for Metamorphic Testing

(i.e., assertions) in MTCs as oracles to verify the generated test pairs. We further employ additional inputs to select the most generalizable input transformation as the result.

We evaluated MR-Adopt with 100 developer-written test cases that encode MRs. The results show that MR-Adopt can generate compilable input transformations for 95 MRs, where 72 can generalize to all potential source inputs prepared in our evaluation. MR-Adopt generates 17.28% more compilable transformations and 33.33% more generalizable transformations than directly prompting GPT-3.5. Besides, MR-Adopt-generated transformations produce follow-up inputs for 91.21% source inputs, representing a 122.10% improvement over GPT-3.5 in generating follow-up inputs. Our ablation study indicates that all three designs (i.e., additional input pairs, date-flow analysis based refinement, and output-relation based validation) contribute to MR-Adopt's overall performance, with validation and additional input pairs having the most impact. Furthermore, incorporating MRs with input transformations and new source inputs leads to 10.62% and 18.91% increases in line coverage and mutation score on top of developer-written test cases, demonstrating the practical usefulness of MR-Adopt-generated transformations in enhancing test adequacy.

Contribution. Our work makes the following contribution:

- To the best of our knowledge, we are the first to generate input transformations for MRs encoded in test cases. With the generated input transformations, more encoded MRs can be reused to enhance the test adequacy of SUTs.
- We propose MR-Adopt, an LLM-based approach to deduce input transformation functions. By generating multiple example input pairs, MR-Adopt mitigates overfitting and produces generalizable transformations. It also incorporates a code refinement strategy based on data-flow analysis and a validation strategy to mitigate the faulty irrelevant code generated by LLMs. This design can be applied to other code generation tasks.
- We extensively evaluate MR-Adopt's effectiveness in generating input transformations. Results show that MR-Adopt can generate effective input transformations, where 72% input transformations are generalizable to all prepared source inputs. When integrated with these transformations, the encoded MRs increase line coverage by 10.62% and mutation score by 18.91%.
- We build a dataset of 100 encoded MRs dated after 01-April, 2023, and released it with our replication package on the website [56].

2 PRELIMINARIES

2.1 Metamorphic Testing

Metamorphic Testing (MT) validates a program P using Metamorphic Relations (MRs). An MR $\mathcal{R}$ can be expressed as a logical implication from an ***input relation*** $\mathcal{R}_i$ to an ***output relation*** $\mathcal{R}_o$ [10, 47, 65].

$$\mathcal{R} = \langle \mathcal{R}_i, \mathcal{R}_o \rangle, \text{ where } \mathcal{R}_i\left(x_s, x_f\right) \implies \mathcal{R}_o\left(y_s, y_f\right)$$

$\mathcal{R}_i$ defines the rule to generate an additional test input (known as *follow-up input* x_f) from a given test input (known as *source input* x_s), and $\mathcal{R}_o$ defines the relation between the expected outputs (y_s, y_f) for the source and follow-up inputs, respectively. For example, given a program P implementing the *sine* function, an MR can be defined over an input relation $\mathcal{R}_i$ as $x_f{=}{-}x_s$ ($\forall x_s \in \mathbb{R}$) and an output relation $\mathcal{R}_o$ as $y_f{=}{-}y_s$, based on the property that $P(x){=}{-}P(-x)$ should hold for a correctly implemented *sine* function.

Given an MR $\mathcal{R}$ for a SUT P, conducting MT for P entails the following five steps: (i) constructing a source input x_s, (ii) executing P on x_s and obtaining the source output y_s, (iii) constructing a follow-up input x_f that satisfies $\mathcal{R}_i$, (iv) executing P on x_f and obtaining the follow-up output y_f, and (v) verifying if the two outputs y_s and y_f satisfy the output relation $\mathcal{R}_o$. Typically, a function referred to as the ***input transformation*** is designed to implement $\mathcal{R}_i$ to generate x_f from the given x_s, and x_s can be written by developers or automatically generated (e.g., random testing) [47, 65].

2.2 MR-Encoded Test Cases

MR-encoded test cases (MTCs), introduced by Xu et al. [65], are test cases encoded domain-specific knowledge that suggests useful MRs. These MTCs are prevalent, with over 11,000 identified across 701 open-source projects in their study. An MTC can be considered as an instance of an MR, already implemented with specific source and follow-up input values, invocations of methods under test, and output relation assertions. These output relations are well-coded and serve as oracles, requiring no further refinement to apply to new test inputs. Such encoded MRs can be generalized to new inputs and facilitate automated MT by incorporating automatic input generation techniques.

Consider the example in Figure 1. The encoded MR in this test case is: "IF a date x_1 in long format ("yyyy-mm-dd hh:mm:ss") is one day ahead of another long-format date x_2 ($\mathcal{R}_i$), THEN x_1 in medium format ("yyyy-mm-dd") should also be one day ahead of medium-format x_2 ($\mathcal{R}_o$)". The SUT method `toMediumDate` is executed on the source input `dateA` and the follow-up input `dateB` separately, and the corresponding outputs are verified by `assertThat(mediumB, is(plusOneDay(mediumA))`, which implements $\mathcal{R}_o$.

Such an implemented MR instance can be reused and generalized to many new inputs. However, the follow-up input `dateB` is hardcoded as `"2024-01-02 00:00:00"` instead of being generated from `dateA` by an input transformation program. While the $\mathcal{R}_o$ is explicitly coded, the $\mathcal{R}_i$ remains **implicit**, hidden within the specific source and follow-up input values `dateA` and `dateB`. According to Xu et al.'s study, over 70% of MR-encoded test cases lack explicitly coded $\mathcal{R}_i$ (i.e., input transformations). This limitation prevents these MRs from being directly applied to new inputs automatically generated by existing tools, e.g., Evosuite [17] and Randoop [44]. While these tools are proficient in generating diverse source inputs, they cannot generate input pairs that satisfy an input relation.

In this paper, we address this limitation by deriving an explicit input relation from a given test case and its hardcoded input pairs. Specifically, our goal is to construct an **input transformation function** that converts a source input into a follow-up input, as shown in Figure 1b. With such input transformations, embedded MRs can be reused with a broader range of test inputs (Figure 1c) to exercise more SUT's behaviors, thereby enhancing test adequacy. Additionally, these developer-written MRs serve as reliable oracles for new test generation.

3 MR-ADOPT

Figure 2 presents an overview of MR-Adopt. It takes a pair of source and follow-up inputs, along with its context (i.e., an MR-encoded test case (MTC) and methods under test (MUT)), and outputs an input transformation function. MR-Adopt works in a two-phase pipeline. In the first phase, it generates additional source-follow-up input pairs and uses them as examples to better describe the input relation, which provides useful guidance for the generation of input transformations. In the second phase, it generates input transformation functions based on these example pairs. This setup, familiar to LLMs for code generation tasks, includes a task description and several examples [30], providing more information to effectively guide LLMs in generating generalized transformations.

In each phase, MR-Adopt employs generation, refinement, and validation procedures. In *Phase1*, MR-Adopt first leverages LLMs to generate candidate test input pairs, then refines them based on data-flow analysis to exclude irrelevant code that can contain errors, and finally filters valid input pairs based on output relation assertions. In *Phase2*, MR-Adopt leverages LLMs to generate candidate input transformations based on the input pairs from *Phase1*. These candidate transformations are then refined by removing irrelevant code elements and adding dependencies, and assessed by applying them to additional source inputs. Ultimately, MR-Adopt outputs the most generalizable transformation function.

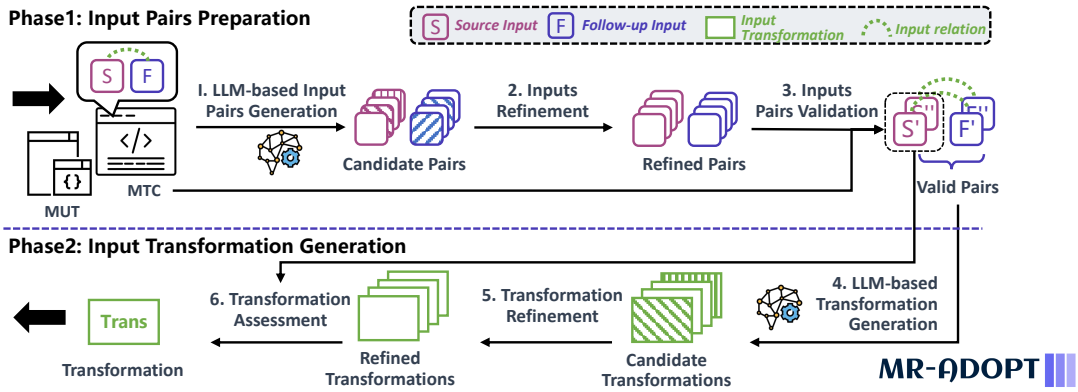


Figure 2: An overview of MR-ADOPT

3.1 *Phase 1:* Input Pair Preparation

3.1.1 Input Pair Generation. MR-ADOPT uses an LLM to produce new source-followup input pairs by imitating a given input pair within the context of an existing MTC (which includes the input pair and developer-written assertions checking the output relation) and corresponding methods under test.

Following the idea of the Chain of Thought strategy [60], MR-ADOPT prompts an LLM in two steps: first to generate source inputs, and then to generate the corresponding follow-up inputs. This step-by-step approach is adopted because we found that LLMs perform better when generating source and follow-up inputs sequentially rather than generating entire input pairs at once. Our source input generation prompt follows recent practices [67, 71], and includes (i) a system message about the role of a Java expert and the task to generate test inputs, (ii) the code of methods under tests (MUTs), (iii) the code of the MR-encoded test case (MTC), and (iv) the output format. Such a prompt provides necessary contextual information (ii and iii) and task description (i and iv) for generating source inputs. Detailed prompt templates and examples are available on MR-ADOPT's website [56]. Listing 1 shows several example source inputs generated by GPT-3.5 with this prompt.

The follow-up input generation prompt is similar to the source input prompt, with the key difference being the addition of previously generated example source inputs to guide the creation of follow-up inputs. We also adjust the task description and output format to instruct LLMs to generate source-followup input pairs, using the original pair in the MTC as a sample. There is a trade-off between providing enough examples to guide the following generation of generalizable transformations and maintaining efficiency in terms of time and cost. To balance these factors, we use five examples. Following recent studies' nucleus sampling [14, 21], MR-ADOPT repeats the above generation process five times with a temperature setting of 0.2 [6, 8]. Listing 2 shows several example input pairs.

3.1.2 Input Pair Refinement. Consistent with observations in existing studies [15], we found that LLM-generated codes often include irrelevant segments, which may introduce errors. For instance, the second input pair in Listing 2 includes an assertion statement `assertEquals(dateTimeB, is(plusOneDay(dateTimeA)));` alongside the desired input pair code. This assertion is irrelevant to constructing `dateA` or `dateB` and incorrectly calls the method

````
## New source input 1:
```java
    Date dateA = new DateTime("2023-12-31 23:59:59").toDate();
```
## New source input 2:
```java
    DateTime dateTimeA = new DateTime("2024-11-30 23:59:59");
    Date dateA = dateTimeA.toDate();
```
## New source input 3:
```java
    Date dateA = new DateTime("2024-01-01 00:00:00").toDate();
```
        ...(other inputs are ommitted)...
````

Listing 1: Examples of LLM-generated source inputs

````
The inferred relation between the source input `dateA` and the follow-up input `dateB` is that `dateA` is one day ahead of `dateB`.
Let's construct the follow-up inputs for the new source inputs:
## New input pair 1:
```java
    Date dateA = new DateTime("2024-12-31 23:59:59").toDate();
    Date dateB = new DateTime("2025-01-01 23:59:59").toDate();
```
## New input pair 2:
```java
    DateTime dateTimeA = new DateTime("2024-11-30 23:59:59");
    Date dateA = dateTimeA.toDate();
    DateTime dateTimeB = new DateTime("2024-12-01 23:59:59");
    Date dateB = dateTimeB.toDate();
    assertEquals( dateTimeB, is(plusOneDay(dateTimeA)) );
```
## New input pair 3:
```java
    Date dateA = new DateTime("2024-01-01 00:00:00").toDate();
    Date dateB = new DateTime("2025-01-01 00:00:00").toDate();
```
        ...(other input pairs are ommitted)...
````

Listing 2: Examples of LLM-generated input pairs

`plusOneDay(Date date)` with a `DateTime` object, resulting in a type mismatch exception.

Our task focuses on constructing source and follow-up inputs. To exclude such irrelevant code and bypass unnecessary errors, we perform a data-flow analysis on the LLM-generated code and build a dependency graph. MR-ADOPT constructs data dependency graphs for source/follow-up inputs by first identifying statements that define source/follow-up inputs, extracting variables used for these definitions, and then recursively tracing these variables' dependencies. Finally, MR-ADOPT isolates the dependent statements related to the source and follow-up inputs and removes other statements. For example, in the second input pair of Listing 2, the source input `dateA` and follow-up input `dateB` depend on objects `dateTimeA`

```
@Test
void testToMediumDate(){
    // LLM-generated new source input
    Date dateA = new DateTime("2024-12-31 23:59:59").toDate();
    // LLM-generated corresponding follow-up input
    Date dateB = new DateTime("2025-01-01 23:59:59").toDate();
    Date mediumA = DateUtils.toMediumDate(dateA);
    Date mediumB = DateUtils.toMediumDate(dateB);
    assertThat(mediumB, is(plusOneDay(mediumA)));
}
```

Listing 3: Validating an LLM-generated input pair

and `dateTimeB`, respectively. Thus, the statements (Lines 11-14) for constructing `dateA`, `dateTimeA`, `dateB`, and `dateTimeB` are considered relevant, while the assertion statement (Line 15) is excluded. Finally, MR-Adopt retains only the statements relevant to constructing source and follow-up inputs, excluding all other irrelevant statements from the LLM-generated code.

3.1.3 Input Pair Validation. The previous refinement step removes the irrelevant code segments generated by LLMs and results in candidate source-followup input pairs. However, a pair of inputs still can be *invalid* if they violate the input relation of an encoded MR. For example, the third input pair shown in Listing 2 is an invalid test pair. The input relation of the embedded MR is that "`dateA` is one day ahead of `dateB`", while an LLM generates an input pair of "2024-01-01" and "2025-01-01". Such test case pairs do not align with the intended input transformation and can mislead the generation of transformations. To address this, we propose a method to discard such invalid input pairs.

We use the output relation of an encoded MR to validate LLM-generated input pairs. Specifically, MR-Adopt executes SUT on generated input pairs and checks the outputs against the output relation, which is an explicit reusable code in the MTC, i.e., the developer-written assertions (Line 7 in Figure 1a). For each input pair, if its outputs of invoking methods under test on the inputs pass the developer-written assertions, MR-Adopt considers it a valid input pair. As shown in Listing 3, if the outputs `mediumA` and `mediumB` pass the assertion (Line 9), the inputs `dateA` and `dateB` are considered valid. This step aims to filter out invalid input pairs generated by LLMs from the example set. It could discard some source-followup input pairs that match the input relation in fact. Factors such as the bugs in a non-regression SUT may lead to false violations and mistaken deletions of these pairs. However, the goal of the first phase is to prepare examples that give more information about the input relation for the second phase. Thus, it does not require *complete* source-followup input pairs.

3.2 *Phase 2:* Transformation Generation

3.2.1 Transformation Generation. In this step, MR-Adopt instructs an LLM to generate candidate input transformation functions for an encoded MR by providing example source-followup input pairs. The examples include the original hard-coded pair and additional pairs generated in *Phase1*. The prompt for transformation generation is similar to the input pair generation prompt (Section 3.1.1), consisting of (i) a system message, (ii) the code of MUT, (iii) example input pairs, (iv) the code of an MTC, and (v) the output format. The difference is that the task shifts from generating source-followup input pairs to generating input transformation functions, whose parameter list and return type are already specified. Detailed prompt template and samples are available on MR-Adopt's website [56].

````
# OUTPUT FORMAT
Generate the transformation function by complementing the following
code skeleton.

    ```java
    public static Date transformation(Date day)  {
        // TODO
        Date nextDay =
        return nextDay;
    }
    ```
````

Listing 4: An example of output format in the prompt

Listing 4 shows the output format specified in the prompt, which defines the skeleton of the input transformation function to generate. It includes the function name, parameter (i.e., source input) types and names, and type of the return value (i.e., follow-up input) [2]. Following recent studies' nucleus sampling [14, 21], for each MR, MR-Adopt instructs an LLM to generate one input transformation function, and repeats the generation process five times with a temperature setting of 0.2 [6, 8]. Finally, five candidate transformation functions can be generated.

MR-Adopt extracts the generated functions by identifying code blocks wrapped with ``` and extracting the code that matches the given transformation function skeleton. This ensures the generated code conforms to the required format and can be easily integrated into given MR-encoded test cases.

3.2.2 Transformation Refinement. Similar to the situation discussed in Section 3.1.2, LLM-generated transformation functions can contain irrelevant code, some of which can cause errors (e.g., invoking non-existing APIs). To address this issue, MR-Adopt constructs data dependency graphs for follow-up inputs by first identifying statements that define these inputs. It then extracts the variables used in these definitions and recursively traces their dependencies. Statements involved in the dependency graph are considered relevant and retained, while others are considered irrelevant and excluded. As the example shown in Listing 5, the follow-up input `nextDay` depends on `localDate`, which further depends on `day`. Statements constructing `nextDay` and `localDate` are retained, while irrelevant statements such as `Date dayAfter=day.after(1)` are excluded.

````
The transformation function can be implemented as follows:

```java
public static Date transformation(Date day)  {
    Date dayAfter = day.after(1); // non-exisitng API and irrelevant
    LocalDate localdate = LocalDate.parse(day);
    Date nextDay = localdate.plusDays(1).todate();
    return nextDay;
}
```
````

Listing 5: An example of LLM-generated transformation

After excluding irrelevant code, MR-Adopt analyzes and imports dependencies needed by the generated transformation function. Using JavaParser [24], MR-Adopt identifies dependent class names through syntax analysis. It then retrieves potential classes defined or imported in source and imports those whose names match the dependent classes. For example, the internal class `LocalDate` Listing 5 will be imported. Although matching dependent classes by name can be inaccurate if LLM-generated code uses incorrect names or

[2] For MRs with multiple follow-up inputs, the return type is a list of objects.

```
@Test
void testToMediumDate() {
    // new source input
    Date dateA = new Date("2024-02-01 00:00:00");
    // invoking generated transformation on new source input
    Date dateB = transformation(dateA);
    Date mediumA = DateUtils.toMediumDate(dateA);
    Date mediumB = DateUtils.toMediumDate(dateB);
    assertThat(mediumB, is(plusOneDay(mediumA)));
}
```

Listing 6: Validating an LLM-generated input transformation with a new source input

```
public static Date transformation(Date day) {
    int dayValue = day.getDate();
    int monthValue = 1; // set the month to 1 since we don't know
    ↪ which month the input represents
    int yearValue = day.getYear();
    Date nextDay = new Date(year, month, day +1);
    return nextDay;
}
```

Listing 7: An example of LLM-generated transformation

third-party libraries not imported in the project, our experimental observations indicate that this issue is minor. Most dependent classes are derived from the context provided to LLMs, such as the MUT or MTC code.

3.2.3 Transformation Assessment. After refining candidate transformations, MR-Adopt further assesses their quality by applying them to new source inputs. In this step, MR-Adopt leverages new source inputs generated in Section 3.1 to assess the generalizability of candidates and then selects the most generalizable one.

Specifically, MR-Adopt uses new source inputs as test inputs and employs developer-written assertions (i.e., output relation assertions) as test oracles. A transformation is considered applicable to a given source input if (a) the input transformation function can successfully generate a corresponding follow-up input without throwing exceptions, and (b) the outputs from executing the methods under test pass the developer-written assertions. For example, in Listing 6, given the source input `dateA`, if the follow-up input `dateB` is successfully generated and the outputs `mediumA` and `mediumB` pass the assertion (Line 9), MR-Adopt considers the transformation applicable to input `dateA`. Conversely, Listing 7 shows a failing transformation that only works for January dates. MR-Adopt assesses all candidate transformation functions using both new and the original source input. It then selects the most generalizable transformation that applies to the most inputs. In case of a tie, MR-Adopt returns the first generated one as the result.

4 EVALUATION

4.1 Research Questions

Our evaluation aims to answer the following research questions:

- **RQ1:** *How effective is MR-Adopt in generating input transformations?* This RQ compares the quality of the input transformation functions generated by MR-Adopt and baselines to evaluate the effectiveness of MR-Adopt in generating generalizable input transformations for MRs encoded in test cases.
- **RQ2:** *How effective are MR-Adopt-generated input transformations in constructing follow-up inputs, compared with LLMs?* This RQ investigates the benefits of generating input transformation functions, by comparing the quality of follow-up inputs produced by these functions versus those directly generated by LLMs.
- **RQ3:** *What is the contribution of each component in MR-Adopt?* This RQ performs an ablation study to reveal how each component contributes to generating input transformations.
- **RQ4:** *How useful are encoded MRs in enhancing test adequacy with the generated input transformations?* With input transformations generated by MR-Adopt, more encoded MRs can be reused with new inputs to test more behaviors of SUT. This RQ investigates the usefulness of such encoded MRs in improving test adequacy, demonstrating the usefulness of generating input transformation.

4.2 Dataset

MR-encoded test cases (MTCs). We followed Xu et al. [65] to collect high-quality Java projects with at least 200 stars over GitHub. Besides, we further excluded the projects created before 01-April 2023 to prevent the experimental LLMs from having potentially learned the code during training, thereby reducing the potential for data leakage [6]. Finally, we collected 2,007 MTCs from qualified projects. From these MTCs, we retained test cases that (i) can be successfully compiled, (ii) can be successfully executed (i.e., passing developer-written assertions), and (iii) contain MRs associated with exactly two method invocations (one for the source input and one for the follow-up input). The third criterion excludes complex and less common MRs involving multiple input groups [65]. MRs with exactly two invocations constitute the majority (65%) of MTCs [65]. Handling MRs with more than two invocations presents significant challenges in identifying source and follow-up inputs. This problem is nontrivial and remains an important future work. Finally, we obtained 180 MTCs, including 54 with explicit input transformations written by developers and 126 without such transformations, consistent with the distribution reported by Xu et al. [65].

Generation Tasks and Ground Truths. Based on the collected 180 MTCs, we prepared a dataset containing (i) 100 MTCs without input transformations as tasks, and (ii) corresponding input transformation functions as ground truths. The preparation process is as follows. Firstly, we tried to utilize all 54 MTCs with ground truths, i.e., developer-written input transformations. For each MTC, we executed the input transformation on the hardcoded source input to obtain the follow-up input. We prepared a task by replacing the developer-written transformation with the hardcoded follow-up input. Some MTCs whose follow-up input cannot be hardcoded are excluded. For example, an MR for a text render class is "the width of a text (source input) should not be greater than its bold version (follow-up input)". The follow-up input (bold text) can only be generated by a method `bold()`, which is a developer-used transformation program. Finally, we built 36 tasks from 36 MTCs with developer-written transformations.

Next, we manually constructed input transformation functions for MTCs lacking developer-written transformations. Specifically, 64 out of 126 MTCs without input transformations were randomly selected as tasks. For each task, one author of this paper examined the SUT and its underlying MRs and then created a transformation function applicable to the original source input and generalizable to new valid source inputs. Another author reviewed these transformations, and any disagreements were discussed and resolved with consensus. This process took approximately 200 human hours. Details of this dataset can be found on MR-Adopt's website [56].

Table 1: Effectiveness of MR-Adopt in generating input transformations for 100 MRs encoded in test cases

Metric (# Trans.)	Direct Prompting			MR-Adopt		
	Llama3	*Deepseek*	*GPT-3.5*	*Llama3*	*Deepseek*	*GPT-3.5*
compilable	79	80	81	86 (+8.86%)	89 (+11.25%)	**95 (+17.28%)**
>0% generalizable	69	72	69	77 (+11.59%)	82 (+13.89%)	**83 (+20.29%)**
>75% generalizable	64	67	63	74 (+15.66%)	80 (+19.40%)	**81 (+28.57%)**
100% generalizable	57	60	54	68 (+19.30%)	71 (+18.33%)	**72 (+33.33%)**

n% generalizable: the number of generated input transformations applicable to at least *n*% of source inputs.

4.3 Environment and Large Language Models

Our experiments were conducted on machines with three RTX4090 GPUs, dual Intel Xeon E5-2683 v4 CPUs, and 256 GB RAM.

The large language models used in our evaluation include GPT-3.5 from OpenAI [42] and three open-source code models: Llama3-8B [37] from Meta, Deepseek-coder-7b [11] from DeepSeek, and CodeQwen1.5-7B-Chat [1] from Alibaba. We use these LLMs since they are popular state-of-the-art [16] code models in well-known LLM families and deployable at our machines.

4.4 Source Input Preparation

To evaluate the generated input transformations, we need new valid source inputs as a "test set". As introduced in Section 2.1, automatic test input generation techniques (such as Evosuite [17] or Randoop [44]) can be employed to prepare source inputs. However, we found that these tools often fail to generate test inputs for many MRs. This is because over 50% of experimental MRs' inputs are user-defined complex objects with complicated preconditions and environments, which are challenging for tools like Evosuite to handle. This aligns with Xu et al.'s observation [65].

Recent studies show that LLMs are good test input generators [67, 71]. In this study, we employed an LLM (Qwen) to generate new source inputs for evaluating transformations, while other experimental LLMs were used to generate transformations. As a reminder, to mitigate circular evaluation, we employed different LLMs for preparing the "test set" and for generating input pairs and transformation functions in MR-Adopt. We reused the prompt template from MR-Adopt's *Phase1*. Qwen was instructed to generate five source inputs at a time, and the process was repeated ten times with a 0.2 temperature setting to produce more source inputs.

For the 100 experimental MRs, Qwen generated a total of 5,355 new source inputs. We first filtered out 3,058 duplicate inputs using string matching. Next, we identified valid source inputs by executing them on the corresponding ground truth transformations. A source input is considered valid only if the ground truth transformation successfully generates a follow-up input, and the outputs of this source input and corresponding follow-up input pass the developer-written assertions (R_o). Qwen failed to generate a new valid source input for 5 MRs whose inputs are complex objects and have strict domain-specific constraints. Finally, we collected 1,366 valid source inputs, averaging 14.37 per MR.

4.5 RQ1: Effectiveness of MR-Adopt

4.5.1 Experiment Setup. This RQ inspects MR-Adopt's effectiveness in generating input transformation functions by examining their comparability and generalizability to new source inputs.

Baselines. To the best of our knowledge, no existing approach generates input transformation functions for MRs across different domains. Given the proven effectiveness of LLMs in code and test generation, we set *directly prompting LLMs* as a baseline. Specifically, we directly prompted GPT-3.5-turbo-0125, Llama3-8B-Instruct, and Deepseek-coder-7b-instruct-v1.5 (shorten as GPT-3.5, Llama3, and Deepseek, respectively). The template is similar to MR-Adopt's and available at [56]. The knowledge cut-off dates for these models are September 2021 [43], March 2023 [38], and March 2023 [12], respectively, before the creation date of our dataset's MTCs (Section 4.2),

Configuration of Baseline LLMs. Following recent studies [14], we used the nucleus sampling [21] and repeated the generation process five times for each task with a temperature setting of 0.2 [6, 8], and selected the best result for comparison. The configuration of MR-Adopt was introduced in Section 3.2.1.

Metrics. For this RQ, we introduced two metrics: (i) *# compilable transformations:* the number of generated input transformations that can successfully compile, and (ii) *# n% generalizable transformations:* the number of generated input transformations applicable to at least *n*% of source inputs prepared in Section 4.4 ($n = 0, 75, 100$ representing at least one, upper-quartile, and all inputs, respectively). A transformation t is considered *applicable* to a source input x_s if t generates a follow-up input x_f for x_s, so that a *correct* SUT does not violate the output relation on the input pair $<x_s, x_f>$.

4.5.2 Result. As shown in Table 1, MR-Adopt effectively produced many compilable input transformation functions that well generalize to prepared source inputs. We found that MR-Adopt works best with GPT-3.5. Specifically, using GPT-3.5 (the last column), MR-Adopt produced compilable transformations for 95 out of 100 MRs, with 72 of these transformations effectively applied to *all* prepared source inputs. MR-Adopt also works well with Llama3 and Deepseek, generating 68 and 71 (100%) generalizable transformations, respectively. Besides, we found that some generated transformations generalize well to some, but not all, source inputs prepared in our experiment. Specifically, with GPT-3.5, 83 out of 95 compilable transformations applied to at least one source input, and 81 of them applied to more than 75% of the prepared source inputs. Similar results were found with Llama3 and Deepseek. We considered these transformations generated by MR-Adopt *still useful* to some extent, as they successfully prepare some valid input pairs. Upon further analysis, we found that their limitations could potentially be addressed with more comprehensive prompts to handle corner cases. LLM-generated transformations effectively handle common cases but struggle with edge cases. For example, an ideal transformation would generate a higher version string in any scenario (e.g., transforming `"1.0-A1"` to `"1.0-B1"`), but the LLM-generated transformation relies on a 'Major.Minor.Revision' convention (e.g., `"1.0.1"`) and fails with cases like `"1.0-A1"`.

There were 5, 14, and 11 transformations generated by MR-Adopt with GPT-3.5, Llama3, and Deepseek, respectively, that failed to compile. The main reasons include: (i) the generated transformations invoke non-existing methods to generate the follow-up input and (ii) they invoke inaccessible APIs due to permission restrictions (e.g., private methods). Additionally, the compilable but not generalizable transformations were primarily due to *Phase1* failing to

Table 2: Effectiveness of MR-Adopt's transformations in constructing follow-up inputs for 1366 source inputs

MR-Adopt	Llama3	Deepseek	GPT-3.5	Improvement
1246	697	724	597	+72.10%∼**+108.71%**
MR-Adopt	**Llama3$^+$**	**Deepseek$^+$**	**GPT-3.5$^+$**	**Improvement**
1246	770	737	708	+61.82%∼**+75.99%**

$^+$ means incorporating MR-Adopt's input refinement procedure for LLMs' answers.

generate valid input pairs for these MRs, leading to LLM-generated transformations overfitted to the given input pair.

We also compared MR-Adopt's performance (columns 5-7) with the baseline of directly prompting LLMs (columns 2-4). Although the output relation encoded in MTC was provided in prompts for baselines to aid transformation generation, MR-Adopt still generated more compilable transformations. This improvement is due to MR-Adopt's code refinement and assessment strategies. Moreover, MR-Adopt demonstrates substantial improvements in generating transformations that are >75% and 100% generalizable, with increases of 15.66% to 28.57% and 18.33% to 33.33%, respectively. This suggests the effectiveness of preparing more examples for LLMs and the benefits of MR-Adopt's refinement and selection strategies.

Answer to RQ1: MR-Adopt significantly outperforms the baseline LLMs across all metrics. Compared to directly prompting LLMs, MR-Adopt achieves 18.33%∼33.33% improvement in generating 100% generalizable input transformations.

4.6 RQ2: Effectiveness of Input Transformations

4.6.1 Experiment Setup. This RQ examined the quality of follow-up inputs produced by input transformations generated by MR-Adopt. We set LLMs as the baselines because they are off-the-shelf black-box transformations that can generate follow-up inputs given source inputs, as introduced in Section 3.1.1. We also included LLMs enhanced with MR-Adopt's refinement procedure (marked with $^+$) for comparison. This can reflect the effectiveness of MR-Adopt's refinement for input pairs preparation (Section 3.1.2).

Metric. We generated follow-up inputs by feeding the 1,366 prepared source inputs (Section 4.4) to input transformations generated by MR-Adopt and the vanilla LLM baselines. To compare the qualities of the follow-up inputs produced by the MR-Adopt-generated transformations and the baselines, we used the number of *valid* follow-up inputs as the metric. Similar to Section 4.5, we consider a follow-up input x_f *valid* if it and its corresponding source input can pass developer-written output relation assertions.

4.6.2 Result. As shown in Table 2, when built with GPT-3.5, input transformation functions generated by MR-Adopt produced valid follow-up inputs for 1246 out of 1366 (91.22%) source inputs. The high validity rate demonstrated that MR-Adopt contributed to abundant useful source-followup input pairs.

In comparison, three vanilla LLMs only generated valid follow-up inputs for 697 (51.02%), 724 (53.00%), and 597 (43.70%) source inputs, respectively. MR-Adopt surpassed them by 72.10%-108.71%. LLMs enhanced with MR-Adopt's input refinement procedure (marked with $^+$) worked better than the vanilla LLMs. This indicates the usefulness of our design to refine the LLM-generated test inputs (Section 3.1.2). Meanwhile, MR-Adopt's transformations still outperformed the enhanced LLMs by generating 61.82% more valid follow-up inputs than Llama3$^+$, 69.06% more than Deepseek$^+$, and 75.99% more than GPT-3.5$^+$. This significant performance gap highlights the effectiveness of MR-Adopt's transformation functions compared to the state-of-the-art LLMs. It also evidenced the usefulness of our idea to codify the input transformation by leveraging the code understanding and generation abilities through the two-phase pipeline and preparation-refinement-validation process.

We also summarized two major limitations of using vanilla LLMs as black-box transformations based on our observation. Firstly, LLMs can generate a follow-up input with a wrong value, which is similar to the case in Listing 2. Another limitation is that LLMs often fail to capture the constraints between multiple arguments of the follow-up input. For instance, consider a method `deserial(data, size)` to deserialize an `ArrayList data` with a given `size`. The `size` should not be greater than the length of `data`. However, LLMs may miss this constraint and generate invalid value for `size`. These issues about value processing could be due to LLMs' limited inference ability. Instead, MR-Adopt asks LLMs to codify the input transformation and uses the code to do calculation and processing, which is recognized as a better way to exert LLMs' abilities [28]. Besides, we argued that using LLMs as transformations can be costly since we need to request LLMs for each source input. Meanwhile, MR-Adopt uses LLMs to generate transformations for once, and there is no need to query LLMs when using the generated transformations.

Answer to RQ2: MR-Adopt's refinement step can effectively enhance follow-up input generation, with up to 18.59% improvement for GPT-3.5. Additionally, MR-Adopt-generated transformations can effectively generate follow-up inputs for 91.21% source inputs, surpassing GPT-3.5$^+$ by 75.99%.

4.7 RQ3: Ablation Study on MR-Adopt

4.7.1 Experiment Setup. We created three variants v_1, v_2, and v_3 of MR-Adopt by ablating three components to analyze the helpfulness of these designs for generating generalizable input transformations. We chose MR-Adopt built with GPT-3.5 which achieves the best result in RQ1 (Section 4.5). The variants are as follows:

- v_1**: MR-Adopt w/o additional input pairs.** This variant used only one source-followup input pair hard-coded in an MTC to guide the input transformation generation. It did not use additional input pairs generated in MR-Adopt's *Phase1* (Section 3.1).
- v_2**: MR-Adopt w/o refinement step.** This variant disabled the refinement step for generated input transformations in MR-Adopt (Section 3.2.2).
- v_3**: MR-Adopt w/o assessment step.** This variant disabled the assessment step for selecting the most generalizable transformations (Section 3.2.3). Instead, it randomly selected one of the compilable transformation functions as the result.

4.7.2 Result. As shown in Table 3, removing additional input pairs (v_1) led to a 19.44% decrease in generating 100% generalizable transformations. This suggests that additional input pairs effectively mitigate the overfitting problem caused by the limited examples

Table 3: Contribution of each component in MR-Adopt

Metrics (# Trans.)	MR-Adopt	v_1: w/o input pairs	v_2: w/o refinement	v_3: w/o assessment
compilable	95	87 (-8.42%)	93 (-2.10%)	95 (0.00%)
>0% generalizable	83	73 (-12.04%)	82 (-1.20%)	70 (-15.66%)
>75% generalizable	81	66 (-18.51%)	75 (-7.40%)	59 **(-27.16%)**
100% generalizable	72	58 **(-19.44%)**	61 **(-15.27%)**	56 (-22.22%)

in PBE [2, 19, 45], helping MR-Adopt generate more generalizable transformation.

Similarly, disabling the refinement step (v_2) reduced 15.27% input transformations that generalize to 100% prepared inputs. This indicates that some generated transformations have minor issues and can be refined by excluding irrelevant code. Besides, disabling the assessment step (v_3) decreased 22.22% input transformation generalizable to 100% inputs. This suggests that, even with additional input pairs and refinement, few 100% generalizable transformations can be generated, and random selection may miss them. The assessment step is necessary to rank the most generalizable function.

Answer to RQ3: All three designs contribute to the effectiveness of MR-Adopt in generating generalizable transformations. The assessment procedure contributes the most, and additional example input pairs contribute similarly.

4.8 RQ4: Usefulness of Input Transformations

4.8.1 Experiment Setup. In this RQ, we integrated the generated input transformations into MTCs to construct generalized MRs and measured how well such MRs enhanced test adequacy. This demonstrated the practical usefulness of MR-Adopt's transformations in enhancing test adequacy.

New Test Cases Construction. We applied generalized MRs to the automatically generated source inputs introduced in Section 4.4 to obtain a set of new test cases (denoted as $\mathcal{M}$). We compare such test cases against two baselines: (i) the developer-written test cases (i.e., MTCs) (denoted as $\mathcal{D}$) and (ii) test cases based on the LLM-generated source and follow-up input pairs (denoted as $\mathcal{L}$). Specifically, we combined the prepared source inputs (Section 4.4) with valid follow-up inputs generated by Llama3$^+$ which performed the best in RQ2 (Section 4.6). Considering generalized MR based test cases and LLM-generated input pairs based test cases are extended from developer-written existing test cases, we followed Xu et al. [65]'s practice to analyze the test adequacy improvement on top of developer-written test cases.

Metrics. We measured test adequacy using two metrics: (i) Line Coverage – percentage of code lines in target classes executed, and (ii) Mutation Score – percentage of mutants killed by test cases.

Mutation Testing: We employed Pitest [46] to conduct mutation testing. Each MR only focused on one or two methods under test in the target class. To include the covered lines or killed mutants in the methods intransitively invoked by MR-involved methods for comparison, we employed Pitest to generate mutants targeting all methods in a target class. Finally, Pitest successfully generated 4,388 mutants for 45 target classes covered by 88 MRs in the dataset (Section 4.2). Pitest failed for the other 12 MRs' classes because of environmental issues (e.g., conflict dependencies).

Table 4: Enhancement of test adequacy from generalized MR based test cases ($\mathcal{M}$) on top of developer-written ($\mathcal{D}$) and LLM-generated input pairs ($\mathcal{L}$) based test cases

Metrics	VS. $\mathcal{D}$			VS. $\mathcal{D}$+$\mathcal{L}$		
	$\mathcal{D}$	$\mathcal{D}$+$\mathcal{M}$	Improve.	$\mathcal{D}$+$\mathcal{L}$	$\mathcal{D}$+$\mathcal{L}$+$\mathcal{M}$	Improve.
Line Coverage	0.2373	0.2625	+10.62%	0.2588	0.2698	+4.25%
Mutation Score	0.1322	0.1572	+18.91%	0.1710	0.1807	+5.67%

4.8.2 Result. As shown in Table 4, compared to developer-written MTCs ($\mathcal{D}$), incorporating new test cases constructed from generalized MR ($\mathcal{D}$+$\mathcal{M}$) increased the line coverage by 10.62% and the mutation score by 18.91%. This suggested that MR-Adopt could enhance the test adequacy by integrating high-quality test oracles (i.e., output relation of the encoded MR) with a diverse set of potential test input pairs of the MR ($\mathcal{M}$). Although the developer-written test inputs hard-coded in MTCs were carefully crafted and invaluable, each typically included one pair of test inputs and could not sufficiently exercise the SUT's behaviors. The new source inputs generated by test generation techniques and the corresponding follow-up inputs enabled by MR-Adopt may reach program states not covered by the hard-coded inputs.

Besides, by analyzing the benefit of using MR-Adopt ($\mathcal{D}$+$\mathcal{L}$+$\mathcal{M}$) over the test suite enhanced by LLM-generated valid input pairs ($\mathcal{D}$+$\mathcal{L}$), we could still observe 4.25% and 5.67% improvements in the line coverage and the mutation score, respectively. This suggested that even if an LLM could act as a black-box transformation to generate some valid source-followup inputs and reach more execution states of SUT, MR-Adopt could generate input transformations that apply to more source inputs and better enhance the test adequacy.

Answer to RQ4: Test cases constructed from generalized MRs could achieve 10.62% and 18.91% increases in the line coverage and mutation score, respectively, demonstrating generalized MRs' practical usefulness in enhancing test adequacy.

5 DISCUSSION

5.1 Threads to Validity

We identified potential threats to the validity of our experiments and have taken measures to mitigate them.

Representativeness of Experimental Subjects. A potential threat is whether our evaluation findings can generalize to different projects. To mitigate this threat, we adopted the criteria from existing studies [22, 58, 65] to select high-quality and well-maintained Java projects as representative subjects (Section 4.2) and evaluated our method on these projects. Besides, evaluating LLMs with subjects seen during model training (known as the data leakage issue) will make the findings biased [61]. To mitigate this threat, we collected MR-encoded test cases created after the training cut-off date of the experimental LLMs, as described in Section 4.2.

Representativeness of Experimental LLMs. Our method depends on LLMs, and we also use LLMs as baselines. A potential threat is whether our evaluation findings based on the selected LLMs are representative. To mitigate this threat, we evaluated our method with LLMs from three well-known LLM families, i.e., GPT-3.5 from OpenAI, Llama3 from Meta, and Deepseek from DeepSeek. They represent the state-of-the-art code LLMs (according to the

EvalPlus leaderborad) that can be deployed with the hardware capacity of our machine, as introduced in Section 4.3.

Quality of the Experimental Source Inputs. As introduced in Section 4.4, we used an LLM to prepare new source inputs to assess the generalizability of generated input transformations. Low-quality source inputs may threaten the evaluation validity. To mitigate this issue, we employed another SOTA code LLM (i.e., Qwen) which is not the experimental subject to prepare the source inputs. We then use the ground truth input transformations to filter out invalid source inputs.

Quality of Ground Truths. Besides directly using developer-written input transformations in MTCs (if available) as ground truths, we also manually prepared ground truths for MTCs without input transformations. There is a potential threat regarding the quality of our prepared ground truths. To mitigate this threat, two authors (PhD students) proficient at MT and with more than four years of Java programming experience implemented the ground truths after understanding the intention of the SUTs and the encoded MRs. Specifically, a ground truth was developed by one participant and reviewed by the other until a consensus was reached. Furthermore, the developed ground truths are validated against the original source input.

5.2 Distinct Advantages of MR-based Tests in Fault Detection

Detecting faults in "non-testable" programs. MR-based tests offer distinct advantages in validating *non-testable programs* whose expected outputs for given inputs are hard to specify [10, 47]. The usefulness of MR-based tests in detecting such faults for such programs has been reported in studies [9, 32, 34, 48]. MR-Adopt targets encoded MRs for testing Java classes. We provide two examples to illustrate this advantage.

As shown in Listing 8, the class `AES` contains methods to encrypt and decrypt a string. The `encrypt` function includes a fault: it mistakenly encrypts the `secret` argument instead of the intended `source`. However, the expected string literal after encryption is difficult to specify. This makes it difficult to construct an explicit test oracle based on the expected output to effectively validate the behavior of `encrypt`.

We collected developer-written tests, EvoSuite-generated tests (following Xu et al.'s practice [65]), and LLM-generated tests (following Yuan et al.'s practice and generating tests with GPT-4 [70]). As shown in Listing 9, we found that the non-MR test (including developer-written and LLM-generated assertions) only checks that the encrypted string is not null or empty. EvoSuite failed to generate any assertions. However, this test is weak in validating whether the encryption process is correctly implemented. It ensures only the existence of a non-empty output.

In contrast, the developer-written MR-based test validates the encrypted string using an MR: $x = AES.decrypt(AES.encrypt(x))$ — *IF* an input x is encrypted and subsequently decrypted, *THEN* the final result should be x. The MR-based test successfully detects the fault in encrypting the `secret` argument instead of the `source`, while the non-MR tests fail to detect the fault.

[3]Available in the CipherHelper.java file from the project FlowCI/flow-core-x

```java
public static abstract class AES {
    public static String encrypt(String source, String secret) {
        ...
        cipher.init(Cipher.ENCRYPT_MODE, key);
        byte[] bytes = cipher.doFinal(toBytes(secret)); // BUG: This
        ↪  should be "toBytes(source)" instead of "toBytes(secret)"
        return Base64.getEncoder().encodeToString(bytes);
    }
    public static String decrypt(String encrypted, String secret) {
    ...
}
```

Listing 8: Faulty class `AES`[3] for encrypting and decrypting

```java
public void AES_NonMRTest() {
  String source = "!@#!@#!@1fsd"; String secret = "ssdkF$HUy2A#D%kd";
  String encrypted = CipherHelper.AES.encrypt(source, secret);
  // Developer-written non-MR assertion
  assertFalse(Strings.isNullOrEmpty(encrypted));

  // LLM-generated non-MR assertion
  assertNotNull(encrypted); assertFalse(encrypted.isEmpty());

  // EvoSuite-generated non-MR test: no assertion
  String string0 = CipherHelper.AES.encrypt("jlyN5n~l(%", "AES");
  String string1 = CipherHelper.AES.decrypt("n<h!/N*WHF", ")");
}

//Developer-written MR-based test, MR: x=AES.decrypt(AES.encrypt(x))
public void AES_MRTest() {
  String source = "!@#!@#!@1fsd"; String secret = "ssdkF$HUy2A#D%kd";
  String encrypted = CipherHelper.AES.encrypt(source, secret);
  String decrypted = CipherHelper.AES.decrypt(encrypted, secret);
  assertEquals(source, decrypted);
}
```

Listing 9: Non-MR and MR based tests[4] for class `AES`

Similarly, when testing the `getRegistryCenterTime`[5] function, which is designed to (i) create a new registry entry and return the creation time, or (ii) update an existing entry and return the updating time, it is difficult to determine the expected output because the exact system time depends on when the code is executed.

The non-MR test, including developer-written and LLM-generated assertions, weakly validates if the registration time is greater than 0 and less than the current system time. EvoSuite failed to generate any assertions. Such a weak non-MR test has a significant limitation: it fails to validate whether the function correctly handles the update of existing entries. In other words, it does not explicitly check the chronological order of the successive calls.

In contrast, a developer-written MR-based test validates the registration time based on an MR: *IF* $t_2 = getRegistryCenterTime(key)$ is called after $t_1 = getRegistryCenterTime(key)$, *THEN* $t_1 < t_2$. When these tests are applied to a faulty `getRegistryCenterTime` implementation that returns an unmodified time due to a missing entry update, only the MR-based test can detect this fault, while non-MR tests cannot.

Exercising a wide range of inputs even with a single MR. Another advantage of MT is that *one MR can be applied to a wide range of automatically generated test inputs (known as source inputs) to exercise various program behaviors.*

Our experimental results show that useful tests can be constructed from MRs by leveraging MR-Adopt's input transformations and the wide range of generated test inputs. The constructed

[4]Available in the CipherHelperTest.java file from the project FlowCI/flow-core-x

[5]Available in the `ZookeeperRegistryCenter` class within the project apache/shardingsphere-elasticjob

[6]Available in the ZookeeperRegistryCenterQueryWithoutCacheTest.java file from the project apache/shardingsphere-elasticjob

```java
public void GetRegistryCenterTime_nonMRTest() {
  String key = "/_systemTime/current";
  long regCenterTime = zkRegCenter.getRegistryCenterTime(key);
  // Developer-written non-MR assertion
  assertTrue(regCenterTime<=System.currentTimeMillis());
  // LLM-generated non-MR assertion
  assertTrue(regCenterTime > 0L);
}

//Developer-written MR-based test: IF t2=getRegistryCenterTime(key)
↪  is called after t1=getRegistryCenterTime(key), THEN t1<t2
public void GetRegistryCenterTime_MRTest() {
  String key = "/_systemTime/current";
  long regCenterTime = zkRegCenter.getRegistryCenterTime(key);
  long updatedRegCenterTime = zkRegCenter.getRegistryCenterTime(key);
  assertTrue(regCenterTime < updatedRegCenterTime);
}
```

Listing 10: MR-based and non-MR-based tests[6] for `ZookeeperRegistryCenter`

tests help improve test adequacy with an increase of 10.62% and 18.91% in line coverage and mutation score, respectively. The results are in line with that reported by Xu et al.'s study [65], which observes an increase of 52.10% and 82.80% in line coverage and mutation score, respectively, by MR-based tests over EvoSuite-generated tests. Even compared with the combination of developer-written and EvoSuite-generated tests, MR-based new tests exclusively covered 113 (+6.95%) mutants and killed 88 (+7.93%) mutants. This is because, although EvoSuite can generate many inputs, it fails to generate effective test oracles that can identify incorrect outputs and solve the setups to trigger target programs. In contrast, MR-based tests combine high-quality oracles with diverse inputs and leverage developer-written setups for triggering target programs, resulting in higher test adequacy.

6 RELATED WORK

6.1 Automated Identification of MRs.

Identification of proper MRs is a key step in applying MT to specific SUTs. To efficiently identify MRs, many automated approaches have been proposed. Earlier approaches identify MRs based on a set of predefined patterns [49, 75]. Zhang et al. [73] and Zhang et al. [72] proposed search-based approaches to inferring MRs. Tsigkanos et al. [57] proposed to use LLMs to identify variable relation and input transformation in scientific software. These approaches mainly synthesize MRs for specific domains. Shin et al. [51] proposed an approach to generating executable MRs from requirements specifications using LLMs, but it still requires human effort to implement supportive functions. Ayerdi et al. [3, 4] extend Terragni et al.'s work [55] to generate MRs via genetic programming. Nolasco et al. [41] proposed MemoRIA to infer equivalence MRs between methods and method sequences. Recently, Xu et al. [65] explored a new source to automatically derive MRs. They synthesize MRs from existing test cases where domain knowledge is embedded. This served as an effective approach to reusing many encoded MRs. Such encoded MRs are prevalent, but over 70% lack an input transformation function to support reusing them on more source inputs.

To reuse these invaluable MRs, in this paper, we propose MR-Adopt to generate input transformation functions for such MRs. Integrated with the input transformations, these MRs are found helpful in enhancing test adequacy in our evaluation.

6.2 LLMs for Test Generation.

Researchers explored various LLM usages for test generation. Yuan et al. [71] studied the performance and limitations of ChatGPT in unit test generation. Xia et al. [62] built a fuzzer using LLMs as a generator of realistic test inputs and an engine for mutation. Tang et al. [54] compared the effectiveness of ChatGPT and Evosuite in unit test generation. Lemieux et al. [27] and Yang et al. [66] tried to promote the coverage of the tests generated by LLMs.

Different from these works, MR-Adopt does not use LLMs to generate tests directly. Instead, it generates the input transformation for the encoded MRs and reuses such MRs to enable more tests. In fact, using LLMs to generate correct and effective oracles and produce a large number of tests is found challenging [71]. In comparison, we reuse the human-written oracles in the encoded MRs, which are generally more reliable than LLM-generated oracles. Besides, MRs can be integrated with test input generation tools to produce abundant tests.

6.3 Enhancing LLMs for Code Generation.

LLMs are found powerful in code generation [28, 30], attracting numerous efforts to enhance the coding ability further. Some researchers designed more effective strategies of pre-training [20, 29, 33] and fine-tuning [13, 50]. Researchers also prompted LLMs with compilation messages to guide them to revise the generated code [25, 40, 71] or built a coding agent [74] to enhance LLM's code generation ability. In light of prompting with analogical reasoning [68], our work guides LLMs to generate more examples, identify the intention, and finally generate an input transformation matching the intention. Also, different from the approaches that rely purely on LLMs, we enhance the generated input transformation's quality by performing data-flow analysis to exclude irrelevant code segments from LLMs' responses and ranking the generated transformation functions based on validation with the output relation.

7 CONCLUSION

This paper presents MR-Adopt, an LLM-based approach to generate input transformations for MRs encoded in test cases that lack explicit input relations. MR-Adopt allows these encoded MRs to be reused with new source inputs, enabling the generation of new tests and achieving higher test adequacy.

Experimental results show that MR-Adopt can generate effective input transformations, where 72% input transformations are generalizable to all prepared source inputs. When integrated with these transformations and new test inputs, encoded MRs increase line coverage by 10.62% and mutation score by 18.91%, demonstrating the practical usefulness of MR-Adopt 's transformations in enhancing test adequacy.

8 DATA AVAILABILITY

We have released the code and data at https://mr-adopt.github.io/.

ACKNOWLEDGMENTS

This work is supported by the National Science Foundation of China (Grant No. 61932021), and the Hong Kong Research Grant Council/General Research Fund (Grant No. 16207120).

REFERENCES

[1] Alibaba. 2024. *codeqwen1.5.* Retrieved June 6, 2024 from https://qwenlm.github.io/blog/codeqwen1.5/

[2] Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. 2013. Syntax-guided synthesis. In *Formal Methods in Computer-Aided Design*. IEEE, 1–8.

[3] Jon Ayerdi, Valerio Terragni, Aitor Arrieta, Paolo Tonella, Goiuria Sagardui, and Maite Arratibel. 2021. Generating metamorphic relations for cyber-physical systems with genetic programming: an industrial case study. In *Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2021)*. 1264–1274.

[4] Jon Ayerdi, Valerio Terragni, Gunel Jahangirova, Aitor Arrieta, and Paolo Tonella. 2024. GenMorph: Automatically Generating Metamorphic Relations via Genetic Programming. *IEEE Transactions on Software Engineering* (2024), 1–12.

[5] Jialun Cao, Meiziniu Li, Yeting Li, Ming Wen, Shing-Chi Cheung, and Haiming Chen. 2022. SemMT: A Semantic-Based Testing Approach for Machine Translation Systems. *ACM Transactions on Software Engineering and Methodology* 31, 2 (2022), 34e:1–34e:36.

[6] Jialun Cao, Wuqi Zhang, and Shing-Chi Cheung. 2024. Concerned with Data Contamination? Assessing Countermeasures in Code Language Model. *CoRR* abs/2403.16898 (2024). arXiv:2403.16898

[7] Junkai Chen, Xing Hu, Zhenhao Li, Cuiyun Gao, Xin Xia, and David Lo. 2024. Code Search is All You Need? Improving Code Suggestions with Code Search. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. Association for Computing Machinery, New York, NY, USA, Article 73, 13 pages.

[8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, and et al. 2021. Evaluating Large Language Models Trained on Code. *CoRR* abs/2107.03374 (2021). arXiv:2107.03374

[9] Songqiang Chen, Shuo Jin, and Xiaoyuan Xie. 2021. Testing Your Question Answering Software via Asking Recursively. In *International Conference on Automated Software Engineering*. IEEE, 104–116.

[10] Tsong Yueh Chen, Fei-Ching Kuo, Huai Liu, Pak-Lok Poon, Dave Towey, T. H. Tse, and Zhi Quan Zhou. 2018. Metamorphic Testing: A Review of Challenges and Opportunities. *Comput. Surveys* 51, 1 (2018), 4:1–4:27.

[11] DeepSeek. 2024. *deepseek-coder-7b-instruct-v1.5.* Retrieved June 6, 2024 from https://huggingface.co/deepseek-ai/deepseek-coder-7b-instruct-v1.5

[12] DeepSeek. 2024. *DeepSeek-Coder Updates.* Retrieved June 6, 2024 from https://github.com/deepseek-ai/DeepSeek-Coder/issues/89

[13] Shihan Dou, Yan Liu, Haoxiang Jia, Limao Xiong, Enyu Zhou, Wei Shen, Junjie Shan, Caishuang Huang, Xiao Wang, Xiaoran Fan, Zhiheng Xi, Yuhao Zhou, Tao Ji, Rui Zheng, Qi Zhang, Xuanjing Huang, and Tao Gui. 2024. StepCoder: Improve Code Generation with Reinforcement Learning from Compiler Feedback. *CoRR* abs/2402.01391 (2024). arXiv:2402.01391

[14] Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2024. Evaluating Large Language Models in Class-Level Code Generation. In *nternational Conference on Software Engineering*. ACM, 81:1–81:13.

[15] Aryaz Eghbali and Michael Pradel. 2024. De-Hallucinator: Iterative Grounding for LLM-Based Code Completion. *CoRR* abs/2401.01701 (2024). arXiv:2401.01701

[16] Evalplus. 2024. *leaderboard.* Retrieved June 6, 2024 from https://evalplus.github.io/leaderboard.html

[17] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In *Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ACM, 416–419.

[18] Mingyang Geng, Shangwen Wang, Dezun Dong, Haotian Wang, Ge Li, Zhi Jin, Xiaoguang Mao, and Xiangke Liao. 2024. Large Language Models are Few-Shot Summarizers: Multi-Intent Comment Generation via In-Context Learning. In *International Conference on Software Engineering*. ACM, 39:1–39:13.

[19] Sumit Gulwani. 2011. Automating string processing in spreadsheets using input-output examples. In *Symposium on Principles of Programming Languages, 2011*. ACM, 317–330.

[20] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming - The Rise of Code Intelligence. *CoRR* abs/2401.14196 (2024). arXiv:2401.14196

[21] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. The Curious Case of Neural Text Degeneration. In *International Conference on Learning Representations*. OpenReview.net.

[22] Kaifeng Huang, Bihuan Chen, Congying Xu, Ying Wang, Bowen Shi, Xin Peng, Yijian Wu, and Yang Liu. 2022. Characterizing usages, updates and risks of third-party libraries in Java projects. *Empirical Software Engineering* 27, 4 (2022), 90.

[23] Maliheh Izadi, Jonathan Katzy, Tim van Dam, Marc Otten, Razvan Mihai Popescu, and Arie van Deursen. 2024. Language Models for Code Completion: A Practical Evaluation. In *International Conference on Software Engineering*. ACM, 79:1–79:13.

[24] JavaParser. 2024. *JavaParser.* Retrieved June 6, 2024 from https://javaparser.org/

[25] Shuyang Jiang, Yuhao Wang, and Yu Wang. 2023. SelfEvolve: A Code Evolution Framework via Large Language Models. *CoRR* abs/2306.02907 (2023). arXiv:2306.02907

[26] Vu Le, Mehrdad Afshari, and Zhendong Su. 2014. Compiler validation via equivalence modulo inputs. In *Conference on Programming Language Design and Implementation*. ACM, 216–226.

[27] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *International Conference on Software Engineering*. IEEE, 919–931.

[28] Chengshu Li, Jacky Liang, Andy Zeng, Xinyun Chen, Karol Hausman, Dorsa Sadigh, Sergey Levine, Li Fei-Fei, Fei Xia, and Brian Ichter. 2023. Chain of Code: Reasoning with a Language Model-Augmented Code Emulator. *CoRR* abs/2312.04474 (2023). arXiv:2312.04474

[29] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, and et al. 2023. StarCoder: may the source be with you! *CoRR* abs/2305.06161 (2023). arXiv:2305.06161

[30] Yujia Li, David H. Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, and et al. 2022. Competition-Level Code Generation with AlphaCode. *CoRR* abs/2203.07814 (2022). arXiv:2203.07814

[31] Mikael Lindvall, Dharmalingam Ganesan, Ragnar Ardal, and Robert E. Wiegand. 2015. Metamorphic Model-Based Testing Applied on NASA DAT - An Experience Report. In *International Conference on Software Engineering*. IEEE Computer Society, 129–138.

[32] Huai Liu, Fei-Ching Kuo, Dave Towey, and Tsong Yueh Chen. 2014. How Effectively Does Metamorphic Testing Alleviate the Oracle Problem? *IEEE Transactions on Software Engineering* 40, 1 (2014), 4–22.

[33] Ziyang Luo, Can Xu, Pu Zhao, Qingfeng Sun, Xiubo Geng, Wenxiang Hu, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. WizardCoder: Empowering Code Large Language Models with Evol-Instruct. *CoRR* abs/2306.08568 (2023). arXiv:2306.08568

[34] Haoyang Ma, Qingchao Shen, Yongqiang Tian, Junjie Chen, and Shing-Chi Cheung. 2023. Fuzzing Deep Learning Compilers with HirGen. In *International Symposium on Software Testing and Analysis*. ACM, 248–260.

[35] Lipeng Ma, Weidong Yang, Bo Xu, Sihang Jiang, Ben Fei, Jiaqing Liang, Mingjie Zhou, and Yanghua Xiao. 2024. KnowLog: Knowledge Enhanced Pre-trained Language Model for Log Understanding. In *International Conference on Software Engineering*. ACM, 32:1–32:13.

[36] Qiuyang Mang, Aoyang Fang, Boxi Yu, Hanfei Chen, and Pinjia He. 2024. Testing Graph Database Systems via Equivalent Query Rewriting. In *International Conference on Software Engineering*. ACM, 143:1–143:12.

[37] Meta. 2024. *Llama-3-8B-Instruct.* Retrieved June 6, 2024 from https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct

[38] Meta. 2024. *Meta-Llama-3 Updates.* Retrieved June 6, 2024 from https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct

[39] Daye Nam, Andrew Macvean, Vincent J. Hellendoorn, Bogdan Vasilescu, and Brad A. Myers. 2024. Using an LLM to Help With Code Understanding. In *International Conference on Software Engineering*. ACM, 97:1–97:13.

[40] Ansong Ni, Srini Iyer, Dragomir Radev, Veselin Stoyanov, Wen-Tau Yih, Sida I. Wang, and Xi Victoria Lin. 2023. LEVER: Learning to Verify Language-to-Code Generation with Execution. In *International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 26106–26128.

[41] Agustín Nolasco, Facundo Molina, Renzo Degiovanni, Alessandra Gorla, Diego Garbervetsky, Mike Papadakis, Sebastián Uchitel, Nazareno Aguirre, and Marcelo F. Frias. 2024. Abstraction-Aware Inference of Metamorphic Relations. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 450–472.

[42] OpenAI. 2024. *GPT-3.5.* Retrieved June 6, 2024 from https://platform.openai.com/docs/models/

[43] OpenAI. 2024. *GPT-3.5 Turbo Updates.* Retrieved June 6, 2024 from https://help.openai.com/en/articles/8555514-gpt-3-5-turbo-updates

[44] Carlos Pacheco and Michael D. Ernst. 2007. Randoop: feedback-directed random testing for Java. In *Conference on Object-Oriented Programming, Systems, Languages, and Applications*. ACM, 815–816.

[45] Rangeet Pan, Vu Le, Nachiappan Nagappan, Sumit Gulwani, Shuvendu K. Lahiri, and Mike Kaufman. 2021. Can Program Synthesis be Used to Learn Merge Conflict Resolutions? An Empirical Analysis. In *IEEE/ACM International Conference on Software Engineering*. IEEE, 785–796.

[46] Pitest. 2024. *Pitest.* Retrieved June 6, 2024 from https://pitest.org/

[47] Sergio Segura, Gordon Fraser, Ana Belén Sánchez, and Antonio Ruiz Cortés. 2016. A Survey on Metamorphic Testing. *IEEE Transactions on Software Engineering* 42, 9 (2016), 805–824.

[48] Sergio Segura, José Antonio Parejo, Javier Troya, and Antonio Ruiz Cortés. 2018. Metamorphic Testing of RESTful Web APIs. *IEEE Transactions on Software Engineering* 44, 11 (2018), 1083–1099.

[49] Sergio Segura, José Antonio Parejo, Javier Troya, and Antonio Ruiz Cortés. 2018. Metamorphic testing of RESTful web APIs. In *International Conference on Software Engineering*. ACM, 882.

[50] Bo Shen, Jiaxin Zhang, Taihong Chen, Daoguang Zan, Bing Geng, An Fu, Muhan Zeng, Ailun Yu, Jichuan Ji, Jingyang Zhao, Yuenan Guo, and Qianxiang Wang. 2023. PanGu-Coder2: Boosting Large Language Models for Code with Ranking Feedback. *CoRR* abs/2307.14936 (2023). arXiv:2307.14936

[51] Seung Yeob Shin, Fabrizio Pastore, Domenico Bianculli, and Alexandra Baicoianu. 2024. Towards Generating Executable Metamorphic Relations Using Large Language Models. *CoRR* abs/2401.17019 (2024). arXiv:2401.17019

[52] Chengnian Sun, Vu Le, and Zhendong Su. 2016. Finding compiler bugs via live code mutation. In *International Conference on Object-Oriented Programming, Systems, Languages, and Applications,*. ACM, 849–863.

[53] Chang-Ai Sun, Yiqiang Liu, Zuoyi Wang, and W. K. Chan. 2016. μMT: a data mutation directed metamorphic relation acquisition methodology. In *International Workshop on Metamorphic Testing*. ACM, 12–18.

[54] Yutian Tang, Zhijie Liu, Zhichao Zhou, and Xiapu Luo. 2024. ChatGPT vs SBST: A Comparative Assessment of Unit Test Suite Generation. *IEEE Transactions on Software Engineering* (2024), 1–19.

[55] Valerio Terragni, Gunel Jahangirova, Paolo Tonella, and Mauro Pezzè. 2020. Evolutionary Improvement of Assertion Oracles. In *Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1178–1189.

[56] MR-Adopt. 2024. *MR-Adopt*. Retrieved June 6, 2024 from https://mr-adopt.github.io/

[57] Christos Tsigkanos, Pooja Rani, Sebastian Müller, and Timo Kehrer. 2023. Variable Discovery with Large Language Models for Metamorphic Testing of Scientific Software. In *Computational Science - ICCS 2023 - 23rd International Conference, Prague, Czech Republic, July 3-5, 2023, Proceedings, Part I (Lecture Notes in Computer Science, Vol. 14073)*. Springer, 321–335.

[58] Ying Wang, Bihuan Chen, Kaifeng Huang, Bowen Shi, Congying Xu, Xin Peng, Yijian Wu, and Yang Liu. 2020. An Empirical Study of Usages, Updates and Risks of Third-Party Libraries in Java Projects. In *International Conference on Software Maintenance and Evolution*. IEEE, 35–45.

[59] Taylor Webb, Keith J Holyoak, and Hongjing Lu. 2023. Emergent analogical reasoning in large language models. *Nature Human Behaviour* 7, 9 (2023), 1526–1541.

[60] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Conference on Neural Information Processing Systems*.

[61] Yi Wu, Nan Jiang, Hung Viet Pham, Thibaud Lutellier, Jordan Davis, Lin Tan, Petr Babkin, and Sameena Shah. 2023. How Effective Are Neural Networks for Fixing Security Vulnerabilities. In *International Symposium on Software Testing and Analysis*. ACM, 1282–1294.

[62] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. 2024. Fuzz4All: Universal Fuzzing with Large Language Models. In *International Conference on Software Engineering*. ACM, 126:1–126:13.

[63] Xiaoyuan Xie, Shuo Jin, and Songqiang Chen. 2023. qaAskeR$^+$: a novel testing method for question answering software via asking recursive questions. *Automated Software Engineering* 30, 1 (2023), 14.

[64] Xiaoyuan Xie, Shuo Jin, Songqiang Chen, and Shing-Chi Cheung. 2024. Word Closure-Based Metamorphic Testing for Machine Translation. *ACM Transactions on Software Engineering and Methodology* (jul 2024).

[65] Congying Xu, Valerio Terragni, Hengcheng Zhu, Jiarong Wu, and Shing-Chi Cheung. 2024. MR-Scout: Automated Synthesis of Metamorphic Relations from Existing Test Cases. *ACM Transactions on Software Engineering and Methodology* 33, 6 (2024), 150.

[66] Chen Yang, Junjie Chen, Bin Lin, Jianyi Zhou, and Ziqi Wang. 2024. Enhancing LLM-based Test Generation for Hard-to-Cover Branches via Program Analysis. *CoRR* abs/2404.04966 (2024). arXiv:2404.04966

[67] Zhen Yang, Fang Liu, Zhongxing Yu, Jacky Wai Keung, Jia Li, Shuo Liu, Yifan Hong, Xiaoxue Ma, Zhi Jin, and Ge Li. 2024. Exploring and Unleashing the Power of Large Language Models in Automated Code Translation. *CoRR* abs/2404.14646 (2024). arXiv:2404.14646

[68] Michihiro Yasunaga, Xinyun Chen, Yujia Li, Panupong Pasupat, Jure Leskovec, Percy Liang, Ed H. Chi, and Denny Zhou. 2023. Large Language Models as Analogical Reasoners. *CoRR* abs/2310.01714 (2023). arXiv:2310.01714

[69] Yuanyuan Yuan, Shuai Wang, Mingyue Jiang, and Tsong Yueh Chen. 2021. Perception Matters: Detecting Perception Failures of VQA Models Using Metamorphic Testing. In *Conference on Computer Vision and Pattern Recognition*. Computer Vision Foundation / IEEE, 16908–16917.

[70] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and improving chatgpt for unit test generation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1703–1726.

[71] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2023. No More Manual Tests? Evaluating and Improving ChatGPT for Unit Test Generation. *CoRR* abs/2305.04207 (2023). arXiv:2305.04207

[72] Bo Zhang, Hongyu Zhang, Junjie Chen, Dan Hao, and Pablo Moscato. 2019. Automatic Discovery and Cleansing of Numerical Metamorphic Relations. In *IEEE International Conference on Software Maintenance and Evolution*. IEEE, 235–245.

[73] Jie Zhang, Junjie Chen, Dan Hao, Yingfei Xiong, Bing Xie, Lu Zhang, and Hong Mei. 2014. Search-based inference of polynomial metamorphic relations. In *ACM/IEEE International Conference on Automated Software Engineering*. ACM, 701–712.

[74] Kechi Zhang, Jia Li, Ge Li, Xianjie Shi, and Zhi Jin. 2024. CodeAgent: Enhancing Code Generation with Tool-Integrated Agent Systems for Real-World Repo-level Coding Challenges. *CoRR* abs/2401.07339 (2024). arXiv:2401.07339

[75] Zhi Quan Zhou, Liqun Sun, Tsong Yueh Chen, and Dave Towey. 2020. Metamorphic Relations for Enhancing System Understanding and Use. *IEEE Transactions on Software Engineering* 46, 10 (2020), 1120–1154.